

**Towards Bridging the Gap between
Knowledge Graphs and Real-World Facts**

Kim, Donghyun

**Department of Computer Science
Graduate School
Yonsei University**

**Towards Bridging the Gap between
Knowledge Graphs and Real-World Facts**

Advisor : Lee, Kyong-Ho

**A Dissertation Submitted
to the Department of Computer Science
and the Committee of the Graduate School
of Yonsei University in Partial Fulfillment of the
Requirements for the Degree of
Doctor of Philosophy**

Kim, Donghyun

August 2026

**Towards Bridging the Gap between
Knowledge Graphs and Real-World Facts**

**This certifies that the dissertation
of Kim, Donghyun is approved.**

Committee Chair	_____ Lee, Kyong-Ho
------------------------	-------------------------------

Committee Member	_____ Han, Yo-Sub
-------------------------	-----------------------------

Committee Member	_____ Yeo, Jinyoung
-------------------------	-------------------------------

Committee Member	_____ Lee, Dongha
-------------------------	-----------------------------

Committee Member	_____ Oh, Byungkook
-------------------------	-------------------------------

**Department of Computer Science
Graduate School
Yonsei University
August 2026**

Acknowledgments

First and foremost, I would like to express my deepest gratitude to my advisor, Prof. Kyong-Ho Lee. It has been a true honor and privilege to pursue my Ph.D. under his guidance. His perspective on research and life has profoundly shaped my own, and I will carry the lessons I learned from him throughout my future career. I am also sincerely grateful to my thesis committee members, Prof. Yo-Sub Han, Prof. Jinyoung Yeo, Prof. Dongha Lee, and Prof. Byungkook Oh, for their insightful feedback, thoughtful advice, and warm encouragement. Their constructive comments greatly contributed to improving and completing this dissertation. My years in the Internet Computing Lab. have been a truly meaningful part of my life. I thank all the lab members for the valuable discussions, shared experiences, and memorable moments along the way. In particular, I owe a special debt of gratitude to Hyeongjun Yang. Without his support, encouragement, and the struggles we shared, this journey would have been far more difficult. I am truly grateful that we were able to walk this path together. Finally, I am deeply thankful to my parents and my brother for their unconditional love and unwavering support. Their trust in me has always been my strongest foundation. This dissertation would not have been possible through my efforts alone. I offer my heartfelt gratitude to everyone who supported me, believed in me, and accompanied me on this journey.

August 2026
Donghyun Kim

TABLE OF CONTENTS

List of Figures	v
List of Tables	x
Abstract	xiii
1 Introduction	1
1.1 Research Issues and Motivations	3
1.2 Main Contributions	7
1.3 Organization of the Dissertation	10
2 Related Work	12
2.1 Knowledge Graph Accuracy	13
2.1.1 KG Accuracy Evaluation	13
2.1.2 KG Error Detection	15
2.2 Knowledge Graph Validation	17
2.2.1 Internal KG Validation	17
2.2.2 External KG Validation	19
2.2.3 LM-based KG Validation	20
2.3 Large Language Models for Knowledge Graphs	21
2.3.1 LLM-based KG Construction	22
2.3.2 LLM-based KG Refinement	24

3	Knowledge Graph Accuracy Improvement via Active Learning	27
3.1	Preliminaries	28
3.1.1	Active Learning	28
3.1.2	Problem Definition	29
3.2	Proposed Approach: AIKA	29
3.2.1	AIKA Scenario	30
3.2.2	Triple Scoring	31
3.2.3	Triple Sampling	33
3.3	Experiments	37
3.3.1	Experimental Setup	37
3.3.2	Experimental Results: Simulated Setting	42
3.3.3	Experimental Results: Human-in-the-loop Setting	51
3.3.4	Discussions	55
3.4	Summary	57
4	Knowledge Graph Validation with Evidence-Grounded Reasoning	58
4.1	Preliminaries	60
4.1.1	Retrieval-Augmented Generation	60
4.1.2	Problem Definition	60
4.1.3	Preliminary Experiments	61
4.2	Proposed Approach: RAVER	65
4.2.1	Evidence Retrieval	65
4.2.2	Evidence-Grounded Reasoning	66

4.2.3	Retriever Alignment	67
4.3	Experiments	71
4.3.1	Experimental Setup	71
4.3.2	Experimental Results	75
4.4	Summary	87
5	Knowledge Graph Refinement via Operation-Guided Editing	88
5.1	Preliminaries	89
5.1.1	Generative Knowledge Extraction	89
5.1.2	Knowledge Graph Refinement	90
5.2	Proposed Approach: GraphRefine	90
5.2.1	Factual Inconsistencies in GKE	90
5.2.2	Fine-tuning LLM as KG Refiner	92
5.2.3	Graph Refinement	96
5.3	Experiments	97
5.3.1	Experimental Setup	97
5.3.2	Experimental Results	106
5.4	Summary	122
6	Conclusions	123
6.1	Key Contributions	124
6.2	Future Works	126
6.2.1	Extending Evidence from Text to Multi-Modal Sources	127
6.2.2	Agentic KG Maintenance	128

6.2.3 Using KGs to Improve the Reliability of AI Systems	130
References	132
Abstract in Korean	152

List of Figures

1.1	Overall framework for bridging the gap between KGs and real-world facts. The alignment gap arises between structured triples in KGs and real-world facts distributed across heterogeneous sources. This dissertation addresses the gap through three complementary stages: sampling, validation, and refinement.	2
1.2	Motivating example for KG accuracy improvement in large-scale settings. Noisy and long-tailed KGs require efficient and balanced triple selection.	3
1.3	Motivating example for evidence-grounded KG validation. Triples involving rare or ambiguous entities require validation beyond internal KG signals and parametric knowledge.	4
1.4	Motivating example for operation-driven KG refinement. Generated triples may require rewriting when entity representations are overly verbose or ambiguous.	6
1.5	Overview of the three research directions and their corresponding challenges, proposed frameworks, and key contributions.	8
3.1	The overall architecture of AIKA, our AL-based approach to enhancing KG accuracy.	30

3.2	An illustrative case showing how erroneous triples can be recognized using nothing but the information contained in the KG itself.	32
3.3	How the selected samples are distributed under standard score-based sampling compared with the proposed balanced sampling.	35
3.4	Progression of KG accuracy over the AL iterations, using a sampling size of 2,000 and 500 clusters.	45
3.5	Progression of KG accuracy over the AL iterations when PALT serves as the backbone, with a sampling size of 2,000 and 500 clusters.	46
3.6	Progression of KG accuracy over the AL iterations on WikiMovie1M, with a sampling size of 10,000 and 500 clusters.	47
3.7	A contrast between the relation makeup of the initial KG and the way triples refined under each AL sampling strategy are distributed, with relations running from the most to the least frequent in the initial KG.	48
3.8	Effect of varying the cluster count on the KG accuracy gains achieved by clustered sampling.	50
3.9	KG accuracy over the AL iterations when the number of clusters is adjusted dynamically during the process.	50
4.1	Contrasting conventional LLM-based KG validation with the retrieval-augmented alternative.	59
4.2	How different evidence conditions affect retrieval-augmented KG validation.	63

4.3	A high-level view of the proposed RAVER framework. The framework runs through two principal stages: (1) Evidence Retrieval, in which documents that are semantically pertinent are drawn from a large external corpus, and (2) Evidence-Grounded Reasoning, in which the LLM decides on triples using the retrieved evidence. The retriever is additionally optimized by gauging each document’s utility from validation outcomes and reshaping the retrieval preferences accordingly.	64
4.4	An illustration of how the retriever’s relevance distribution shifts from before to after alignment.	68
4.5	How rank changes for useful evidence are distributed once retriever alignment is applied. Every bar reports the fraction of useful documents that rose, held steady, fell, entered anew, or left the retrieved set.	80
4.6	Long-tail triple validation performance measured across the datasets. . .	81
4.7	A sample of the validation outcomes produced by RAVER and by LLM-based baselines for a long-tail triple.	82
4.8	How the error types are distributed in retrieval-augmented KG validation.	84
5.1	Illustrative cases of the factual inconsistencies that arise during generative knowledge extraction.	91

5.2	A high-level view of the GraphRefine pipeline. We first synthesize operation-tied supervision that captures the inconsistency categories seen in GKE, and an LLM is then fine-tuned on it to act as a KG refiner. Working one triple at a time, the refiner processes a draft KG, and its per-triple outputs are afterward combined into the refined graph.	93
5.3	Statistical profiles of the datasets: the upper panel reports the mean triple count per document for each dataset, while the lower panel presents how document lengths are distributed.	99
5.4	Across the manually annotated KGs of DocRED and SciERC, the breakdown of the various factual inconsistency types; here “+” denotes drafts that have undergone refinement.	108
5.5	Multi-dimensional comparison between draft KGs and their post-hoc refined counterparts. For clearer visualization, we min-max normalize each metric and then map it into the $[0.7, 0.9]$ interval.	109
5.6	Confusion matrix over refinement operations, computed for GraphRefine on the synthetically labeled test split of DocRED.	111
5.7	Multi-dimensional comparisons of draft KGs against their post-hoc refined versions produced by GraphJudge or GraphRefine, covering every dataset (GenWiki, DocRED, SciERC, and CDR) and every base extractor (Llama-2-7B, Llama-2-13B, GPT-4o-mini, and GPT-4o). Every metric is min-max normalized and then mapped onto the range $[0.7, 0.9]$ for readability.	115

6.1	Unified framework for improving the factual consistency of KGs. The framework transforms an input KG into a refined KG through three connected stages: suspicious triple sampling, evidence-grounded validation, and evidence-supported refinement. This pipeline enables systematic KG maintenance by identifying, verifying, and correcting triples that may be inconsistent with real-world facts.	124
6.2	Conceptual overview of a multi-modal KG maintenance pipeline that integrates suspicious triple selection, evidence-grounded validation, and refinement.	127
6.3	Conceptual overview of an agentic KG maintenance framework that iteratively coordinates triple selection, validation, and refinement using memory and feedback from previous actions.	128
6.4	A reliable KG enhances the trustworthiness of AI systems through grounding, verification, and reasoning support, leading to more factual answers, explainable decisions, and consistent responses.	130

List of Tables

3.1	Summary of the benchmark KG datasets.	38
3.2	Gains in KG accuracy obtained on the FB15K237 and WikiMovie300K datasets, where R indicates the fraction of erroneous triples contained in each dataset. Sampling sizes of 1,000 (R=0.1) and 2,000 (R=0.2) are used, and clustered sampling operates with 500 clusters.	43
3.3	Link prediction results on FB15K237. Scores computed over all triples appear on the left, while the right side covers only triples belonging to the minor 50 relations.	49
3.4	KG accuracy improvement attained by AIKA when human oracles with different degrees of domain expertise carry out the rectification.	54
4.1	Summary statistics for the KG benchmarks. Within the validation and test splits, positive (+) and negative (−) samples appear in equal numbers, with the negatives taken from the original datasets.	72
4.2	Comparison of overall KG validation performance over every benchmark. Bold marks the best Accuracy and F1-score, and underlining marks the second-best.	76
4.3	Validation performance under varying retrieval strategies. Bold denotes the best Accuracy and F1-score, and underlining denotes the second-best.	77

4.4	Shifts in evidence utility and retrieval success brought about by retriever alignment. <i>Imp.</i> refers to the absolute gain measured from before to after alignment.	79
4.5	Sample top-1 documents retrieved by the baseline methods and by our proposed approach.	83
4.6	Illustrative instances of triple misinterpretation and of incorrect reasoning.	85
5.1	Summary statistics for the four benchmark datasets. Avg. Triples gives the mean number of triples per KG, and Avg. Length gives the mean word count per document.	98
5.2	Hyperparameter values and model choices adopted when computing each of the GenRES metrics.	101
5.3	Comparison of overall performance across GenWiki, DocRED, SciERC, and CDR. KG quality is measured with BL/RO/BS (P/R/F1), and the multi-faceted GenRES scores (TS/US/FS/GS/CS) capture improvements from complementary angles. Bold marks the best score in each column and underlining the second-best.	106
5.4	Cross-dataset generalization measured by transferring a model between domain pairs (train $\rightarrow$ test); Δ indicates the difference relative to the in-domain training result.	110
5.5	Per-KG triple counts (averages) for the draft KGs produced by each base extractor on every dataset, together with the counts remaining once GraphJudge or GraphRefine has been applied.	117

- 5.6 Operation-level ablation of GraphRefine evaluated on DocRED and Sci-
ERC; the abbreviations D, F, and R stand for DELETE, FIX, and REWRITE. 118
- 5.7 Case study of KG extraction and refinement for a single document, com-
paring the draft KG with the outputs of GraphJudge and GraphRefine. . 120

Abstract

Towards Bridging the Gap between Knowledge Graphs and Real-World Facts

Knowledge graphs (KGs) represent real-world knowledge by structuring entities and their semantic relationships into a graph of relational triples. They serve as foundational infrastructure for a wide range of applications, including question answering, recommendation, conversational agents, and fact-checking. Despite their broad applicability, real-world KGs inevitably suffer from inconsistencies with real-world facts. These inconsistencies arise from errors in automated KG construction, failures in KG integration, and real-world changes that make previously correct triples outdated. Such inaccuracies undermine the reliability of KG-based applications, making factual alignment a critical requirement for trustworthy knowledge-driven systems.

To bridge the gap between KGs and real-world facts, KG maintenance should be viewed as a continuous process involving three complementary stages: selecting triples that require attention, validating their factual correctness, and refining erroneous or poorly represented triples. However, each stage poses distinct challenges. First, large-scale KGs contain numerous triples in which correct and erroneous facts coexist, making exhaustive human verification impractical. Second, reliable validation requires external evidence and evidence-aware reasoning, since KG-internal signals or parametric knowledge alone may be insufficient for rare, ambiguous, or unseen facts.

Third, refinement should not be reduced to deletion, because some problematic triples are partially correct and can be repaired or rewritten instead of being discarded.

This dissertation addresses these challenges through three interconnected directions: *KG accuracy improvement*, *KG validation*, and *KG refinement*. First, we propose **AIKA**, an active learning framework for KG accuracy improvement. AIKA iteratively trains a KG embedding model, scores triples, and selects suspicious triples for human-in-the-loop rectification under a limited annotation budget. By combining uncertainty-based selection with balanced and clustered sampling, AIKA efficiently improves KG accuracy while reducing bias toward frequent relations.

Second, we introduce **RAVER**, a retrieval-augmented KG validation framework with evidence-grounded reasoning. RAVER retrieves external evidence for a target triple and guides a large language model (LLM) to determine whether the triple is factually correct based on the retrieved documents. To improve the quality of evidence retrieval, RAVER further aligns the retriever with the LLM’s evidence utility, enabling the system to prioritize documents that are useful for downstream validation rather than merely topically relevant.

Third, we propose **GraphRefine**, an operation-driven KG refinement framework for KGs constructed via generative knowledge extraction. GraphRefine employs an LLM-based refiner to assess generated triples against the source document, assign an explicit refinement operation to each triple, and update the KG accordingly. This enables the model to remove unsupported triples, repair incorrect components, and rewrite ambiguous or overly verbose representations while preserving valid information.

Extensive experiments on diverse benchmarks demonstrate that the proposed methods consistently improve KG accuracy, validation reliability, and refinement quality across different datasets, model architectures, and evaluation settings. The results further show that effectively maintaining factual consistency requires not only detecting erroneous triples, but also prioritizing which facts should be examined, grounding validation decisions in reliable external evidence, and applying appropriate refinement strategies according to the type of error.

Overall, this dissertation provides a unified perspective on bridging the gap between KGs and real-world facts by enabling efficient suspicious triple selection, evidence-grounded factual validation, and operation-driven refinement. By connecting these complementary stages, the dissertation establishes a systematic foundation for continuous KG maintenance and highlights the importance of combining structured knowledge, external evidence, and LLM-based reasoning. This perspective further opens opportunities for multimodal evidence integration, autonomous and agentic KG maintenance, continual adaptation to evolving real-world knowledge, and trustworthy AI systems built upon accurate, explainable, and verifiable structured knowledge.

Keywords : Knowledge Graph Accuracy, Knowledge Graph Validation, Knowledge Graph Refinement, Active Learning, Retrieval Augmentation, Large Language Models

1. Introduction

Knowledge graphs (KGs) organize real-world knowledge into structured triples, each capturing a semantic relationship between entities. They serve as a fundamental infrastructure for a wide range of applications, including question answering [28, 105, 57], recommendation [89, 90, 101], conversational agents [60, 36, 83], and fact verification [88, 37, 9]. By representing facts as triples, KGs support semantic querying, structured reasoning, and explainable decision making across diverse domains.

Despite their broad applicability, real-world KGs inevitably suffer from inconsistencies with real-world facts [91, 97]. These inconsistencies stem from a variety of factors. First, errors can be introduced during KG construction, especially when knowledge is automatically extracted from heterogeneous sources such as text, tables, reports, and web documents [113, 114]. Pipeline-based extraction systems may propagate errors across intermediate stages, while generative knowledge extraction models may introduce hallucinated or imprecise triples [31, 106]. Second, errors can occur during KG integration, where entity alignment failures, duplicate triples, and conflicting facts may lead to incorrect or inconsistent graph structures [107, 86]. Third, even initially correct triples can become outdated as real-world facts change over time, and delayed updates may cause the KG to gradually diverge from the current state of the world [7, 22]. As a result, the gap between KGs and real-world facts remains a central obstacle to building reliable knowledge-driven systems. Such inconsistencies can undermine the reliability of KG-based applications, making factual alignment a

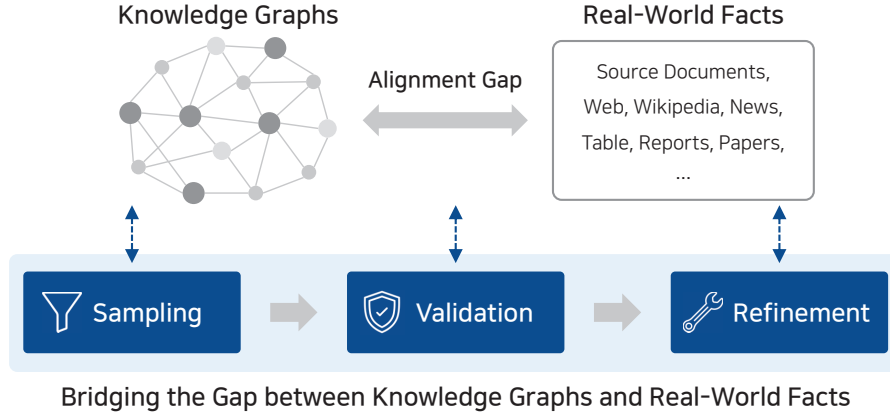


Figure 1.1: Overall framework for bridging the gap between KGs and real-world facts. The alignment gap arises between structured triples in KGs and real-world facts distributed across heterogeneous sources. This dissertation addresses the gap through three complementary stages: sampling, validation, and refinement.

critical requirement for trustworthy knowledge-driven systems.

To bridge the gap between KGs and real-world facts, this dissertation conceptualizes KG maintenance as a continuous process that goes beyond isolated correction. As Figure 1.1 shows, we decompose this process into three mutually reinforcing stages: selecting triples that require attention, validating their factual correctness, and refining erroneous or poorly represented triples. Each stage addresses a distinct challenge in maintaining large-scale KGs: deciding which triples to inspect, determining whether they are factually correct, and deciding how to revise them when errors are found. Therefore, maintaining the consistency of KGs with real-world facts requires a unified perspective that connects triple selection, evidence-grounded validation, and operation-driven refinement. Building on this perspective, this dissertation investigates three interconnected research directions: **KG accuracy improvement**, **KG validation**,

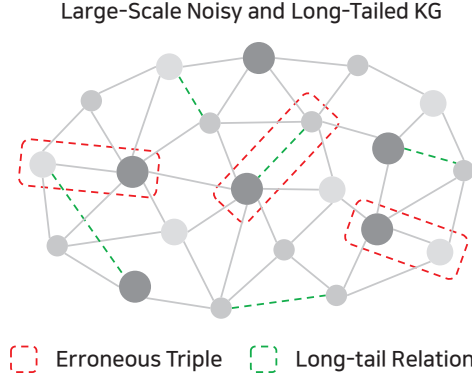


Figure 1.2: Motivating example for KG accuracy improvement in large-scale settings. Noisy and long-tailed KGs require efficient and balanced triple selection.

and **KG refinement**. Together, these directions provide a systematic approach to improving the factual reliability of KGs.

1.1 Research Issues and Motivations

Despite substantial progress in maintaining and validating KGs, several fundamental challenges still limit their ability to remain consistent with real-world facts. This dissertation addresses these challenges by identifying research issues that correspond to the key stages of the continuous maintenance process described above.

Efficient KG Accuracy Improvement in Large-Scale Settings Large-scale KGs consist of numerous triples, among which correct and erroneous triples coexist. As illustrated in Figure 1.2, erroneous triples can appear in different regions of the graph, while long-tail relations are sparsely distributed and may be easily overlooked by naive sampling strategies. Verifying every triple by hand yields trustworthy judgments, yet the steep annotation cost makes such exhaustive checking infeasible. Fully

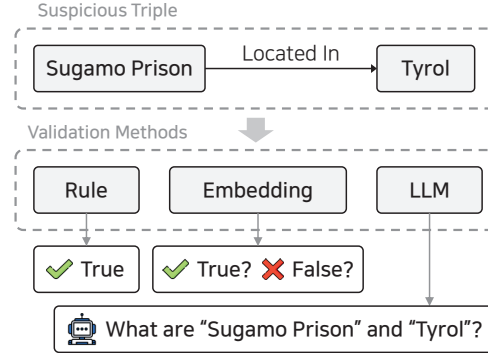


Figure 1.3: Motivating example for evidence-grounded KG validation. Triples involving rare or ambiguous entities require validation beyond internal KG signals and parametric knowledge.

automated methods can process large KGs efficiently, but their predictions may be unreliable when the model is uncertain or when the KG contains sparse entities and long-tail relations. In addition, naively sampling triples for verification can lead to biased improvement, because frequent relations and well-represented regions of the graph may dominate the annotation process. This motivates the need for an efficient KG accuracy improvement framework that can prioritize suspicious triples under a limited verification budget while considering the long-tail distribution of KGs. Rather than treating all triples equally, such a framework should actively select triples whose correctness is uncertain or whose correction is likely to contribute to more balanced KG accuracy improvement.

Reliable KG Validation with External Evidence Existing KG validation methods often rely on KG-internal signals, such as paths, rules, and embeddings, or on parametric knowledge embedded in language models (LMs). However, internal validation

can amplify existing KG errors and is vulnerable to rare or unseen facts. As illustrated in Figure 1.3, given a suspicious triple such as (*Sugamo Prison, located in, Tyrol*)¹, rule-based methods may incorrectly accept the triple based on structural patterns, while embedding-based methods may produce uncertain predictions. LM-based validation can exploit parametric knowledge, but it may still fail when the model does not reliably know what “*Sugamo Prison*” and “*Tyrol*” refer to, potentially leading to hallucinated or unsupported judgments. To reliably determine whether a triple is consistent with real-world facts, validation should therefore be grounded in explicit external evidence. This is particularly important in open-domain settings, where triples may involve obscure entities, ambiguous relations, or facts that require contextual interpretation. However, evidence-grounded validation introduces its own challenges. Relevant evidence must be retrieved from a large external corpus, and the model must reason over the retrieved documents to decide whether they support or contradict the target triple. Therefore, reliable KG validation requires both effective evidence retrieval and evidence-aware reasoning.

Operation-Driven Refinement of Generated KGs Progress in generative knowledge extraction now makes it possible to build KGs straight from source documents by means of generative models. Although such methods improve the scalability and flexibility of KG construction, they also introduce complex inconsistency types. As illustrated in Figure 1.3, a generative extractor may produce a triple such as (*Historiographic metafiction, coined by, Canadian literary theorist Linda Hutcheon in the late*

¹Sugamo Prison was a Japanese correctional facility situated in Tokyo. Its site lay in the Ikebukuro area, a district that today falls within Toshima ward.

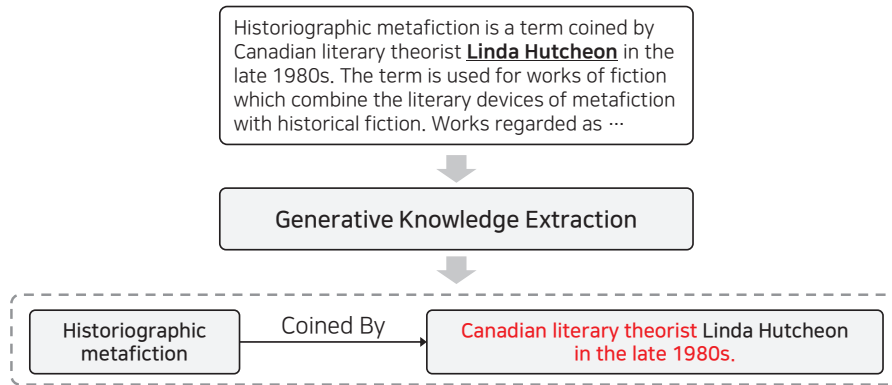


Figure 1.4: Motivating example for operation-driven KG refinement. Generated triples may require rewriting when entity representations are overly verbose or ambiguous.

1980s). While this triple is semantically related to the source document, its object entity is overly verbose and mixes the target entity with contextual information, indicating that the triple should be rewritten rather than simply kept or deleted. Generated triples fail in heterogeneous ways: some lack any grounding in the source document, others carry the wrong entities or relations, and still others state correct facts yet express them through ambiguous or excessively wordy surface forms. Coping with such a mixture of defects through a single deletion rule is inadequate. Removing an erroneous triple may raise precision, but it simultaneously shrinks the graph’s coverage and throws away partially correct content that could otherwise have been salvaged. Thus, KG refinement should be formulated as an operation-driven editing problem. For each generated triple, the system should determine whether to keep it, delete it, fix incorrect components, or rewrite its representation so that the resulting KG better aligns with the source document and real-world facts.

1.2 Main Contributions

The contributions of this dissertation are organized around three complementary stages of KG maintenance: accuracy improvement, validation, and refinement. As summarized in Figure 1.5, this dissertation develops a corresponding framework for each stage. AIKA improves KG accuracy through efficient and balanced selection of informative triples, RAVER enables reliable validation through external evidence and evidence-grounded reasoning, and GraphRefine addresses heterogeneous factual and representational errors through operation-driven post-hoc refinement.

Together, these frameworks provide a systematic approach to maintaining the alignment between KGs and real-world facts, progressing from identifying triples that require attention, to verifying their factual validity, and finally to correcting erroneous or poorly represented knowledge. The main contributions of each research direction are summarized below.

Knowledge Graph Accuracy Improvement via Active Learning (AIKA):

- We present **AIKA**, an active learning framework that raises the accuracy of large-scale KGs in which correct and erroneous triples are intermixed.
- We formulate KG accuracy improvement as an iterative human-in-the-loop process that trains a KG embedding model, scores triples, selects suspicious triples, and updates the KG based on rectification.
- We introduce uncertainty- and plausibility-based sampling strategies that prioritize triples likely to require correction under a limited annotation budget.

Bridging the Gap between Knowledge Graphs and Real-World Facts				
<i>Dissertation Direction</i>	KG Accuracy Improvement via Active Learning	KG Validation with Evidence-Grounded Reasoning	KG Refinement via Operation-Driven Editing	
<i>Research Subjects</i>				
<i>Research Challenges</i>	Efficient and balanced triple selection in large-scale, noisy, and long-tailed KGs	Reliable triple validation when internal signals are insufficient and retrieved evidence is noisy	Correction of heterogeneous factual and representational errors without unnecessary information loss	
<i>Proposed Frameworks</i>	AIKA Active Learning Framework for Improving KG Accuracy	RAVER Retrieval-Augmented KG Validation with Evidence-Grounded Reasoning	GraphRefine Operation-Driven KG Refinement for KGs with Heterogeneous Errors	
<i>Key Contributions</i>	• Informative triple selection • Balanced and diverse sampling • Iterative human-in-the-loop improvement	• External evidence retrieval • Evidence-grounded LLM reasoning • Retriever alignment with evidence utility	• Taxonomy of heterogeneous error types • Operation-aware triple editing • Model-agnostic post-hoc refinement	

Figure 1.5: Overview of the three research directions and their corresponding challenges, proposed frameworks, and key contributions.

- We incorporate balanced and clustered sampling strategies to mitigate bias caused by long-tail relation distributions and to encourage more diverse triple selection.
- We empirically show that AIKA consistently improves KG accuracy across different KG embedding models and datasets, and that balanced sampling strategies contribute to improved performance on long-tail relations and downstream link prediction.

Knowledge Graph Validation with Evidence-Grounded Reasoning (RAVER):

- We propose **RAVER**, a retrieval-augmented KG validation framework that verifies whether a target triple is consistent with real-world facts using external evidence.
- We formulate KG validation as an evidence-grounded reasoning task, where a target triple is used as a query to retrieve relevant documents and an LLM generates both a factual judgment and an explanation based on the retrieved evidence.
- We design a retriever alignment strategy that aligns the retriever’s relevance distribution with the LLM’s evidence utility, enabling the retriever to prioritize documents that are useful for downstream validation.
- We demonstrate that evidence-grounded validation reduces reliance on model-parametric knowledge and improves validation reliability, especially for long-tail triples where KG-internal signals and LLM knowledge are insufficient.
- Extensive experiments on multiple KG validation benchmarks show that RAVeR achieves strong validation performance compared with KG embedding, PLM-based, LLM-based, and retrieval-based baselines.

Knowledge Graph Refinement via Operation-Driven Editing (GraphRefine):

- We introduce **GraphRefine**, an LLM-driven framework that refines, in a post-hoc manner, KGs produced through generative knowledge extraction.
- We formulate KG refinement as an operation-driven editing problem, where each generated triple is assigned one of four operations: KEEP, DELETE, FIX, or REWRITE.
- We design a triple-level refinement process that uses the source document and draft KG as input, predicts the appropriate refinement operation, and generates a corrected triple when necessary.
- We show that GraphRefine can be applied independently of the underlying knowledge extraction model, making it a general post-hoc refinement framework for generated KGs.
- Empirical results demonstrate that GraphRefine improves the factual alignment of generated KGs while reducing the information loss caused by deletion-only refinement.

1.3 Organization of the Dissertation

This dissertation is composed of six chapters. Chapter 1 introduces the problem of bridging the gap between KGs and real-world facts and presents the research issues and contributions of this dissertation. Chapter 2 reviews related work on KG accuracy, KG validation, and LLMs for KGs. Chapter 3 presents AIKA, an active learning framework for KG accuracy improvement. Chapter 4 introduces RAVER, a retrieval-

augmented KG validation framework with evidence-grounded reasoning. Chapter 5 describes GraphRefine, an operation-driven KG refinement framework for generated KGs. Finally, Chapter 6 concludes the dissertation by summarizing the key findings and discussing future research directions.

2. Related Work

This chapter reviews prior studies closely related to this dissertation, organizing major research threads and situating our work within the existing literature. We focus on how the accuracy of knowledge graphs (KGs) has been defined and evaluated, how KG quality has been validated and improved, and how recent advances in large language models (LLMs) have influenced research in this area.

We first review studies that address the accuracy of KGs, including methods for efficiently estimating graph-level correctness and approaches that improve KG quality by identifying and correcting erroneous triples. Together, this body of work provides the conceptual and methodological foundation for understanding and improving the overall correctness of large-scale KGs.

Next, we survey research that focuses on assessing the factual validity of individual triples. Prior work in this area can be broadly grouped according to the types of validation signals they employ, ranging from KG-internal cues to external evidence and language-model-based judgments. This line of research shifts the emphasis from graph-level quality assessment to triple-level factual analysis.

Finally, we summarize recent efforts that apply LLMs to KG-related tasks, with particular attention to methods for constructing KGs from unstructured text and refining existing KGs. These studies illustrate how LLMs can be leveraged to automate knowledge acquisition and enhance KG quality, forming an important basis for the approaches developed in this dissertation.

2.1 Knowledge Graph Accuracy

Because large-scale KGs are populated automatically by extracting knowledge from diverse and heterogeneous sources, the presence of faulty triples is essentially unavoidable. Such errors lower the quality of a KG and, in turn, weaken the many downstream applications that rely on it. This concern has driven a considerable body of work aimed at making KGs more accurate and trustworthy.

In this section, we review prior studies along two main directions: (i) *KG accuracy evaluation*, which aims to estimate the graph-level factual correctness of a KG under limited budgets; and (ii) *KG error detection*, which aims to identify suspicious or unreliable triples that are likely to be erroneous within an existing KG.

2.1.1 KG Accuracy Evaluation

KG accuracy evaluation aims to statistically estimate the overall factual correctness of a large-scale KG by verifying only a limited number of sampled triples. Formally, given a KG $\mathcal{G}$, an evaluation method first selects a sampled subset of triples $\mathcal{G}_s \subset \mathcal{G}$ using a sampling strategy $S(\cdot)$:

$$\mathcal{G} \xrightarrow{S(\cdot)} \mathcal{G}_s \subset \mathcal{G}.$$

The sampled triples are then manually or automatically verified, and their verification results are used to estimate the graph-level accuracy of $\mathcal{G}$, often with statistical uncertainty. KG accuracy evaluation aims to effectively assess the factual correctness of large-scale KGs without exhaustively verifying all triples. Since full assessment is

infeasible at scale, existing approaches typically rely on sampling-based estimation or constraint-aware selection to obtain a small yet informative subset of triples for verification. The goal is to produce a reliable estimate of graph-level accuracy, often accompanied by uncertainty quantification, under limited annotation budgets.

KGEval [62] gauges graph-level correctness from only a handful of verified triples by exploiting coupling constraints among them. Deciding which triples to annotate is cast as a submodular optimization problem, which admits a greedy solution accompanied by theoretical guarantees. In a related direction, Gao et al. [18] design an accuracy-evaluation framework that draws representative samples so as to keep annotation cost low without giving up statistical reliability. Their approach additionally accommodates evolving KGs, refreshing the estimate incrementally as new triples arrive rather than recomputing it from scratch. Pursuing a different angle, Qi et al. [67] devise an optimized human-machine collaboration scheme that measures accuracy jointly at the triple and entity levels to trim annotation effort further; here, uncertain or high-impact cases are routed to human annotators, whereas confidently predicted ones are resolved automatically. More recently, Marchesin and Silvello [58] strengthen KG accuracy estimation by adopting robust confidence intervals that remain reliable under practical sampling designs. They show that commonly used confidence-interval methods can be misleading near extreme accuracy regimes and provide more reliable alternatives.

However, these studies primarily focus on estimating the overall correctness of a KG, rather than translating evaluation outcomes into concrete actions for improving the target KG. Moreover, their triple selection strategies are designed for statistically

representative evaluation, and thus are not necessarily suitable for prioritizing triples that are likely to be erroneous. Therefore, additional approaches are needed to move beyond accuracy estimation and support the detection and correction of erroneous triples, ultimately improving KG accuracy.

2.1.2 KG Error Detection

KG error detection aims to identify triples that are likely to be erroneous within an existing KG. Unlike KG accuracy evaluation, which estimates graph-level accuracy from sampled triples, KG error detection assigns an error-related score to each triple and ranks triples according to their likelihood of being incorrect. Formally, given a KG $\mathcal{G}$, a detection model $\mathcal{D}(\cdot)$ produces a ranked set of triples with corresponding suspiciousness scores:

$$\mathcal{G} \xrightarrow{\mathcal{D}(\cdot)} \{(t, s_t) \mid t \in \mathcal{G}\},$$

where s_t denotes the score assigned to triple t . Triples with higher suspiciousness scores are then prioritized for inspection, deletion, or correction.

The accuracy of a KG depends on how effectively erroneous triples are identified and handled within the graph. In this context, KG error detection identifies suspicious or unreliable triples under the assumption that some observed triples in a KG may be incorrect. By assessing whether each triple conforms to the structural and statistical patterns of the KG, it prioritizes triples that behave as outliers or exhibit low trustworthiness, thereby providing a basis for improving KG accuracy.

CAGED [109] approaches error detection through contrastive learning, pairing multi-view graph augmentation with an error-sensitive encoder so that complemen-

tary structural signals from different views are absorbed, which yields more discriminative triple representations for spotting anomalies. Exploiting the complementary strengths of different detectors, Kael [12] assembles an active ensemble that couples several heterogeneous error detectors and refines its selection policy across iterations while requiring only limited human labeling, thereby boosting detection quality under scarce supervision. HEAR [112] further explores structural evidence by leveraging hierarchical path structures to model both local and global contexts around a triple. By explicitly encoding multi-hop relational dependencies in a hierarchical manner, HEAR improves robustness to complex structural errors that are difficult to detect using single-triple representations alone. Reaching beyond structure-only cues, CCA [51] adopts an interactive contrastive learning scheme that fuses textual meaning with graph topology and reads mismatches between the two as signals for flagging triples that appear plausible yet are factually wrong. Similarly, SeSICL [52] integrates semantic representations derived from textual descriptions with local and global structural embeddings, and applies contrastive learning with encoder perturbations to obtain robust joint representations for KG error detection. More recently, KGDMC [79] extends this line of research to multimodal KGs by aiming to identify not only structural triple errors but also cross-modal inconsistencies (e.g., mismatches between visual and textual evidence) in multimodal KG data.

Despite these advances, most existing KG error detection methods improve KG accuracy primarily by deleting detected erroneous triples. Although this strategy can effectively remove obvious noise, the predictions of detection models are not always reliable, making it difficult to judge the actual likelihood of an error based solely

on plausibility scores. Furthermore, deletion may discard valid information, thereby reducing the coverage of the KG.

2.2 Knowledge Graph Validation

KG validation aims to determine whether an individual KG triple is factually correct. Formally, given a KG $\mathcal{G}$ and a triple $t \in \mathcal{G}$, a validation model $\mathcal{V}(\cdot)$ predicts a binary factuality label y_t :

$$t \xrightarrow{\mathcal{V}(\cdot)} y_t \in \{\text{True}, \text{False}\}.$$

Unlike KG error detection, which ranks triples according to their relative abnormality based on internal structural or statistical patterns, KG validation assesses the absolute factual correctness of each triple with respect to real-world facts.

In this section, we review prior studies on KG validation by categorizing them into three groups: (i) *internal KG validation*, (ii) *external KG validation*, and (iii) *LM-based KG validation*. For each category, we summarize the core ideas and discuss their respective limitations.

2.2.1 Internal KG Validation

Internal KG validation methods assess the validity of individual triples by leveraging structural and statistical signals inherent to the KG, such as relational paths, rules, and recurring patterns. These approaches perform validation solely based on the KG itself, without relying on external information sources.

Path-based methods evaluate the validity of a target triple by analyzing relational paths that connect its subject and object, assuming that valid relations are

supported by recurrent multi-hop paths or semantically coherent relation sequences in the graph. DPP [76] extracts type-anchored predicate paths between the subject and object and selects discriminative ones as explicit features, using them as structural evidence to score triple validity. Knowledge Stream [78] casts validation as a network flow problem and scores a triple by aggregating evidence from multiple supporting paths, leveraging their costs/reliabilities rather than relying on any single path. COPAAL [81] is an unsupervised approach that searches for corroborative paths supporting the target triple while incorporating RDFS-level semantic constraints (e.g., domain/range and class hierarchies) to favor semantically consistent evidence.

Rule-based methods assess triple validity by extracting recurring relational patterns or logical constraints from a KG and encoding them as explicit rules [17, 16, 41], which can then be used as validation criteria. RUDIK [63] discovers both positive rules that support valid facts and negative rules that capture contradictions indicating invalid triples. It learns weighted soft rules to remain robust to noise and incompleteness, scoring triples based on how well rules cover positives while avoiding counter examples. Kim and Choi [38] propose an unsupervised approach that learns weighted rules for positive and negative evidential paths and combines them to compute a truth score for a target triple. Their weighting explicitly accounts for counter examples, enabling a more balanced use of supporting and refuting evidence.

Embedding-based methods assess triple validity by leveraging continuous vector representations learned from the statistical regularities of a KG and scoring a target triple in the embedding space [4, 93, 48, 99, 87, 13, 80]. KGTm [32] estimates triple trustworthiness by learning distributed vector representations of entities, relations, and

triples, and then combining embedding-derived signals through a crisscrossing neural architecture. Specifically, it fuses triple-level semantic representations with KG-global inference cues, producing a unified confidence score aggregated from entity-, relation-, and KG-global estimators. Likewise, CrossVal [92] formulates validation as an embedding-driven ranking problem via cross-graph representation learning: it jointly learns KG embeddings across a target KG and an auxiliary KG, incorporates cross-KG negative sampling based on mutually conflicting relations, and employs a confidence estimation module to rank triples by their probability of being true.

Despite their effectiveness, internal KG validation methods share inherent limitations because they rely solely on signals derived from the target KG. Since their judgments are based on internal consistency and distributional regularities, they may over-trust triples that are structurally well-supported yet factually wrong, while penalizing correct but rare or sparsely connected facts. Moreover, given that real-world KGs are often incomplete and noisy, missing links can weaken support for true triples, and existing errors can be propagated and reinforced through the learned patterns. These limitations motivate incorporating external evidence to ground validation in verifiable real-world information.

2.2.2 External KG Validation

External KG validation assesses the factual correctness of individual triples by leveraging evidence from sources outside the target KG. Unlike internal validation, which relies on signals within the KG, external validation focuses on whether a triple is supported by observable information in text or external knowledge sources. It typi-

cally involves verbalizing a triple, retrieving relevant documents, and judging whether the retrieved evidence supports the fact.

DeFacto [20] performs text-based fact checking by verbalizing RDF triples and retrieving evidence sentences from web documents or large text corpora, using the retrieved textual evidence to score triple validity. FactCheck [82] extends this paradigm by moving beyond surface-level matching and incorporating syntactic structure, document reliability, and topical coherence to improve evidence-based validation. Tracy [15] and ExFaKT [14] perform external KG validation via rule-based query rewriting, converting a target triple into simpler, observable queries and gathering evidence from external text or knowledge sources to assess factual correctness. This enables validation even without explicit mentions in text by combining relevant clues, while providing supporting evidence to improve interpretability.

Despite grounding validation in external evidence, these methods face persistent challenges. Their performance heavily depends on reliable evidence retrieval, and missing or irrelevant documents can lead to incorrect judgments. Moreover, interpreting evidence is non-trivial because support in text can be implicit, ambiguous, or partial, and sources may be noisy or even conflicting. As a result, many approaches struggle to make robust decisions at scale, motivating more integrated methods that jointly retrieve, interpret, and reason over evidence.

2.2.3 LM-based KG Validation

Recent advances in LMs have spurred growing interest in applying them to KG validation, where the goal is to assess the factual correctness of individual triples by

leveraging the parametric knowledge encoded in pre-trained LMs.

KG-BERT [102] linearizes KG triples as textual sequences and applies BERT [11] to compute plausibility scores via a classification-style objective. By scoring candidate triples, it demonstrates strong performance in KG completion and triple classification. PKGC [56] expresses triples as natural-language prompts and augments them with contextual signals, so PLMs can better leverage implicit knowledge when judging plausibility. Building on PLM-based approaches, recent work explores LLMs for KG validation by casting triple judgment as prompt-based validity prediction. KG-Llama [103] reformulates triples as natural-language prompts and fine-tunes LLMs for supervised plausibility judgment, leveraging stronger semantic reasoning than smaller PLMs. KoPA [111] enhances LLM-based validation with a knowledge prefix adapter that encodes KG structural signals into a learnable prefix, conditioning the LLM for structure-aware triple classification while retaining prompt-based inference.

Despite their promising performance, LM-based KG validation methods face notable limitations. Since they rely primarily on model-internal knowledge, they are vulnerable to instability and hallucinations, and often struggle to distinguish factual correctness from surface-level plausibility. As a result, their validation decisions may be unreliable, particularly for rare facts or cases requiring explicit evidence.

2.3 Large Language Models for Knowledge Graphs

With the rapid advancement of LLMs, a growing body of work has explored their application to KGs. Owing to their strong language understanding and generation capabilities learned from large-scale text corpora, LLMs have been leveraged to au-

tomate knowledge acquisition and to enhance the quality of existing KGs.

In this section, we review prior studies on LLMs for KGs along two main directions: (i) *LLM-based KG construction*, which focuses on extracting and organizing structured knowledge from text into graph form, and (ii) *LLM-based KG refinement*, which aims to improve existing KGs by completing missing facts and identifying erroneous facts, thereby enhancing both the coverage and accuracy of the KG.

2.3.1 LLM-based KG Construction

LLM-based KG construction refers to the task of generating a structured KG from unstructured or semi-structured source documents in an end-to-end manner. Formally, given a source document $\mathcal{D}$, a generative extraction model $\mathcal{E}(\cdot)$ produces a document-specific KG $\mathcal{G}_{\mathcal{D}}$:

$$\mathcal{D} \xrightarrow{\mathcal{E}(\cdot)} \mathcal{G}_{\mathcal{D}}.$$

Here, $\mathcal{G}_{\mathcal{D}}$ consists of entities and relations extracted from the source document and organized into a graph-structured representation.

Methods in this line use LLMs to turn unstructured or semi-structured text into structured KGs, letting the model identify entities and relations and assemble them into a graph. Because they depend far less on handcrafted rules and rigid schemas, such approaches scale more easily and adapt flexibly across a wide range of domains.

SAC-KG [8] targets the automatic creation of domain-specific KGs with an LLM at its center. The framework is split into three parts: a generator that builds the model input by pulling pertinent content from domain corpora and DBpedia, a verifier that corrects the outputs via rules, and a pruner that discards off-topic entities using a

classifier. EDC [108] divides the construction process into three consecutive phases—extraction, definition, and canonicalization—in order to curb both schema dependence and redundancy. Pairing LLM-produced definitions with similarity-based canonicalization, it can assemble structured knowledge even when no ontology is specified beforehand. Tree-KG [61] exploits the naturally hierarchical layout of knowledge-intensive documents to assemble an explicit tree-shaped KG. A collection of LLM-based operators then grows this tree, so that knowledge can be merged and extended efficiently without breaking structural consistency. KARMA [54] introduces an LLM-based multi-agent collaboration framework for automated KG expansion. In this framework, specialized agents separately handle extraction, alignment, verification, and integration, thereby distributing complex KG construction processes across a set of coordinated agents. KGGen [59] runs an LLM-driven pipeline that first extracts entities and relations from raw text and then integrates and deduplicates them into a KG. By merging embedding-based clustering with LLM-based normalization, it suppresses semantic redundancy and renders the final graph denser and more coherent.

For all their success at automating knowledge acquisition, LLM-based construction methods are not free of shortcomings. When supporting evidence is thin, they may emit factually wrong triples, frequently in the form of hallucinations, and preserving global consistency—for instance, steering clear of duplicate entities or contradictory relations—grows harder as freshly extracted knowledge is folded into an existing KG at scale.

2.3.2 LLM-based KG Refinement

LLM-based KG refinement refers to the task of reviewing and improving a generated or existing KG by identifying and revising its errors. Formally, given a generated KG $\mathcal{G}_{\mathcal{D}}$ and optionally its source document $\mathcal{D}$, a refinement model $\mathcal{R}(\cdot)$ produces a refined KG $\mathcal{G}_{\mathcal{D}}^*$:

$$(\mathcal{D}, \mathcal{G}_{\mathcal{D}}) \xrightarrow{\mathcal{R}(\cdot)} \mathcal{G}_{\mathcal{D}}^*.$$

Here, the source document $\mathcal{D}$ may be used as evidence for checking whether triples in $\mathcal{G}_{\mathcal{D}}$ are faithful to the original context, while $\mathcal{G}_{\mathcal{D}}^*$ denotes the corrected and improved KG.

LLM-based KG refinement methods focus on improving existing KGs, particularly by addressing missing facts and enhancing performance on KG completion (KGC) tasks. These approaches typically integrate LLMs with conventional KGC models to inject richer semantic information.

KICGPT [94], FtG [49], and KGR3 [46] adopt a decision-level integration strategy, where LLMs are used to reason over and re-rank candidate entities proposed by conventional KGC models. In this setting, the models exploit LLMs’ semantic understanding and contextual reasoning to judge which candidates are more plausible, without requiring LLMs to exhaustively score the entire entity space. In contrast, Contextualization Distillation [44] and CP-KGC [100] focus on representation-level augmentation, leveraging LLMs’ generation capability to produce (or distill) contextual descriptions that enrich textual representations for text-based KGC models. This improves semantic expressiveness while keeping the downstream inference procedure

unchanged. MPIKGC [96] further extends this direction by exploiting LLMs’ ability to generate diverse semantic perspectives, producing multiple contextual views that provide complementary signals for enriching entity and relation representations.

While these methods effectively exploit LLMs to fill missing knowledge in KGs, they primarily operate within standard KGC settings and do not explicitly verify the factual correctness of generated facts against external evidence.

To address this limitation, KGValidator [6] introduces an LLM-based validation framework that treats predicted triples as inputs to a post-hoc verification process. Each triple is validated by conditioning the LLM on multiple sources of information, including parametric knowledge, retrieved documents, and external knowledge sources, and the model outputs structured validation decisions (e.g., validity labels and justifications). This design leverages LLMs’ reasoning and evidence aggregation capabilities to assess factual correctness beyond plausibility-based completion. Building on a similar motivation, GraphJudge [26] further explores LLM-based KG validation by reformulating triple assessment as a direct judgment task. Given a target triple with optional contextual information, the LLM is prompted to explicitly decide whether the triple is valid and to generate a natural-language rationale, emphasizing transparent reasoning over latent scoring mechanisms.

Despite these advances, such LLM-as-a-judge frameworks remain limited in fully addressing the diverse error patterns observed in practice [43]. They largely frame refinement as a correctness-level judgment, deciding whether a triple should be accepted or discarded. As a result, they struggle with fine-grained inconsistencies—such as imprecise entity mentions, subtle relational semantic shifts, or partially supported

facts—that do not fit cleanly into a binary decision. Moreover, binary judgments can remove potentially useful triples, reducing KG coverage even when the issues are minor. These limitations motivate more nuanced refinement strategies that improve both the correctness and usability of KGs.

3. Knowledge Graph Accuracy Improvement via Active Learning

This chapter tackles the task of raising KG accuracy by means of active learning. KGs offer a structured encoding of real-world facts, yet at scale they unavoidably mix valid triples with faulty ones—a consequence of automated construction pipelines, mistakes introduced during integration, and the heterogeneous origins of the underlying knowledge. Because checking every triple exhaustively is not feasible, a central question arises: how can the triples most likely to need correction be pinpointed efficiently when the verification budget is restricted?

One obvious option is manual checking: human judgments are dependable, yet applying them across a massive KG is impractical. Conversely, fully automated techniques handle large KGs with ease, but their predictions lose reliability whenever the model is uncertain or the triples in question involve sparse entities and long-tail relations. These complementary weaknesses motivate an active learning approach in which model-based assessment is paired with selective human involvement. Rather than giving every triple equal attention, the model concentrates on suspicious ones—those with low plausibility or uncertain correctness—which makes the pursuit of higher accuracy considerably more efficient.

Building on this idea, we introduce **AIKA**, an active learning framework aimed at improving KG accuracy. In each round, AIKA fits a KG embedding model to the current KG, assigns plausibility scores to its triples, and picks out a suspicious

subset to be rectified. A human-in-the-loop procedure then corrects the chosen triples, after which the KG is updated with the results. AIKA additionally employs balanced and clustered sampling schemes that promote diverse and representative selection, counteracting the sampling bias induced by the long-tail distribution of relations and entities.

Repeated over multiple rounds, this procedure steadily raises KG accuracy while spending scarce annotation effort economically. Because verification is directed toward suspicious and uncertain triples instead of being applied at random, the framework offers a scalable route to higher KG quality in large-scale settings where valid and invalid triples are intermixed.

3.1 Preliminaries

3.1.1 Active Learning

Active learning (AL) denotes a paradigm in which a model reaches competitive performance from far fewer labels: instead of annotating data indiscriminately, informative samples are singled out for annotation under a fixed labeling budget [68]. Central to AL is the acquisition step, which decides which data points to query next according to criteria such as prediction uncertainty and sample diversity [40, 73, 3]. Given its proven benefits, AL has been studied in numerous domains, and a growing body of work has begun applying it to tasks involving KGs [64, 66, 50, 74, 27]. Even so, harnessing AL for the specific purpose of raising KG accuracy remains far from straightforward. We address this gap with an AL framework that couples machine-guided acquisition with human verification, enabling KG refinement that is

both scalable and economical in labels.

3.1.2 Problem Definition

We write the starting KG as $\mathcal{G}_{initial} = \mathcal{D} \cup \mathcal{D}'$, a collection of triples $(h, r, t) \in \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ in structured form, defined over an entity set $\mathcal{E}$ and a relation set $\mathcal{R}$. The subset $\mathcal{D}$ contains triples that are factually accurate, whereas $\mathcal{D}'$ gathers erroneous ones that contradict reality—(*BillGates*, *CEO*, *Google*) being one example. The task is to uncover and repair such faulty triples by running AL rounds over the initial KG $\mathcal{G}_{initial}$, so that the outcome is a KG $\mathcal{G}_{improve}$ of higher accuracy. A further objective concerns efficiency: the framework should extract the greatest achievable accuracy gain from a fixed, limited budget.

3.2 Proposed Approach: AIKA

This section describes AIKA, an AL framework devised to raise KG accuracy. An overview of the framework appears in Figure 3.1. AIKA runs through four successive stages. In the Training stage (1), a model is fitted to the target KG; the Scoring stage (2) then assigns plausibility values to every triple. During Sampling (3), the triples flagged as most suspicious are singled out, and in the final KG update stage (4), triples confirmed to be wrong are discarded while the corrected versions supplied by the human oracle take their place.

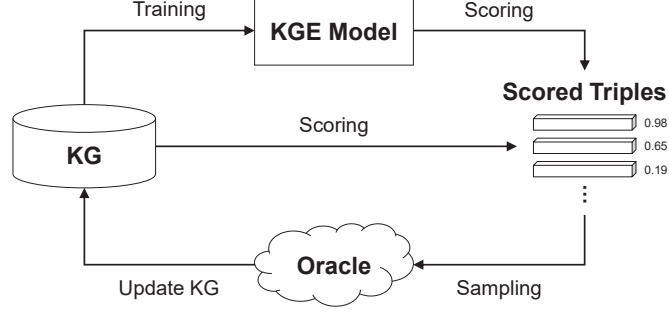


Figure 3.1: The overall architecture of AIKA, our AL-based approach to enhancing KG accuracy.

3.2.1 AIKA Scenario

Conventional AL grows a labeled set by drawing the most informative instances out of an unlabeled pool. AIKA departs from this scheme: its purpose is to refine the KG itself, flagging the triples that appear most suspicious and requesting corrections for them. The complete AL procedure followed by AIKA is given in Algorithm 1. During iteration k , a model M_k is freshly initialized and fitted to $\mathcal{G}_k$, the KG available at that point. Once training finishes, a scoring function assigns every triple $(h, r, t) \in \mathcal{G}_k$ a plausibility value, and these values together form $\mathcal{X}_k$. An acquisition function A then consults the scores to pick out the most suspicious subset $\mathcal{S} \subset \mathcal{G}_k$. A human oracle receives these triples and corrects them, which produces the rectified set $\mathcal{S}_k$. The KG is subsequently updated by deleting $\mathcal{S}$ and inserting $\mathcal{S}_k$, giving $\mathcal{G}_{k+1}$. Iterating these steps drives the accuracy of the KG steadily upward and eventually yields the enhanced graph $\mathcal{G}_{\text{improve}}$.

Algorithm 1 AIKA Scenario

Input: Initial KG $\mathcal{G}_{initial}$, Model M , Acquisition function A , Total iterations K

Output: Improved KG $\mathcal{G}_{improve}$

```
1: for iterations  $k = 1 \dots K$  do
2:    $M_k \leftarrow$  Initialize and train on  $\mathcal{G}_k$ 
3:    $\mathcal{X}_k \leftarrow \emptyset$ 
4:   for triple  $(h, r, t) \in \mathcal{G}_k$  do
5:      $\mathcal{X} \leftarrow M_k(h, r, t)$  ▷ Compute score
6:      $\mathcal{X}_k \leftarrow \mathcal{X}_k \cup \{\mathcal{X}\}$ 
7:   end for
8:    $\mathcal{S} \leftarrow A(\mathcal{G}_k, \mathcal{X}_k)$  ▷ Sample acquisition
9:    $\mathcal{S}_k \leftarrow$  Oracle labels  $\mathcal{S}$ 
10:   $\mathcal{G}_k \leftarrow \mathcal{G}_k \setminus \mathcal{S}$ 
11:   $\mathcal{G}_k \leftarrow \mathcal{G}_k \cup \mathcal{S}_k$  ▷ Update KG
12: end for
13: return  $\mathcal{G}_{improve}$ 
```

3.2.2 Triple Scoring

Detecting erroneous triples requires a model capable of assigning each triple a plausibility estimate. We obtain this estimate from a KGE model’s scoring function $f_r(h, t)$. This raises a difficulty, however. Since the target KG blends accurate triples with faulty ones, the KGE model unavoidably ends up trained as though each triple it encounters were correct.

Our remedy rests on exploiting how the plausibility scores of triples compare with one another rather than their absolute magnitudes. Consider the example in Figure 3.2: at training time, no distinction is made and the whole KG is taken to be valid. At inference time, though, a genuinely wrong triple like $(BillGates, CEO, Google)$ typically scores below a correct one like $(DonaldTrump, bornin, NewYork)$ —even when both are labeled true—because such wrong triples correspond to patterns rarely

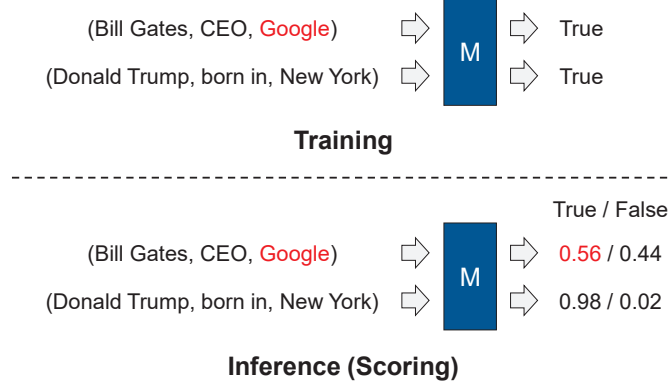


Figure 3.2: An illustrative case showing how erroneous triples can be recognized using nothing but the information contained in the KG itself.

observed in real-world KGs. This observation leads us to cast the detection of incorrect triples as a binary classification problem: deciding whether a given triple holds or not. To achieve this, the model is trained to report, for each triple, how likely it is to be true; in practice, we obtain this value by feeding the original score through a sigmoid:

$$s(h, r, t) = \text{sigmoid}(f_r(h, t)). \quad (3.1)$$

With this formulation, correct and incorrect triples can be separated effectively even though the model sees nothing beyond the KG itself. Because the change amounts to attaching one lightweight output layer, AIKA stays model-agnostic: a broad range of KGE architectures can be plugged in without substantial rework. Recall that training proceeds as though every triple in the KG were valid. Learning a binary classifier nonetheless demands explicit negative evidence. We therefore turn to negative sam-

pling [4], which supplies the model with explicitly labeled negatives. Each negative sample is formed by randomly replacing the head or the tail entity of a triple:

$$\mathcal{G}' = \{(h', r, t) | h' \in \mathcal{E}\} \cup \{(h, r, t') | t' \in \mathcal{E}\}. \quad (3.2)$$

In this formulation, $\mathcal{G}'$ is the pool of artificial negatives created solely for training; it should not be confused with $\mathcal{D}'$, the genuinely erroneous triples the KG already contains, where $\mathcal{G} = \mathcal{D} \cup \mathcal{D}'$. The scoring model is subsequently optimized to drive the following binary cross-entropy objective to its minimum:

$$-\sum_{\tau \in \mathcal{G}} \sum_{\tau' \in \mathcal{G}'} (y \cdot \log(s(\tau)) + (1 - y) \cdot \log(1 - s(\tau'))), \quad (3.3)$$

in which τ is a positive triple, τ' a negative one, and $y \in \{1, 0\}$ its associated label.

In line with Algorithm 1, each AL iteration starts the model afresh and retrains it from scratch on the newly updated KG.

3.2.3 Triple Sampling

A central design choice in AIKA is triple sampling, which decides which triples are forwarded to the human oracle when the budget is constrained. Given the KG $\mathcal{G}$ together with the model scores $\mathcal{X}$, the aim is to pick a subset $\mathcal{S} \subset \mathcal{G}$ that concentrates erroneous or low-quality triples—those whose correction benefits the KG the most. Besides a random baseline, we study score-driven strategies built on plausibility and on uncertainty, and we add balanced and clustered variants designed to counter the skewed, long-tailed relation distribution typical of real-world KGs.

Random Sampling (RD)

The simplest and most widely adopted selection scheme is to sample at random.

AIKA uses it as a baseline, drawing triples from the KG uniformly:

$$\mathcal{S} = \text{random}(\psi_1, \dots, \psi_{|\mathcal{G}|}), \quad (3.4)$$

in which $\psi_i = (h, r, t)$ represents an individual triple of $\mathcal{G}$.

Plausibility Sampling (PS)

In plausibility sampling, selection is guided by the plausibility value π that the scoring function assigns to each triple. The lower this value, the more suspicious the triple is deemed, so the lowest-scoring triples are chosen:

$$\pi_i = s(\psi_i), \quad (3.5)$$

$$\mathcal{S} = \text{argmin}(\pi_1, \dots, \pi_{|\mathcal{G}|}). \quad (3.6)$$

Uncertainty Sampling (UC)

Uncertainty sampling rests on the premise that the model's predictions are least confident on erroneous triples. Since the plausibility score can be read as the probability that a triple is correct, it lends itself to an entropy formulation. AIKA accordingly derives the entropy η from this score and retains the triples whose uncertainty is

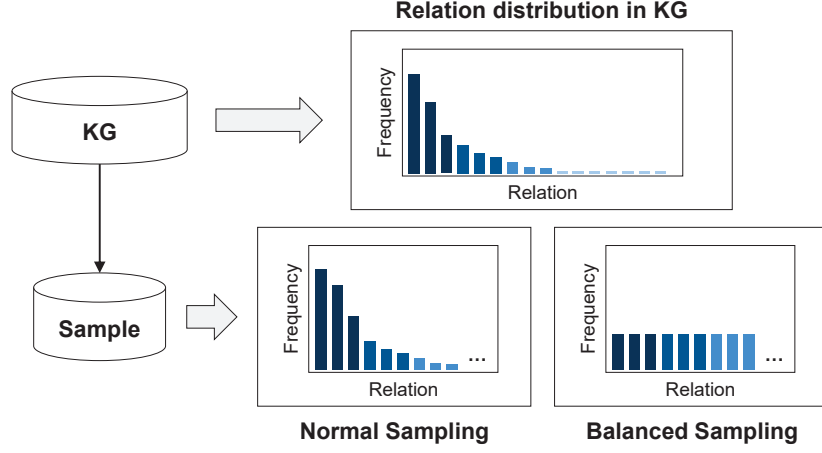


Figure 3.3: How the selected samples are distributed under standard score-based sampling compared with the proposed balanced sampling.

greatest:

$$\eta_i = -\pi_i \cdot \log \pi_i - (1 - \pi_i) \cdot \log(1 - \pi_i), \quad (3.7)$$

$$\mathcal{S} = \operatorname{argmax}(\eta_1, \dots, \eta_{|\mathcal{G}|}). \quad (3.8)$$

Balanced Sampling

In practice, the relation frequencies of a KG are heavily skewed [95]: a handful of relations cover most of the triples, and the remaining relations occur only rarely. Selecting triples strictly by plausibility or uncertainty tends to reinforce this skew, a tendency illustrated in Figure 3.3. Refinement effort then concentrates on a few high-frequency relations, leaving the rest of the KG largely unimproved. To counter this, AIKA adopts a balanced sampling strategy in which the relation distribution is fac-

Algorithm 2 Balanced Sampling

Input: KG $\mathcal{G}$, Computed score $\mathcal{X}$, Number of relations N_r , Sampling size N_s

Output: Sampled triples $\mathcal{S}$

```
1:  $\mathcal{S} = \emptyset$ 
2:  $N_{pr} \leftarrow N_s / N_r$   $\triangleright$  Number of sample per relation
3:  $\mathcal{X}_r \leftarrow$  Score of triples with relation  $r$ 
4: for each relation  $r$  in  $\mathcal{G}$  do
5:    $\mathcal{B} \leftarrow \text{argsort}(\mathcal{X}_r)[1:N_{pr}]$ 
6:    $\mathcal{S} = \mathcal{S} \cup \{\mathcal{B}\}$ 
7: end for
8: return  $\mathcal{S}$ 
```

tored in explicitly. Under balanced sampling, triples are still ranked by score, but an equal quota is drawn for every relation. The procedure is detailed in Algorithm 2. Balancing is combined with both PS and UC; the $\text{argsort}(\cdot)$ in Line 5 sorts in ascending order when paired with PS and in descending order when paired with UC.

Clustered Sampling

Clustered sampling offers a second route to easing selection imbalance, one that supplements the relation distribution with the semantic similarity among triples. Whereas balanced sampling operates on explicit relation labels, the clustered variant taps into implicit semantic structure that triples share. Balancing by relation alone can fall short when certain relations simply do not carry enough triples to spread refinement effort adequately. AIKA therefore groups triples according to their embeddings, so that the members of a cluster are semantically close to one another. The benefit of a correction can then spread through a cluster even across differing explicit relations. The steps of clustered sampling are outlined in Algorithm 3. Embeddings are

Algorithm 3 Clustered Sampling

Input: KG $\mathcal{G}$, Computed score $\mathcal{X}$, Trained embedding model M , Number of clusters N_c , Sampling size N_s
Output: Sampled triples $\mathcal{S}$

- 1: $\mathcal{S} = \emptyset$
- 2: $N_{pc} \leftarrow N_s / N_c$ $\triangleright$ Number of samples per cluster
- 3: $\mathbf{Z} \leftarrow M(\mathcal{G})$ $\triangleright$ Triple embeddings
- 4: $\mathcal{C} \leftarrow \text{k-means++}(\mathbf{Z}, N_c)$ $\triangleright$ Cluster assignments
- 5: **for** each cluster c in $\mathcal{C}$ **do**
- 6: $\mathcal{X}_c \leftarrow$ Score of triples in cluster c
- 7: $\mathcal{B} \leftarrow \text{argsort}(\mathcal{X}_c)[1:N_{pc}]$
- 8: $\mathcal{S} = \mathcal{S} \cup \mathcal{B}$
- 9: **end for**
- 10: **return** $\mathcal{S}$

first computed for every triple and partitioned with the k-means++ clustering algorithm [2]. Then N_{pc} triples are drawn from each cluster following their score ranking. Clustered sampling, like its balanced counterpart, is paired with both the PS and UC strategies.

3.3 Experiments

3.3.1 Experimental Setup

We assess AIKA through two complementary experimental protocols that work in tandem: a simulated setting and a human-in-the-loop setting. The simulated setting replaces manual triple rectification with an automatic emulation, which permits controlled experimentation at scale. We rely on it to gauge how well AIKA raises KG accuracy and to study the influence that the KGE backbone and the sampling strategy exert on performance. Under the human-in-the-loop setting, rectification is in-

Table 3.1: Summary of the benchmark KG datasets.

Dataset	# Relation	# Entity	Train	Valid	Test
FB15K237	237	14,541	272,115	17,535	20,466
WikiMovie300K	294	36,001	240,000	23,310	23,373
WikiMovie1M	394	104,500	950,000	18,912	18,984
UMLS	46	135	5,216	652	661

stead carried out by human oracles working under several conditions, which lets us observe how the human factor shapes AIKA’s accuracy gains in practice.

Datasets

Three benchmark KGs serve as our testbeds: FB15K237 [84], WikiMovie [64], and UMLS [10]. FB15K237, derived from Freebase, is among the most popular benchmarks in KG research. WikiMovie is a movie-oriented slice of Wikidata covering facts about directors, actors, genres, and the like; it exists in two variants—WikiMovie300K, which spans 286,683 triples over 294 relations and 36,001 entities, and WikiMovie1M, which spans 987,896 triples over 394 relations and 104,500 entities. Its large inventories of relations and entities make WikiMovie a good proxy for realistic conditions, which is why we include it. UMLS originates from the Unified Medical Language System and organizes biomedical concepts into 135 semantic types connected by 46 relations. We reserve UMLS for the human-in-the-loop experiments, where deeper domain expertise is called for. The statistics of all datasets are collected in Table 3.1. Since the benchmarks can reasonably be assumed to consist almost entirely of valid triples, the noisy starting KG $\mathcal{G}_{\text{initial}}$ for AIKA is created by injecting synthetic errors. Starting from a correct triple (h, r, t) , either the head or the tail is swapped for

another entity, which yields (h', r, t) or (h, r, t') . To keep such errors plausible, the replacement h' (or t') must already occur as a head (or tail) of relation r somewhere else in the graph. Every corrupted triple generated in this manner is verified to be absent from the original KG. Two noise levels are considered, 10% and 20%. An untouched copy of the original triples is additionally kept as ground truth, allowing oracle corrections to be simulated. The validation and test partitions are held out for the downstream-task evaluation.

KG Embedding Models

AIKA’s backbone can be drawn from either of two established KGE families, translational distance models and semantic matching models, and we experiment with representatives of both. Models of these families are lightweight yet competitive, offering fast computation and solid accuracy while drawing only on the KG’s own content. Concretely, TransE [4], TorusE [13], DistMult [99], and ComplEx [87] serve as base models. TransE, the canonical translational distance method, places entities and relations in a vector space and requires each triple to satisfy the translation property $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$. TorusE removes the regularization problem that affects TransE by working in an n -dimensional torus space; under the mapping $x \mapsto [x]$, its embeddings are trained to obey the translation $[\mathbf{h}] + [\mathbf{r}] = [\mathbf{t}]$. DistMult, from the semantic matching family, evaluates each triple with a bilinear form built on a diagonal relation matrix, $\mathbf{h}^\top \mathbf{M}_r \mathbf{t}$. ComplEx generalizes DistMult to complex-valued embeddings; by moving entities and relations into a complex space, it gains the capacity to handle asymmetric relations. We emphasize that AIKA places no restriction on the backbone, so alterna-

tive KGE models may be substituted to match the user’s requirements and computing environment. As a demonstration of this flexibility, PALT [75], an LM-based method, is added as a more modern backbone. PALT transfers a pre-trained language model to the KG task via a parameter-efficient encoder that touches only a small fraction of the weights. By sidestepping full fine-tuning, the approach cuts computational cost considerably—a property well matched to the repeated retraining inherent in AIKA’s AL loop.

Sampling Strategies

The following acquisition schemes are compared for choosing which suspicious triples AIKA submits for rectification:

- **RD:** Triples are drawn from $\mathcal{G}$ uniformly at random.
- **PS:** The triples whose plausibility scores are smallest are chosen.
- **PS-balance:** PS combined with a relation-level balanced quota.
- **PS-cluster:** PS augmented with clustering over triple semantics.
- **UC:** The triples exhibiting the greatest uncertainty are chosen.
- **UC-balance:** UC combined with a relation-level balanced quota.
- **UC-cluster:** UC augmented with clustering over triple semantics.

Evaluation Metrics

The first metric, KG Accuracy (Acc), gives the share of triples in the KG that are correct; we track its value after every iteration.

$$Acc = \frac{|\mathcal{G}^{correct}|}{|\mathcal{G}|} \quad (3.9)$$

with $\mathcal{G}^{correct}$ standing for the correct portion of $\mathcal{G}$. The second metric, Sampling Efficiency (Eff_{samp}), quantifies how well a sampling strategy performs. It expresses the fraction of the selected triples that prove to be genuinely erroneous:

$$Eff_{samp} = \frac{\sum_{k=1}^K \frac{|\mathcal{S}_k^{incorrect}|}{|\mathcal{S}_k|}}{K} \quad (3.10)$$

where $\mathcal{S}_k^{incorrect}$ is the set of erroneous triples contained in the sample $\mathcal{S}_k$ drawn at iteration k . Larger Eff_{samp} values mean human effort is spent more productively, stretching the limited AL budget further. To connect accuracy gains with downstream utility, link prediction experiments are additionally carried out. Their outcomes are reported in terms of MRR (Mean Reciprocal Rank) together with Hits@N. Evaluation follows the filtered protocol: before the ranked list is produced, every corrupted triple is filtered out of the Train, Valid, and Test sets.

Implementation Details

Our implementation is written in Python on top of the open-source TorchKGE library [5], together with the official code release of PALT [75]. The four embedding backbones—TransE, TorusE, DistMult, and ComplEx—use an embedding dimension

of 200 with 25 negative samples per positive. Optimization relies on Adam [39], with the learning rate varied between $1e-6$ and $1e-3$. We tune the training batch size over $\{2048, 4096, 8192\}$ and the scoring batch size over $\{128, 256, 512\}$. Each backbone runs for at most 500 epochs, and triple scores are taken from the final-epoch output. The PALT backbone builds on pre-trained BERT-base [11] and is optimized with AdamW [53]. Its batch sizes are fixed at 32 for training and 16 for scoring. PALT runs for at most 30 epochs, again with triple scores taken from the final-epoch output. The remaining hyper-parameters are left at the values of the original release. A GeForce RTX 3090 GPU hosted all experiments.

3.3.2 Experimental Results: Simulated Setting

We now turn to the simulated evaluation of AIKA’s effectiveness. Here, rectification is emulated automatically by consulting the predefined ground-truth triples. Every reported figure is the mean of five runs, each launched from a different random initialization. The evaluation is organized around four research questions:

- RQ1: How large an accuracy gain can AIKA deliver on a KG?
- RQ2: How is AIKA’s effectiveness influenced by the choice of KGE backbone?
- RQ3: What role does the sampling strategy play in the achieved accuracy gains?
- RQ4: What influence do the sampling strategies have on downstream task performance?

Table 3.2: Gains in KG accuracy obtained on the FB15K237 and WikiMovie300K datasets, where R indicates the fraction of erroneous triples contained in each dataset. Sampling sizes of 1,000 (R=0.1) and 2,000 (R=0.2) are used, and clustered sampling operates with 500 clusters.

		FB15K237				WikiMovie300K			
		$R = 0.1$		$R = 0.2$		$R = 0.1$		$R = 0.2$	
KGE Model	Sampling	Acc_{Δ}	Eff_{samp}	Acc_{Δ}	Eff_{samp}	Acc_{Δ}	Eff_{samp}	Acc_{Δ}	Eff_{samp}
TransE	RD	0.9102 _($\uparrow$0.0102)	0.0929	0.8399 _($\uparrow$0.0399)	0.1811	0.9099 _($\uparrow$0.0099)	0.0953	0.8371 _($\uparrow$0.0371)	0.1783
	PS	0.9819 _($\uparrow$0.0819)	0.7429	0.9681 _($\uparrow$0.1681)	0.7626	<u>0.9651</u> _($\uparrow$0.0651)	<u>0.6252</u>	<u>0.9372</u> _($\uparrow$0.1372)	<u>0.6589</u>
	PS-balance	0.9479 _($\uparrow$0.0479)	0.4349	0.9316 _($\uparrow$0.1316)	0.5970	0.9392 _($\uparrow$0.0392)	0.3768	0.8987 _($\uparrow$0.0987)	0.4712
	PS-cluster	<u>0.9817</u> _($\uparrow$0.0817)	<u>0.7406</u>	<u>0.9680</u> _($\uparrow$0.1680)	<u>0.7621</u>	0.9653 _($\uparrow$0.0653)	0.6264	0.9375 _($\uparrow$0.1375)	0.6601
	UC	0.80 ₍₋₎	-	0.80 ₍₋₎	-	0.80 ₍₋₎	-	0.80 ₍₋₎	-
	UC-balance	0.80 ₍₋₎	-	0.80 ₍₋₎	-	0.80 ₍₋₎	-	0.80 ₍₋₎	-
	UC-cluster	0.80 ₍₋₎	-	0.80 ₍₋₎	-	0.80 ₍₋₎	-	0.80 ₍₋₎	-
TorusE	RD	0.9101 _($\uparrow$0.0101)	0.0918	0.8393 _($\uparrow$0.0393)	0.1785	0.9097 _($\uparrow$0.0097)	0.0926	0.8377 _($\uparrow$0.0377)	0.1812
	PS	<u>0.9465</u> _($\uparrow$0.0465)	<u>0.4220</u>	0.9318 _($\uparrow$0.1318)	0.5982	<u>0.9314</u> _($\uparrow$0.0314)	<u>0.3014</u>	<u>0.8874</u> _($\uparrow$0.0874)	<u>0.4197</u>
	PS-balance	0.9392 _($\uparrow$0.0392)	0.3552	0.9191 _($\uparrow$0.1191)	0.5404	0.9229 _($\uparrow$0.0229)	0.2202	0.8652 _($\uparrow$0.0652)	0.3131
	PS-cluster	0.9469 _($\uparrow$0.0469)	0.4251	<u>0.9314</u> _($\uparrow$0.1314)	<u>0.5962</u>	0.9315 _($\uparrow$0.0315)	0.3021	0.8889 _($\uparrow$0.0889)	0.4271
	UC	0.80 ₍₋₎	-	0.80 ₍₋₎	-	0.80 ₍₋₎	-	0.80 ₍₋₎	-
	UC-balance	0.80 ₍₋₎	-	0.80 ₍₋₎	-	0.80 ₍₋₎	-	0.80 ₍₋₎	-
	UC-cluster	0.80 ₍₋₎	-	0.80 ₍₋₎	-	0.80 ₍₋₎	-	0.80 ₍₋₎	-
DistMult	RD	0.9102 _($\uparrow$0.0102)	0.0932	0.8395 _($\uparrow$0.0395)	0.1791	0.9101 _($\uparrow$0.0101)	0.0973	0.8382 _($\uparrow$0.0382)	0.1835
	PS	0.9640 _($\uparrow$0.0640)	0.5807	0.9499 _($\uparrow$0.1499)	0.6800	0.9382 _($\uparrow$0.0382)	0.3664	0.8785 _($\uparrow$0.0785)	0.3772
	PS-balance	0.9470 _($\uparrow$0.0470)	0.4259	0.9327 _($\uparrow$0.1327)	0.6020	0.9335 _($\uparrow$0.0335)	0.3218	0.8720 _($\uparrow$0.0720)	0.3458
	PS-cluster	0.9634 _($\uparrow$0.0634)	0.5751	0.9505 _($\uparrow$0.1505)	0.6826	<u>0.9397</u> _($\uparrow$0.0397)	<u>0.3809</u>	<u>0.8802</u> _($\uparrow$0.0802)	<u>0.3853</u>
	UC	<u>0.9666</u> _($\uparrow$0.0666)	<u>0.6044</u>	<u>0.9569</u> _($\uparrow$0.1569)	<u>0.7117</u>	0.9393 _($\uparrow$0.0393)	0.3774	0.8787 _($\uparrow$0.0787)	0.3780
	UC-balance	0.9468 _($\uparrow$0.0468)	0.4245	0.9335 _($\uparrow$0.1335)	0.6056	0.9334 _($\uparrow$0.0334)	0.3208	0.8727 _($\uparrow$0.0727)	0.3490
	UC-cluster	0.9688 _($\uparrow$0.0688)	0.6236	0.9593 _($\uparrow$0.1593)	0.7228	0.9400 _($\uparrow$0.0400)	0.3844	0.8806 _($\uparrow$0.0806)	0.3873
ComplEx	RD	0.9106 _($\uparrow$0.0106)	0.0959	0.8402 _($\uparrow$0.0402)	0.1826	0.9098 _($\uparrow$0.0098)	0.0934	0.8373 _($\uparrow$0.0373)	0.1793
	PS	0.9642 _($\uparrow$0.0642)	0.5825	0.9357 _($\uparrow$0.1357)	0.6156	0.9335 _($\uparrow$0.0335)	0.3218	0.8915 _($\uparrow$0.0915)	0.4395
	PS-balance	0.9471 _($\uparrow$0.0471)	0.4272	0.9283 _($\uparrow$0.1283)	0.5821	0.9291 _($\uparrow$0.0291)	0.2794	0.8797 _($\uparrow$0.0797)	0.3829
	PS-cluster	0.9619 _($\uparrow$0.0619)	0.5611	0.9373 _($\uparrow$0.1373)	0.6231	0.9343 _($\uparrow$0.0343)	0.3289	0.8910 _($\uparrow$0.0910)	0.4372
	UC	<u>0.9696</u> _($\uparrow$0.0696)	<u>0.6313</u>	<u>0.9414</u> _($\uparrow$0.1414)	<u>0.6414</u>	0.9370 _($\uparrow$0.0370)	0.3552	0.8932 _($\uparrow$0.0932)	0.4475
	UC-balance	0.9477 _($\uparrow$0.0477)	0.4323	0.9283 _($\uparrow$0.1283)	0.5822	0.9269 _($\uparrow$0.0269)	0.2583	0.8794 _($\uparrow$0.0794)	0.3812
	UC-cluster	0.9707 _($\uparrow$0.0707)	0.6412	0.9437 _($\uparrow$0.1437)	0.6520	<u>0.9366</u> _($\uparrow$0.0366)	<u>0.3516</u>	<u>0.8923</u> _($\uparrow$0.0923)	<u>0.4433</u>
PALT	RD	0.9104 _($\uparrow$0.0104)	0.0943	0.8397 _($\uparrow$0.0397)	0.1802	0.9102 _($\uparrow$0.0102)	0.0979	0.8382 _($\uparrow$0.0382)	0.1835
	PS	0.9651 _($\uparrow$0.0651)	0.5904	0.9551 _($\uparrow$0.1551)	0.7035	0.9403 _($\uparrow$0.0403)	0.3868	0.8942 _($\uparrow$0.0942)	0.4522
	PS-balance	0.9474 _($\uparrow$0.0474)	0.4299	0.9394 _($\uparrow$0.1394)	0.6323	0.9321 _($\uparrow$0.0321)	0.3081	0.8833 _($\uparrow$0.0833)	0.4000
	PS-cluster	0.9645 _($\uparrow$0.0645)	0.5850	0.9564 _($\uparrow$0.1564)	0.7094	0.9447 _($\uparrow$0.0447)	0.4291	0.8950 _($\uparrow$0.0950)	0.4563
	UC	<u>0.9712</u> _($\uparrow$0.0712)	<u>0.6458</u>	<u>0.9589</u> _($\uparrow$0.1589)	<u>0.7205</u>	<u>0.9492</u> _($\uparrow$0.0492)	<u>0.4723</u>	<u>0.8984</u> _($\uparrow$0.0984)	<u>0.4726</u>
	UC-balance	0.9502 _($\uparrow$0.0502)	0.4553	0.9407 _($\uparrow$0.1407)	0.6383	0.9308 _($\uparrow$0.0308)	0.2956	0.8827 _($\uparrow$0.0827)	0.3971
	UC-cluster	0.9781 _($\uparrow$0.0781)	0.7084	0.9650 _($\uparrow$0.1650)	0.7482	0.9514 _($\uparrow$0.0514)	0.4934	0.9011 _($\uparrow$0.1011)	0.4854

Results on KG Accuracy Improvement (RQ1)

The first question concerns AIKA’s overall ability to enhance KG accuracy. Table 3.2 confirms that accuracy rises consistently, whichever dataset or backbone is used. Taking FB15K237 at a 90% starting accuracy ($R = 0.1$), the best gains reach 8.19%p for TransE, 4.69%p for TorusE, 6.88%p for DistMult, 7.07%p for ComplEx, and 7.81%p for PALT. With the lower starting point of 80% ($R = 0.2$), the corresponding gains increase to 16.81%p, 13.18%p, 15.93%p, 14.37%p, and 16.50%p.

Comparable benefits emerge on WikiMovie300K, a graph richer in entities and relations than FB15K237. At the 90% starting level ($R = 0.1$), peak improvements amount to 6.53%p (TransE), 3.15%p (TorusE), 4.00%p (DistMult), 3.70%p (ComplEx), and 5.14%p (PALT). When accuracy instead starts at 80% ($R = 0.2$), they climb to 13.75%p, 8.89%p, 8.06%p, 9.32%p, and 10.11%p. Overall, the evidence indicates that AIKA transfers well across heterogeneous KGE backbones and delivers meaningful accuracy gains even for large, structurally intricate KGs.

Impacts of KGE Models on KG Accuracy (RQ2)

Turning to the influence of the backbone, Table 3.2 shows that every model profits from AIKA, yet the largest average gains belong to TransE. Its improvements are also comparatively steady across datasets and noise levels, and it holds its own even against the LM-based PALT. Evidently, greater model complexity is not a prerequisite for accuracy improvement within AIKA—plain KGE models already make strong backbones.

A further observation is that the uncertainty criterion (UC) fails to yield reliable

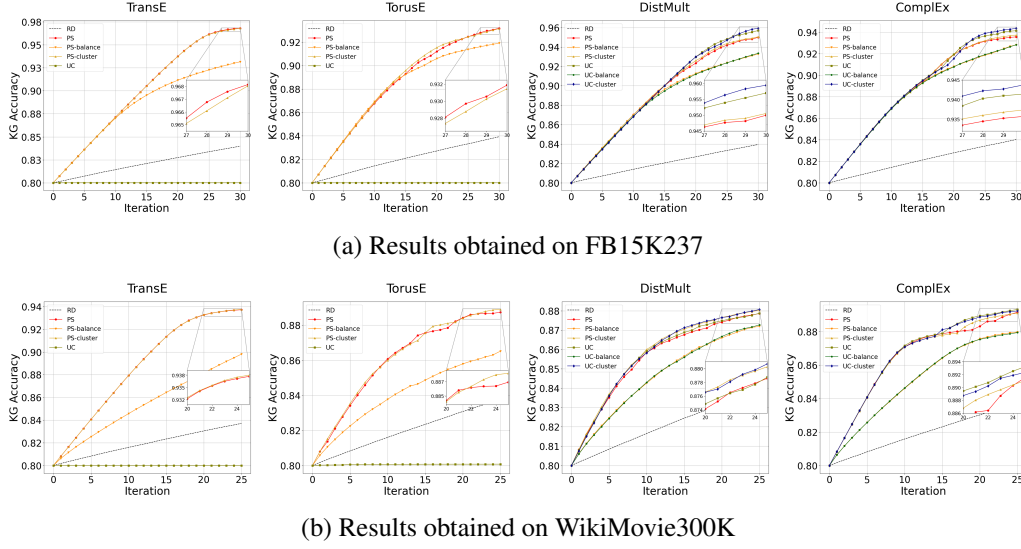


Figure 3.4: Progression of KG accuracy over the AL iterations, using a sampling size of 2,000 and 500 clusters.

accuracy gains when paired with translational distance models. The likely cause lies in how these models score triples: they chiefly impose translation constraints (e.g., $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$) and lack any explicit parameterization from which uncertainty could naturally be estimated. Semantic matching models behave differently; DistMult, for instance, embeds the relation-specific matrix $\mathbf{M}_r$ within its scoring function $\mathbf{h}^\top \mathbf{M}_r \mathbf{t}$, and this extra trainable structure helps the model express uncertainty in its outputs. Uncertainty-driven selection therefore tends to be more fruitful with semantic matching backbones than with translational ones.

Impacts of Sampling Strategies on KG Accuracy (RQ3)

Next we examine how the choice of sampling strategy shapes the accuracy gains. Figures 3.4, 3.5, and 3.6 trace KG accuracy across AL iterations for a KG beginning

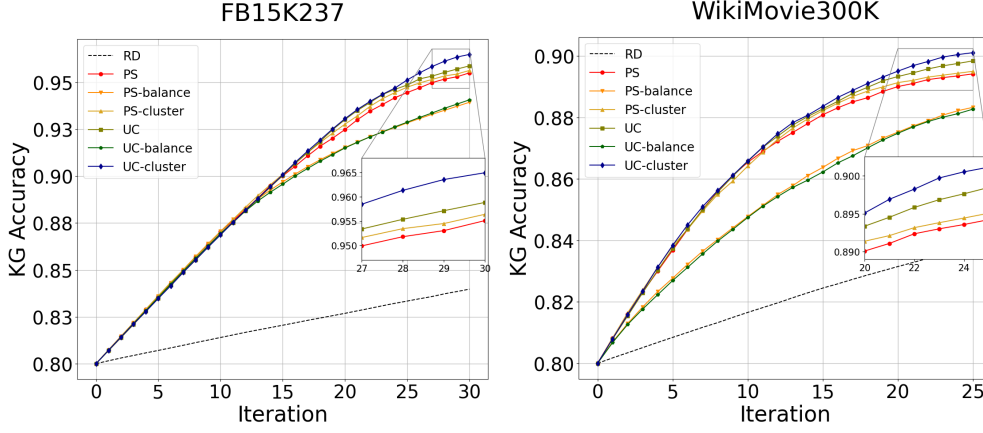


Figure 3.5: Progression of KG accuracy over the AL iterations when PALT serves as the backbone, with a sampling size of 2,000 and 500 clusters.

at 80% accuracy ($R = 0.2$). On every dataset, the score-driven strategies PS and UC clearly beat RD, pairing larger accuracy gains with markedly better sampling efficiency. Their efficiency is in fact no less than double that of RD, showing that score-guided selection stretches a fixed annotation budget considerably further.

Between the two score-driven strategies, UC tends to come out ahead on both accuracy and efficiency. Figures 3.4 and 3.5 show UC surpassing PS under DistMult, ComplEx, and PALT. The two behave much alike through the early and middle iterations, but UC pulls ahead late in the process—an indication that, once the remaining errors become hard to tell apart, uncertainty conveys more signal than the raw plausibility score. Figure 3.6 reveals the same pattern on WikiMovie1M: even at that scale, UC edges out PS during the closing iterations.

We then assess the distribution-aware variants motivated by the long-tailed relation statistics of real-world KGs. Figure 3.7 contrasts the relation distribution of the

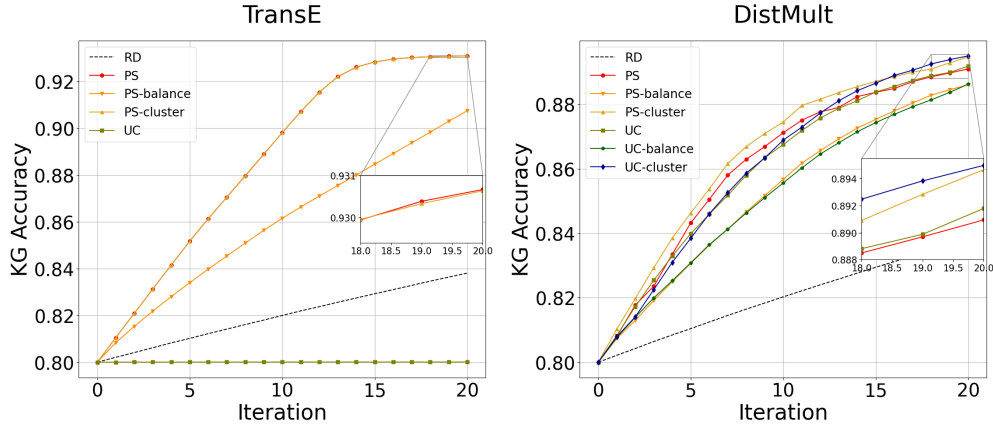


Figure 3.6: Progression of KG accuracy over the AL iterations on WikiMovie1M, with a sampling size of 10,000 and 500 clusters.

initial FB15K237 KG with the distributions of the triples that each strategy selects for repair. RD, PS, and UC channel most corrections toward high-frequency relations, essentially reproducing the original long tail. PS-balance and UC-balance, by comparison, spread the corrections far more evenly over the relation inventory. Figures 3.4 and 3.5 indicate, though, that this evenness comes at the price of slightly smaller absolute accuracy gains relative to unconstrained score-based selection. The clustered variants strike a middle ground: their improvements are spread more evenly than RD's, yet their selection profile stays close to plain PS and UC. Strikingly, PS-cluster and UC-cluster also outgain their non-clustered counterparts by a clear margin, evidence that leveraging semantic similarity among triples makes finding and fixing errors more effective.

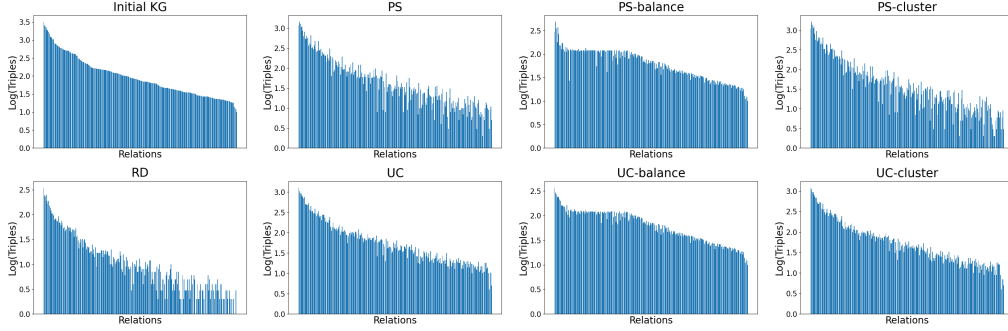


Figure 3.7: A contrast between the relation makeup of the initial KG and the way triples refined under each AL sampling strategy are distributed, with relations running from the most to the least frequent in the initial KG.

Impacts of Sampling Strategies on Downstream Tasks (RQ4)

To see whether the accuracy improvements delivered by each sampling strategy also benefit downstream applications, we carry out link prediction experiments. The setup uses FB15K237 with 20% injected incorrect triples, and AIKA runs on a DistMult backbone. A separate TransE model is then trained to handle the link prediction task itself. The implementation relies on OpenKE¹ with its default hyperparameter configuration.

Consider first the left half of Table 3.3, which covers every relation in FB15K237. Skipping AIKA entirely (No AL) lowers Hit@10 by roughly 0.1p relative to training on the clean KG (Original). Once AIKA is applied, MRR and Hit@N both recover, and the size of the recovery broadly tracks the accuracy gain that each sampling strategy delivered. In short, the quality improvements AIKA brings to the KG are reflected directly in stronger downstream link prediction.

¹<https://github.com/thunlp/OpenKE>

Table 3.3: Link prediction results on FB15K237. Scores computed over all triples appear on the left, while the right side covers only triples belonging to the minor 50 relations.

	All Relations				Minor 50 Relations			
Sampling	MRR	Hits@10	Hits@3	Hits@1	MRR	Hits@10	Hits@3	Hits@1
Original (100%)	0.289	0.476	0.327	0.194	0.414	0.611	0.475	0.306
No AL (80%)	0.199	0.380	0.234	0.105	0.308	0.536	0.381	0.188
RD	0.211	0.393	0.245	0.117	0.327	0.561	0.392	0.202
PS	0.257	0.442	0.291	0.164	0.388	0.579	0.447	0.280
PS-balance	0.234	0.420	0.271	0.137	0.390	<u>0.587</u>	0.461	0.284
PS-cluster	0.259	0.444	0.293	0.167	0.391	0.588	0.458	0.281
UC	<u>0.261</u>	<u>0.445</u>	0.298	<u>0.169</u>	<u>0.393</u>	0.582	0.454	0.283
UC-balance	0.232	0.416	0.266	0.137	0.391	0.583	<u>0.459</u>	0.286
UC-cluster	0.263	0.450	<u>0.295</u>	0.170	0.394	0.583	0.455	<u>0.285</u>

The right half of Table 3.3 isolates triples from the 50 rarest relations of the original FB15K237. Mirroring the overall trend, AIKA again lifts both MRR and Hit@N. What stands out is the strength of the distribution-aware strategies—balanced and clustered sampling—on these minor relations. Remarkably, the balanced variants achieve the strongest link prediction results even though their raw KG-accuracy gains trail those of the other strategies. Relation-aware sampling thus appears to counteract long-tail imbalance effectively, raising the quality of triples tied to rare relations and thereby amplifying the downstream payoff.

Further Analysis of Sampling Strategies

A set of supplementary experiments probes the sampling strategies in greater depth. The first examines how sensitive clustered sampling is to the cluster count. Figure 3.8 plots the resulting KG accuracy gains as the number of clusters is varied. Empirically,

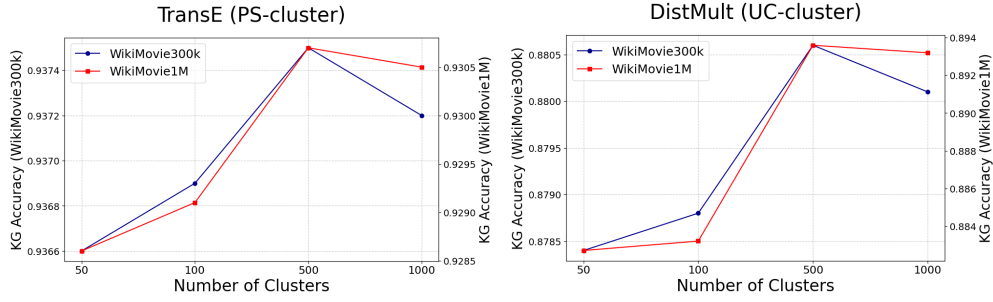


Figure 3.8: Effect of varying the cluster count on the KG accuracy gains achieved by clustered sampling.

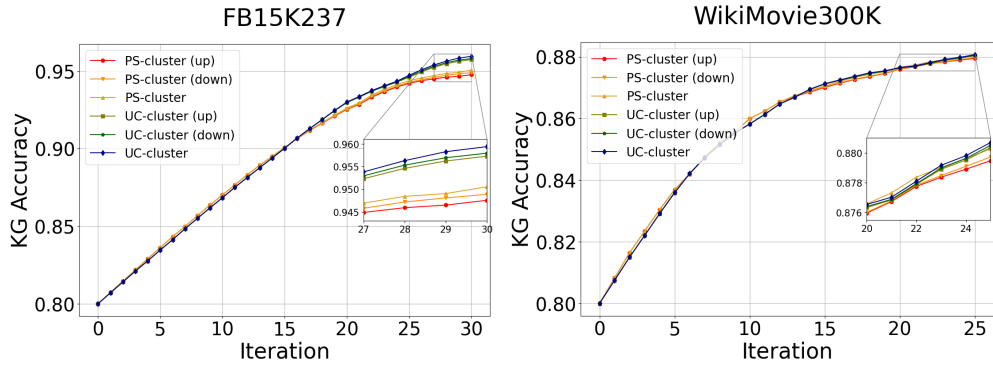


Figure 3.9: KG accuracy over the AL iterations when the number of clusters is adjusted dynamically during the process.

the strategy works best whenever the cluster count is no smaller than the number of relations in the KG. For PS-cluster and UC-cluster alike, gains stay consistently higher beyond that threshold and keep growing as clusters are added. Semantic partitioning, in other words, can guide sampling decisions more effectively than partitioning by relation labels. The benefit is more visible for UC than for PS, hinting that uncertainty is an especially informative signal inside semantically coherent clusters. The advantage also widens on WikiMovie1M relative to WikiMovie300K, implying that clustered sampling pays off most on bigger KGs with a richer mix of relations

and entity types.

The second experiment asks whether adjusting the cluster count on the fly alters the accuracy outcome. Using DistMult and a default of 500 clusters, we double (up) or halve (down) the count once half of the AL iterations have elapsed. Figure 3.9 shows that the overall trajectory of improvement barely changes under either adjustment. Halving the cluster count performs marginally better than doubling it, though the difference is small. UC also proves more robust to these adjustments than PS. Taken together, the results suggest that on-the-fly cluster adjustment gives practitioners a workable lever for trading computational cost against sampling performance in AIKA.

3.3.3 Experimental Results: Human-in-the-loop Setting

We now describe experiments in which real annotators participate, so as to study the role of human involvement in AIKA. Since the oracles are the ones who repair the selected triples, their expertise bears directly on the accuracy the KG ultimately reaches and on how efficiently the refinement proceeds. The stakes are highest for domain-specific KGs, whose triples often cannot be corrected without specialized knowledge. We therefore analyze how the oracle’s expertise level shapes the accuracy improvement that AIKA delivers.

Expertise Level of Human Oracles

Human domain competence relevant to KG improvement is stratified into three tiers: Expert, Semi-expert, and Non-expert. An expert holds specialized knowledge within a

specific field or industry. Because that knowledge is systematically organized, an expert can give dependable answers built on a thorough command of the field. A semi-expert possesses advanced—but not yet fully specialized—knowledge of the field in question. Such a person cannot deploy structured domain knowledge unaided, yet can interpret and apply it productively whenever the relevant context is supplied. A non-expert, finally, lacks the specialized knowledge that tasks in a given domain demand.

These tiers are translated into concrete oracle conditions for the human-in-the-loop rectification task:

- Non-expert: Annotators receive neither contextual nor structured knowledge while rectifying triples.
- Semi-expert: Each triple is accompanied by descriptive text covering its head, relation, and tail. Annotators taking this role rectify triples with the help of the aligned text.
- Expert: Rectification is supported by an external KG, which annotators taking this role can browse through a dedicated exploration tool.

The evaluation target for studying these expertise effects is UMLS, a biomedical-domain KG. Annotators acting as semi-experts rely on Yao et al.’s [102] UMLS triple-text alignment resource as their reference material. In the expert condition, the UMLS Semantic Network Browser² plays the role of the external KG exploration tool.

²<https://uts.nlm.nih.gov/uts/umls/semantic-network>

Annotators and Guidelines

Three graduate students took part in the human-in-the-loop experiments as annotators. All were fluent in English and well versed in KGs and ontologies, but none brought expertise in the biomedical field. Each annotator’s job was to fix the wrong entity in every triple presented, operating under whichever expertise condition had been assigned. To streamline the evaluation, the erroneous entity in each triple was pointed out to the annotators in advance. A search function over the current state of the target KG was additionally made available, ensuring that annotators would not enter rectified triples that already existed.

Additional Settings

The configuration adopted for these experiments is as follows: TransE serves as the KGE backbone, PS-cluster as the sampling strategy, the cluster count is 100, the sampling size is 100, and 10 AL iterations are run. Erroneous triples are injected at a 0.2 noise ratio.

Rectification quality at each expertise level is quantified through annotation efficiency (Eff_{anno}):

$$Eff_{anno} = \frac{1}{K} \sum_{k=1}^K \frac{|\mathcal{S}_k^{rectified}|}{|\mathcal{S}_k^{incorrect}|}, \quad (3.11)$$

where, for the sample $\mathcal{S}_k$ drawn at AL iteration k , $\mathcal{S}_k^{incorrect}$ is its subset of erroneous triples and $\mathcal{S}_k^{rectified}$ the subset the oracle repaired correctly. The larger Eff_{anno} becomes, the greater the share of errors the oracle fixes—that is, the better the annotation quality.

Table 3.4: KG accuracy improvement attained by AIKA when human oracles with different degrees of domain expertise carry out the rectification.

Lv. of Expertise	$Acc_{(\Delta)}$	Eff_{anno}	$instance/sec$
Non-expert	0.8092 _(↑0.0092)	0.0545	-
Semi-expert	0.8553 _(↑0.0553)	0.5214	178.32
Expert	0.8857 _(↑0.0857)	0.8307	41.64

Impacts of Human Oracles on KG Accuracy

The dependence of accuracy improvement on oracle expertise is summarized in Table 3.4. Under the non-expert condition, accuracy improves by only 0.92%p while annotation efficiency stands at 0.0545. Meaningful gains, in other words, are hard to obtain from annotators who lack domain support. With semi-experts in the oracle role, the improvement climbs to 5.53%p at an annotation efficiency of 0.5214. Experts push these figures considerably higher, reaching an 8.57%p improvement together with an annotation efficiency of 0.8307. It is also worth noting that, on average, corrections took semi-experts about four times longer than experts. Together, the results demonstrate how strongly the oracle’s expertise conditions the framework’s overall performance. They likewise signal that monetary cost, time expenditure, and efficiency can all shift substantially with the expertise of the participating oracles. Matching the oracle pool to an appropriate expertise level is therefore a prerequisite for accuracy improvement that is both effective and economical under a fixed budget.

3.3.4 Discussions

Here we consider the factors that practitioners ought to weigh when putting the AL framework into practice for improving KG accuracy in realistic deployments. We further point out a number of shortcomings inherent to our method.

Cost of KG Accuracy Improvement

Identifying suspicious triples in AIKA presupposes that a KGE model has been trained on the target KG. Real-world KGs, however, often hold enormous numbers of triples, so fitting a backbone—and refitting it round after round—can consume considerable computation and time, a burden that only grows with larger graphs and more complex models. Selecting the backbone therefore deserves care: the choice should reflect both the characteristics of the KG at hand and the computing resources one can afford. Although outside the scope of this work, incremental training or distributed learning could be explored in the future to lighten the retraining load on large-scale KGs.

Human labor constitutes a second major expense, given that the oracles must rectify the selected triples correctly. Many KG facts are domain-specific rather than commonsense, and hiring annotators qualified for such material does not come cheap. Making the most of a bounded annotation budget consequently hinges on adopting sampling strategies whose sampling efficiency is as high as possible. The oracle’s expertise, moreover, bears directly on both how well and how quickly the refinement is completed. When planning a KG refinement effort in practice, the anticipated expertise of the participants therefore merits explicit consideration.

Goal of KG Accuracy Improvement

Which KGE model and sampling strategy suit best depends on what the KG improvement effort is meant to accomplish. If the priority is the highest possible overall accuracy, the natural pick is the model–strategy combination that has shown the largest accuracy gains. If the priority is instead to raise accuracy on triples of rarely occurring relations, a strongly generalizing model paired with a distribution-aware sampling strategy is the better fit. Practitioners should therefore settle on a KGE model and a sampling strategy only after clarifying the concrete objective of the intended refinement.

Limitations and Future Work

AIKA’s error identification draws exclusively on information internal to the KG. Yet many real KGs come with auxiliary signals—textual descriptions, provenance records, ontological schemas—that could sharpen both error detection and the subsequent refinement. Work on multimodal KGs likewise indicates that combining heterogeneous evidence benefits KG quality management. Extending AIKA to exploit such auxiliary information, and thereby to support more comprehensive refinement, is a direction we intend to pursue.

A further limitation is the framework’s dependence on human oracles for triple rectification, an arrangement that carries nontrivial time and labor costs. To ease this dependence, we plan to investigate automated assistants—reinforcement learning agents or large language models, for example—that could take over part of the rectification workload without sacrificing refinement quality.

3.4 Summary

This chapter introduced AIKA, an AL framework for raising KG accuracy when the annotation budget is tight. By repeatedly flagging suspicious triples and having them repaired through human-in-the-loop rectification, AIKA achieves accuracy gains at low cost. To boost sampling efficiency, we further devised distribution-aware sampling strategies tailored to the long-tailed relation statistics of real-world KGs. Extensive experiments show that AIKA together with these strategies reliably improves KG accuracy over a range of datasets and KGE backbones, with benefits that extend to downstream tasks. Drawing on this empirical analysis, we also offered practical guidance for bringing AL to bear on KG refinement.

4. Knowledge Graph Validation with Evidence-Grounded Reasoning

This chapter investigates KG validation, a task concerned with judging whether individual triples are factually sound. Because KGs encode real-world facts as structured triples, they act as a knowledge backbone for a variety of downstream applications, including question answering [28], recommendation [89], and dialogue generation [60]. The difficulty, however, is that real-world KGs are typically built and maintained by automated extraction pipelines, so erroneous triples are unavoidably introduced and can compromise the trustworthiness of systems that depend on the KG.

Earlier studies have tackled KG validation by drawing on signals internal to the KG or on external textual evidence gathered from corpora [25]. This research direction has since been pushed forward by LM-based techniques, which cast triple validity as a natural-language inference decision and predict it directly [102, 56, 111, 103]. Even so, arriving at dependable triple-level verdicts stays difficult whenever internal cues fall short or the retrieved evidence is noisy and offers only weak support.

As Figure 4.1 depicts, current LLM-based validators depend mainly on the structural regularities the model has learned from the KG together with the knowledge stored in its parameters. While these methods can make good use of relational patterns and parametric knowledge, the resulting judgments become unreliable whenever the target fact cannot be adequately deduced from such signals by themselves.

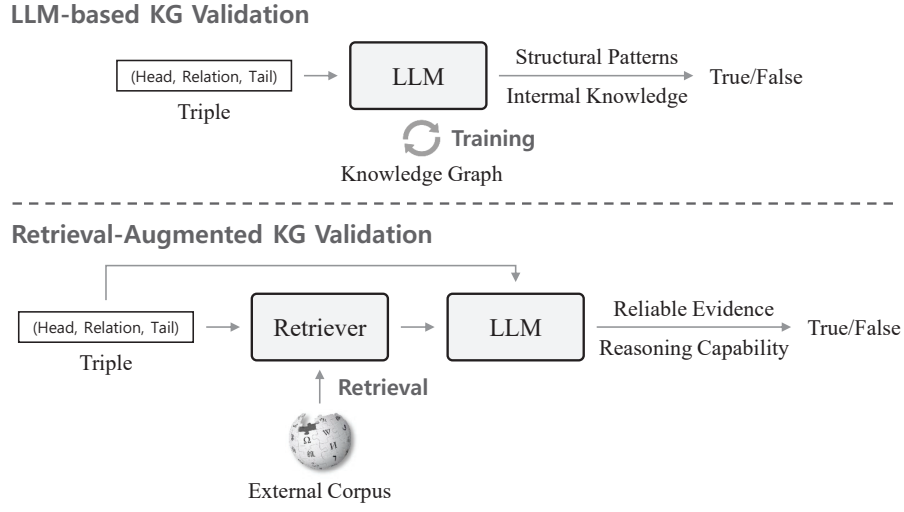


Figure 4.1: Contrasting conventional LLM-based KG validation with the retrieval-augmented alternative.

Retrieval-augmented KG validation follows a different approach: it introduces external textual evidence into the validation process, so the model can evaluate a triple using retrieved contexts alongside its own reasoning. This approach, however, is effective only when the retrieved documents actually supply trustworthy, decision-relevant evidence about the triple in question.

We tackle these issues with **RAVER**, a retrieval-augmented framework in which evidence retrieval is paired with evidence-grounded LLM reasoning. Given a target triple, RAVeR pulls semantically related documents and asks an LLM to render a validation verdict anchored in those retrieved contexts. Beyond this, it sharpens retrieval quality by tuning the retriever with feedback derived from the reasoning outcomes, yielding evidence retrieval that is better tailored to robust KG validation.

4.1 Preliminaries

4.1.1 Retrieval-Augmented Generation

Despite their impressive capabilities, LLMs remain prone to hallucination, producing inaccurate or invented content because their parametric knowledge is inherently bounded. To counter this, retrieval-augmented generation (RAG) anchors the generation process in external knowledge repositories [42], and it has shown clear value on knowledge-intensive tasks that demand factual accuracy and grounding in outside sources [23]. Under the RAG scheme, a retriever pulls pertinent documents from a sizable external corpus given a query, after which a generator emits an output conditioned on the retrieved material. Such a design lessens dependence on memorized knowledge and steers the model toward outputs that rest on explicit evidence. Here, we carry this paradigm over to KG validation, grounding each validation decision in external evidence so that the resulting judgments are more accurate and dependable than those relying on structural patterns or parametric knowledge alone.

4.1.2 Problem Definition

Knowledge Graph

We formalize a KG as $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{F})$, in which $\mathcal{E}$ denotes the collection of entities and $\mathcal{R}$ the collection of relations. The component $\mathcal{F}$ gathers the factual triples: every fact $f = (h, r, t) \in \mathcal{F}$ is composed of a head entity $h \in \mathcal{E}$, a relation $r \in \mathcal{R}$, and a tail entity $t \in \mathcal{E}$. A triple asserts that relation r links entities h and t , thereby packaging a single unit of factual knowledge inside the KG.

Retrieval-Augmented KG Validation

The goal of KG validation is to decide whether a triple $f = (h, r, t)$ holds as a factually correct statement. For this purpose, we employ an LLM as the validator, leveraging both its pre-trained knowledge and its reasoning ability to judge triple correctness. To make such LLM-based validation more reliable, we adopt a retrieval-augmented formulation in which external evidence documents are retrieved and the LLM’s reasoning is anchored in them. Concretely, for a triple f and an external document corpus $\mathcal{D}$, a retriever $\phi(\cdot)$ treats f as the query and yields a set of evidence documents $\mathcal{D}_f$:

$$\mathcal{D}_f = \phi(f, \mathcal{D}). \quad (4.1)$$

Such a retriever may be realized either through sparse lexical matching or through dense semantic representations. Conditioned on both the triple and the retrieved evidence, the LLM subsequently carries out evidence-grounded validation as shown below:

$$\text{LLM}(f, \mathcal{D}_f) \rightarrow \{\text{True}, \text{False}\}. \quad (4.2)$$

Because it reasons jointly over the triple and the retrieved evidence, this retrieval-augmented framework raises validation accuracy while delivering decisions that are more dependable and evidence-grounded.

4.1.3 Preliminary Experiments

A proper study of retrieval-augmented KG validation calls for looking at two things together: how well LLMs reason on their own, and how much the retrieved evidence

shifts that performance. With this in mind, we conducted a set of preliminary experiments examining the contribution of retrieval and the benefit of evidence-grounded reasoning in LLMs. To this end, we set up three experimental conditions, each supplying a different kind of evidence:

- *w/o Evidence*: Judging factuality from the triple alone, with no external evidence supplied at all.
- *w/ Random Evidence*: Judging with documents drawn at random as evidence, irrespective of whether they relate to the triple.
- *w/ Gold Evidence*: Judging with gold documents furnished as supporting evidence.

Because these settings hinge on triples being paired with their documents, we turned to DocRED [104], a document-level relation extraction corpus in which documents come matched with the triples they express. This built-in pairing directly provides gold evidence, making DocRED well suited to our preliminary study. From the DocRED development split we then drew 1,000 positive triples at random and constructed 1,000 negatives by corrupting tail entities, producing 2,000 candidate triples split evenly in a 1:1 ratio. We cast every triple together with its accompanying document into an instruction-style prompt and passed it to the LLM for evidence-grounded validation. The study used two widely adopted open-source LLMs, namely Llama3-8B [21] and Qwen2.5-7B [98].

The KG validation results across the three evidence conditions are reported in Figure 4.2. Under the *w/o Evidence* condition, each model reached a respectable level

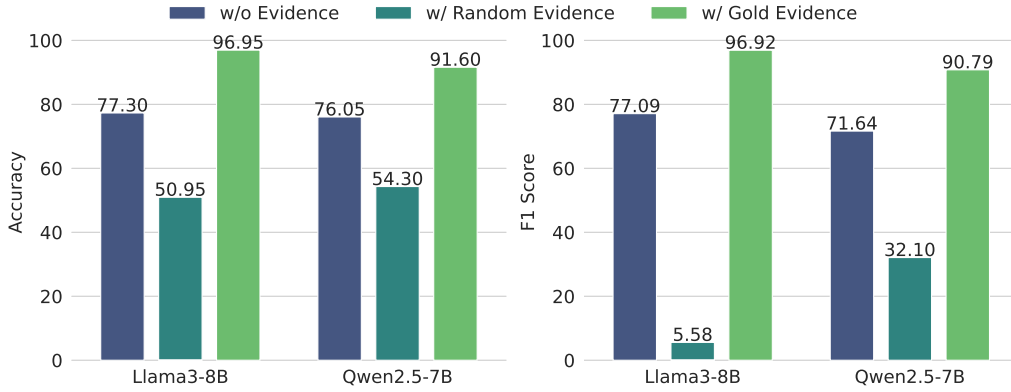


Figure 4.2: How different evidence conditions affect retrieval-augmented KG validation.

of performance drawing only on its pre-trained knowledge. Yet with no external evidence to lean on, their reasoning proved fragile and ran into obvious limits. Under the *w/ Random Evidence* condition, feeding in irrelevant documents caused performance to fall off steeply. This tells us that such evidence badly distracts the LLM’s reasoning and points to retrieval quality as a decisive factor. It further exposes the hazard of noisy retrieval, in which off-topic documents hurt validation rather than help it. The *w/ Gold Evidence* condition tells the opposite story: well-matched supporting documents produced striking gains. This confirms that top-quality evidence lets LLMs bring their full reasoning power to bear. Read as a whole, these observations establish that dependable evidence is indispensable for effective LLM reasoning and that retrieval quality is pivotal for KG validation, which is precisely what motivates the retrieval-augmented framework and retriever alignment developed in this work.

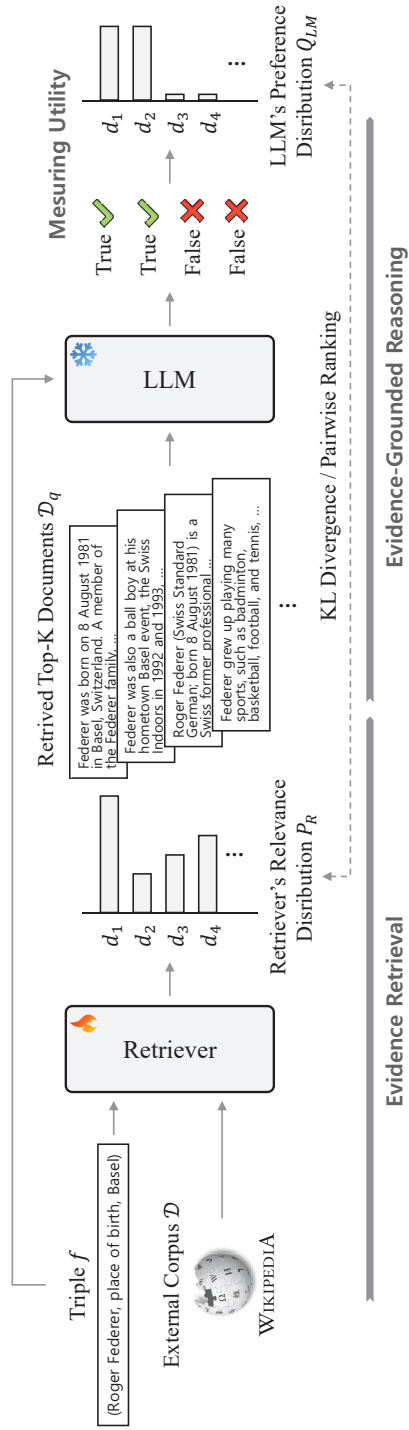


Figure 4.3: A high-level view of the proposed RAVER framework. The framework runs through two principal stages: (1) Evidence Retrieval, in which documents that are semantically pertinent are drawn from a large external corpus, and (2) Evidence-Grounded Reasoning, in which the LLM decides on triples using the retrieved evidence. The retriever is additionally optimized by gauging each document’s utility from validation outcomes and reshaping the retrieval preferences accordingly.

4.2 Proposed Approach: RAVER

In this section we present RAVER, a framework that performs retrieval-augmented KG validation by means of evidence-grounded LLM reasoning. RAVER is organized into two stages, as illustrated in Figure 4.3. During the evidence retrieval stage, a dense retriever surfaces documents that are relevant to the target triple. During the evidence-grounded reasoning stage, an LLM validator draws on the retrieved evidence to determine factual validity. For added reliability, RAVER gauges how useful each retrieved document is from the validation outcomes and then optimizes the retriever so that its retrieval preferences fall in line with the LLM’s judgments. Applied iteratively, this alignment keeps refining retrieval, which in turn produces better evidence and stronger validation performance.

4.2.1 Evidence Retrieval

For fetching documents relevant to validating a given triple, we rely on Contriever [30], a bi-encoder dense retriever well suited to semantic retrieval at scale. Concretely, for a triple $f = (h, r, t)$ and an external corpus $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$, we begin by forming a query $q = [h; r; t]$ that concatenates the textual forms of the head, relation, and tail. The query q and every document $d_i \in \mathcal{D}$ are then encoded independently by two distinct encoders:

$$\mathbf{q} = E_Q(q), \mathbf{d}_i = E_D(d_i), \quad (4.3)$$

in which E_Q and E_D are the query and document encoders, respectively, mapping their inputs into a common embedding space. At retrieval time, the similarity between the query and each document embedding is obtained as:

$$s(d_i) = \cos(\mathbf{q}, \mathbf{d}_i) = \frac{\mathbf{q} \cdot \mathbf{d}_i}{\|\mathbf{q}\| \|\mathbf{d}_i\|}. \quad (4.4)$$

The K documents scoring highest in similarity are then taken to constitute the retrieved evidence set:

$$\mathcal{D}_q = \text{TopK}_{d_i \in \mathcal{D}}(q, d_i). \quad (4.5)$$

To keep large-scale retrieval efficient, we pre-compute every document embedding and index them with FAISS [35].

4.2.2 Evidence-Grounded Reasoning

Once evidence retrieval is complete, the LLM carries out evidence-grounded reasoning to gauge whether a given triple is factually valid. Through this step the model reaches past its pre-trained knowledge, reading the contextual signals in the retrieved documents and treating them as the ground for its factual judgment. Following Eq. 4.2, the LLM receives the target triple f along with its retrieved evidence set $\mathcal{D}_q$, and outputs a binary decision $\hat{y} \in \{\text{True}, \text{False}\}$. To make reasoning easier, we arrange the triple $f = (h, r, t)$ and $\mathcal{D}_q$ into an instruction-style prompt that directly asks, on the basis of the supplied evidence, whether relation r holds between h and t . This design taps the LLM’s capacity to fuse external context with prior knowledge, enabling reasoning over implicit relational hints as well as cross-sentence and

cross-document signals. Anchoring the verdict in retrieved evidence lets the LLM deliver validation outcomes that are factually backed and easier to interpret. A sample prompt appears below:

Prompt Template for Retrieval-Augmented KG Validation

```
Based on the provided document, reason and determine whether the following
KG triple (head, relation, tail) is 'True' or 'False'.
Document: {retrieved documents}
Triple: {target triple}
Answer:
```

4.2.3 Retriever Alignment

To lift retrieval quality, RAVER tunes the dense retriever so that the way it ranks documents lines up more closely with the LLM’s evidence preferences. Drawing on earlier studies [70, 77], we adopt the notion of aligning retrieval with the LLM’s internal preferences, which lets the retriever adjust its relevance distribution according to signals extracted from the model’s validation decisions. Training pushes the retriever to hand out higher relevance scores to the documents that prove most helpful for KG validation, reshaping the relevance distribution so that it better mirrors document utility. Figure 4.4 illustrates this: documents that aid the LLM’s validation are assigned larger softmax probabilities, while the less useful ones are pushed down.

Computing Retrieval Likelihood

Given a query q and its top- K retrieved documents $\mathcal{D}_q = \{d_1, d_2, \dots, d_K\}$, the retriever derives a similarity score $s(d_i)$ for each d_i from the query-document cosine similarity computed at retrieval time. These scores are passed through a softmax to

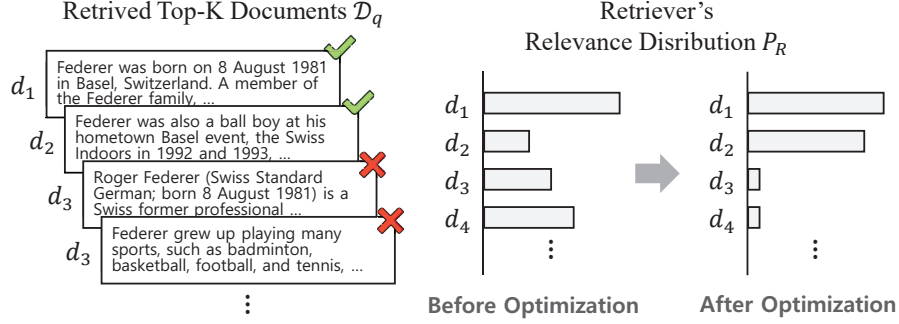


Figure 4.4: An illustration of how the retriever’s relevance distribution shifts from before to after alignment.

yield the retriever’s relevance distribution over the candidate set:

$$P_R(d_i | q) = \frac{e^{s(d_i)/\gamma}}{\sum_{j=1}^K e^{s(d_j)/\gamma}}, \quad (4.6)$$

where the temperature parameter γ governs how sharply peaked the distribution is. The distribution $P_R(d_i | q)$ that emerges encodes the retriever’s estimate of how relevant each document is, relative to the others, to the target triple.

Measuring Document Utility

To judge how useful each retrieved document is, RAVER quantifies the part it plays in the LLM’s validation. Every document d_i is supplied to the LLM on its own together with the triple f , and the LLM then reasons over this evidence to predict factual validity:

$$\hat{y}_i = \text{LLM}(f, d_i) \in \{\text{True}, \text{False}\}. \quad (4.7)$$

Writing y^* for the triple’s ground-truth label, we express each document’s utility as a binary signal:

$$u_i = \begin{cases} 1, & \text{if } \hat{y}_i = y^* \\ 0, & \text{otherwise.} \end{cases} \quad (4.8)$$

A utility of 1 signals that the document steers the LLM toward a correct judgment, while 0 marks it as unhelpful or even misleading. We then turn these utility scores into a smoothed document preference distribution that captures how useful each document is relative to the others in the top- K set:

$$Q_{\text{LM}}(d_i \mid f, y^*) = (1 - \varepsilon) \frac{e^{u_i/\beta}}{\sum_{j=1}^K e^{u_j/\beta}} + \frac{\varepsilon}{K}, \quad (4.9)$$

where β is a temperature parameter tuning how sharply the distribution peaks and ε serves as a smoothing factor. Since the binary utility signal can drive the distribution to collapse or peak too strongly, we add label smoothing to obtain a steadier and more informative preference distribution. What results is a distribution expressing document preferences that stem from the LLM’s validation decisions.

Optimization Objective

In the last stage, we bring the retriever’s relevance distribution $P_R(d_i \mid q)$ into agreement with the LLM’s preference distribution $Q_{\text{LM}}(d_i \mid f, y^*)$, at the same time imposing the right relative order over the documents. We accomplish this with a joint objective built from two complementary terms.

KL Divergence. So that the retriever’s relevance distribution matches the evidence preferences read off from LLM validation, we minimize the Kullback-Leibler (KL)

divergence separating the two distributions:

$$\mathcal{L}_{\text{KL}} = \frac{1}{|\mathcal{B}|} \sum_{f \in \mathcal{B}} KL(Q_{\text{LM}}(d \mid f, y^*) \parallel P_R(d \mid q)), \quad (4.10)$$

in which $\mathcal{B}$ stands for the set of target triples awaiting validation. This term nudges the retriever toward the LLM-derived preference distribution, placing more probability mass on documents that lend stronger support to validation.

Pairwise Ranking. Although distribution alignment reflects the overall shape of the preferences, it stops short of explicitly modeling the relative order between helpful and unhelpful documents. We remedy this with a pairwise ranking loss that pushes helpful documents to sit above unhelpful ones. Denoting by $P = \{i \mid u_i = 1\}$ and $N = \{j \mid u_j = 0\}$ the helpful and unhelpful document sets, respectively, we build a collection of document pairs $\Pi_q = \text{TopM}(P) \times \text{BottomN}(N)$ for each query q and specify the ranking loss as:

$$\mathcal{L}_{\text{rank}} = \frac{1}{|\mathcal{B}|} \sum_{q \in \mathcal{B}} \mathbb{E}_{(i,j) \in \Pi_q} \left[-\log \sigma \left(\frac{s(d_i) - s(d_j)}{\tau} \right) \right], \quad (4.11)$$

where $\sigma(\cdot)$ is the sigmoid function and τ is a temperature parameter setting how smooth the margin is. By promoting documents that back correct validation to higher positions, this term optimizes the relative ranking directly.

Final Objective. The full optimization objective merges the two terms:

$$\mathcal{L}_{\text{final}} = \mathcal{L}_{\text{KL}} + \lambda \mathcal{L}_{\text{rank}}, \quad (4.12)$$

where λ acts as a weighting coefficient. Minimizing $\mathcal{L}_{\text{final}}$ simultaneously brings the retriever’s distribution into line with the LLM’s evidence preferences and imposes a ranking that mirrors how much each document contributes to validation. Consequently, RAVER learns to reliably favor the evidence that best serves accurate KG validation.

4.3 Experiments

4.3.1 Experimental Setup

Implementation Details

RAVER is developed using PyTorch¹, with all experiments executed on two NVIDIA V100 32GB GPUs. For retriever training we use the Adam optimizer [39], fixing the learning rate at 2×10^{-5} and the warmup ratio at 0.1. The batch size is held at 32 throughout, while the number of epochs is selected from $\{3, 4, 5\}$.

Hyperparameters. Both the softmax temperature γ used for retrieval likelihood and the temperature β used for the LLM preference are set to 0.3. The label smoothing coefficient ε is fixed at 0.05, and the pairwise ranking temperature τ is set to 0.2. For the weighting coefficient λ that balances the KL and ranking terms, we tune within $[0.1, 0.5]$.

Retrieval Configuration. While training, the retriever pulls the top- $K = 20$ documents for each query, and ranking pairs are assembled from the top- $M = 5$ helpful

¹<https://pytorch.org/>

Table 4.1: Summary statistics for the KG benchmarks. Within the validation and test splits, positive (+) and negative (−) samples appear in equal numbers, with the negatives taken from the original datasets.

Dataset	$ \mathcal{E} $	$ \mathcal{R} $	Train	Valid (+/-)	Test (+/-)	Passages
CoDEx-S	2,034	42	32,888	1,827/1,827	1,828/1,828	117,839
FB15k-237N	13,104	93	87,282	7,041/7,041	8,226/8,226	568,171
Wiki27k	27,112	62	74,793	10,121/10,121	10,122/10,122	781,888

and bottom- $N = 5$ unhelpful documents. Document embeddings are refreshed and the FAISS index is rebuilt at regular intervals of $T = 2000$ steps. For inference, the retriever hands back the top- $K = 10$ documents.

LLM Configuration. Every input pairs a triple with its retrieved evidence, laid out as one instruction-style prompt. The decoding temperature is fixed at 0 so that outputs remain deterministic. Across all experiments, we use Llama3-8B-Instruct [21] and Qwen2.5-7B-Instruct [98] as the LLM backbones.

Datasets

Our experiments span three KG benchmarks: CoDEx-S [71], FB15k-237N [56], and Wiki27k [56]. For dependable evaluation, CoDEx-S comes with human-annotated negatives. FB15k-237N is obtained from FB15k-237 [84] by stripping out mediator-node triples so that validation is more realistic. Wiki27k is a sizable Wikidata subset covering 27k entities across a broad range of relations. In evaluation we rely on the negative triples that the original datasets supply, keeping positive and negative samples balanced in the validation and test splits. The dataset statistics are collected in Table 4.1.

External Corpus

To underpin retrieval-augmented KG validation, we assemble a large external corpus that serves as the evidence pool for candidate triples. Per KG benchmark, we issue its entities as queries and retrieve the full Wikipedia articles that correspond to them.² The articles are obtained through SPARQL together with the Wikipedia API.³ In the case of FB15k-237N, Freebase entities are mapped onto Wikidata via a converter.⁴ Each article is then split into 100-word passages, which produces 118k passages for CoDEX-S, 568k for FB15k-237N, and 782k for Wiki27k.

KG Validation Baselines

For the evaluation of RAVER, we benchmark it against four representative families of KG validation methods: 1) KG embedding, 2) PLM fine-tuning, 3) LLM training-free, and 4) LLM fine-tuning.

KG Embedding. As representative baselines we take TransE [4], DistMult [99], ComplEx [87], and RotatE [80]. Each of these is re-implemented with the OpenKE⁵ library. We fix the decision threshold on the validation set and then apply it at test time.

PLM Fine-tuning. Here we use KG-BERT [102] and PKGC [56] as baselines. We reproduce KG-BERT from its official implementation, whereas for PKGC we cite

²All articles were collected as of May 15, 2025.

³<https://github.com/martin-majlis/Wikipedia-API>

⁴<https://github.com/google/freebase-wikidata-converter>

⁵<https://github.com/thunlp/OpenKE>

the numbers reported in the original paper so that the comparison is made under the same benchmark setting.

LLM Training-free. We additionally assess how LLMs fare on KG validation when they draw exclusively on their internal parametric knowledge, testing them in zero- and few-shot settings.

LLM Fine-tuning. Two LLM fine-tuning methods are considered: KG-Llama [103] and KoPA [111]. To keep the comparison fair, we retrain both on one of our backbone LLMs, Llama3-8B, adhering to their original configurations, which gives KG-Llama3-8B and KoPA-Llama3-8B, respectively.

Retrieval Baselines

To measure how much retrieval quality matters, we contrast retrieval-augmented KG validation across several retrieval strategies. The retrievers we examine span representative sparse and dense choices: TF-IDF [1], BM25 [69], Contriever [30], and HyDE [19]. Both TF-IDF and BM25 belong to the sparse lexical family, and BM25 adds IDF weighting together with length normalization. Contriever is applied as-is, without any task-specific optimization. In the reranking variant, candidate documents are first fetched by Contriever and subsequently reordered by an external reranker⁶. HyDE, in turn, synthesizes a hypothetical document per query and retrieves on the basis of that document’s dense representation.

⁶<https://huggingface.co/BAAI/bge-reranker-v2-m3>

Evaluation Metrics

Our model is assessed with four standard classification metrics: accuracy (A), precision (P), recall (R), and F1-score (F1). Accuracy reflects the share of the entire triple set that receives a correct label. Precision reports the proportion of triples flagged as true that really are true, while recall reports the proportion of all truly true triples that the model succeeds in retrieving. By taking the harmonic mean of precision and recall, the F1-score condenses the two into one balanced summary of overall performance.

4.3.2 Experimental Results

Overall Performance Comparison

The results over all KG benchmarks are laid out in Table 4.2. Across the board, RAVER surpasses prior methods and, with the Llama3-8B backbone, attains the top overall accuracy and F1-score. Set against the LLM-based baselines, these numbers make clear why grounding KG validation in external evidence matters. Training-free methods do reasonably well on parametric knowledge alone, but they frequently falter on hard cases that call for verifiable evidence. Fine-tuning the LLM tends to lift performance above those baselines, though it can still slip on unseen facts and long-tail entities. RAVER, by contrast, retrieves pertinent evidence and folds it into evidence-grounded reasoning, delivering validation performance that is consistently stronger and more robust. Taken together, the findings indicate that statistical pattern learning and internal knowledge on their own do not suffice for accurate KG validation, which reinforces the value of reasoning anchored in dependable external evidence.

Table 4.2: Comparison of overall KG validation performance over every benchmark. Bold marks the best Accuracy and F1-score, and underlining marks the second-best.

Paradigm	Model	CoDEX-S				FB15k-237N				Wiki27k			
		A	P	R	F1	A	P	R	F1	A	P	R	F1
KG Embedding	TransE	69.80	72.05	64.72	68.18	62.64	61.02	69.95	65.18	62.41	60.78	69.99	65.06
	DistMult	68.93	64.77	82.99	72.76	63.17	63.44	62.13	62.78	69.00	78.19	52.70	62.96
	ComplEx	71.69	68.93	78.99	73.62	63.69	62.41	68.83	65.46	69.47	75.89	57.07	65.15
	RotatE	75.55	76.35	74.02	75.17	68.18	71.03	61.41	65.87	73.83	77.72	66.81	71.86
PLM Fine-tuning	KG-BERT	68.41	61.78	96.55	75.35	60.90	56.26	97.90	71.46	81.56	74.56	95.81	83.86
	PKGc	-	-	-	-	<u>79.60</u>	-	-	79.50	<u>87.00</u>	-	-	<u>87.10</u>
LLM Training-free	ZS(Qwen2.5-7B)	74.95	86.31	59.30	70.30	63.10	74.64	39.68	51.81	72.77	87.52	53.12	66.11
	ZS(Llama3-8B)	72.76	66.67	91.03	76.91	67.30	61.76	90.85	73.53	72.23	65.23	95.22	77.42
	2-Shot(Qwen2.5-7B)	78.09	81.95	72.05	76.68	73.73	76.36	68.73	72.35	74.96	86.73	58.93	70.18
	2-Shot(Llama3-8B)	79.68	84.14	73.18	78.26	75.47	71.33	85.17	77.64	80.52	75.74	89.79	82.17
	8-Shot(Qwen2.5-7B)	77.22	76.94	77.74	77.33	73.77	73.06	75.32	74.17	76.88	78.25	74.44	76.30
	8-Shot(Llama3-8B)	80.61	79.00	83.37	81.13	75.64	70.99	86.73	78.07	80.87	75.70	90.93	82.62
LLM Fine-tuning	KG-Llama3-8B	<u>81.89</u>	80.33	84.46	<u>82.35</u>	75.10	67.45	97.03	79.58	82.70	78.74	89.59	83.82
	KoPA-Llama3-8B	81.04	77.14	88.24	82.32	77.81	70.70	94.99	<u>81.07</u>	82.33	80.13	85.97	82.95
Retrieval-Augmented	RAVER-Qwen2.5-7B	81.73	85.37	76.59	80.74	78.14	82.45	71.50	76.59	81.54	83.38	78.78	81.01
	RAVER-Llama3-8B	86.16	89.02	82.49	85.63	82.08	81.06	83.73	82.38	88.38	89.34	87.17	88.24

RAVER likewise generalizes well across LLM backbones, delivering steady improvements with both Qwen2.5-7B and Llama3-8B. In the absence of retrieval augmentation, Qwen2.5-7B trails Llama3-8B on every benchmark, a sign of the capacity gap between them. Once RAVeR is applied, both models post sizable gains in accuracy and F1-score, showing that the framework strengthens validation no matter which LLM sits underneath. Notably, these gains persist even with the smaller backbone, which speaks to RAVeR’s model-agnostic reach. This implies that RAVeR’s improvements owe less to richer parametric knowledge than to higher-utility evidence and more dependable evidence-grounded reasoning, a benefit that carries across LLMs of differing capacity. Collectively, the results confirm that RAVeR’s advantage is not bound to any one backbone but arises from its retrieval and evidence-grounded reasoning pipeline.

Table 4.3: Validation performance under varying retrieval strategies. Bold denotes the best Accuracy and F1-score, and underlining denotes the second-best.

LLM	Retrieval		CoDEX-S				FB15k-237N				Wiki27k			
			A	P	R	F1	A	P	R	F1	A	P	R	F1
Qwen2.5-7B	-	Zero-Shot	74.95	86.31	59.30	70.30	63.10	74.64	39.68	51.81	72.77	87.52	53.12	66.11
	Sparse	TF-IDF	69.94	90.73	44.42	59.64	69.58	86.11	46.69	60.55	72.23	96.42	46.17	62.44
		BM25	70.68	90.30	46.33	61.24	69.63	87.24	45.98	60.22	73.14	86.10	55.20	67.27
	Dense	Contriever	75.25	91.32	55.80	69.27	72.99	85.12	55.72	67.35	75.71	94.89	54.33	69.10
		Contriever+Rerank	78.36	89.48	64.27	74.82	75.28	84.16	62.28	71.58	79.14	90.72	64.91	75.70
		HyDE	<u>80.35</u>	87.91	70.38	<u>78.17</u>	<u>76.76</u>	83.31	66.94	<u>74.23</u>	<u>80.55</u>	87.42	71.36	<u>78.58</u>
		RAVER	81.73	85.37	76.59	80.74	78.14	82.45	71.50	76.59	81.54	83.38	78.78	81.01
Llama3-8B	-	Zero-Shot	72.76	66.67	91.03	76.91	67.30	61.76	90.85	73.53	72.23	65.23	95.22	77.42
	Sparse	TF-IDF	78.28	85.66	67.94	75.78	75.25	76.61	72.69	74.60	81.10	91.08	68.95	78.49
		BM25	78.34	84.53	69.37	76.20	74.35	75.88	71.39	73.56	80.19	88.15	69.76	77.88
	Dense	Contriever	81.70	86.61	75.00	80.39	75.85	74.78	78.01	76.36	83.47	87.99	77.52	82.42
		Contriever+Rerank	83.30	87.38	77.84	82.34	78.62	77.52	80.61	79.04	85.39	88.56	81.28	84.76
		HyDE	<u>84.79</u>	88.21	80.31	<u>84.07</u>	<u>80.41</u>	79.36	82.19	<u>80.75</u>	<u>87.05</u>	88.94	84.63	<u>86.73</u>
		RAVER	86.16	89.02	82.49	85.63	82.08	81.06	83.73	82.38	88.38	89.34	87.17	88.24

Impact of Retriever on KG Validation

The outcomes of the various retrieval strategies and how each affects KG validation are reported in Table 4.3. Broadly, validation performance climbs as retrieval quality rises, which evidences a tight link between the quality of the retrieved evidence and the downstream validation results.

Sparse vs. Dense Retrieval. Conventional sparse retrievers such as TF-IDF and BM25 add only marginal value compared with validation that uses no retrieval at all. On occasion, most visibly with Qwen2.5-7B, sparse retrieval even dips below the zero-shot baseline, a hint that poor-quality evidence can work against KG validation. Contriever, on the other hand, brings cleaner and steadier gains across datasets and LLM backbones. These results indicate that semantic retrieval tends to furnish more useful validation evidence than lexical matching does.

Advanced Dense Retrieval Strategies. Strengthening dense retrieval with either reranking or hypothetical document generation pushes validation performance up further. Across all datasets and LLM backbones, Contriever+Rerank steadily beats the plain Contriever baseline, showing that polishing the initial ranking helps surface more informative evidence. HyDE secures extra gains and reliably lands the second-best accuracy and F1-score. Because it produces a hypothetical document holding the information required to validate a triple, HyDE narrows the semantic gap between structured triple queries and natural-language documents. Even so, neither method reaches RAVER, which suggests that better relevance modeling by itself does not fully capture how task-specifically useful a piece of evidence is for KG validation.

Effectiveness of Retriever Alignment. On every dataset and LLM backbone, RAVER records the top accuracy and F1-score. With both Qwen2.5-7B and Llama3-8B it consistently edges out Contriever, Contriever+Rerank, and HyDE. This tells us that RAVER moves past generic semantic relevance by identifying the documents that directly drive correct validation decisions and elevating them in the ranking. Such utility-aware alignment closes the gap between general retrieval relevance and the LLM’s task-specific evidence requirements. In sum, getting the most out of KG validation demands not just a capable LLM and retriever but retrieval that is aligned with the validation objective.

Table 4.4: Shifts in evidence utility and retrieval success brought about by retriever alignment. *Imp.* refers to the absolute gain measured from before to after alignment.

Dataset	Utility@5			Utility@10		
	Before	After	Imp.	Before	After	Imp.
CoDEX-S	0.382	0.442	+0.060	0.337	0.389	+0.052
FB15k-237N	0.365	0.418	+0.053	0.326	0.371	+0.045
Wiki27k	0.408	0.501	+0.093	0.361	0.441	+0.080

Dataset	Hits@5			Hits@10		
	Before	After	Imp.	Before	After	Imp.
CoDEX-S	0.594	0.705	+0.111	0.716	0.817	+0.101
FB15k-237N	0.568	0.661	+0.093	0.689	0.773	+0.084
Wiki27k	0.662	0.814	+0.152	0.784	0.865	+0.081

Evidence Utility and Ranking after Retriever Alignment

To study the effect of retriever alignment on both the quality and the ordering of retrieved evidence, we build an evidence pool by using Contriever to pull the top-100 candidate documents for each triple. A document receives a binary utility score of 1 when conditioning the LLM on it yields a correct validation decision, and 0 when it does not.

Evidence Utility. Table 4.4 sets Utility@ K and Hits@ K side by side for the periods before and after retriever alignment. Utility@ K captures the mean binary utility among the top- K documents, whereas Hits@ K signals whether at least one useful document sits within that set. Both measures rise consistently over every dataset and cutoff value. Specifically, Utility@5 climbs by as much as 0.093 and Hits@5 by as much as 0.152. These figures demonstrate that RAVER heightens both the density of useful evidence near the top of the ranking and the chance of retrieving evidence that underpins correct KG validation.

Ranking of Useful Evidence. Figure 4.5 traces how the positions of useful documents shift once alignment is applied. Over the datasets, as many as 45.8% of useful

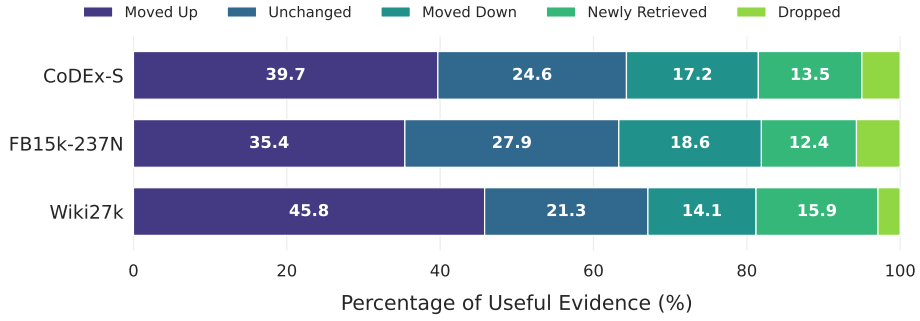


Figure 4.5: How rank changes for useful evidence are distributed once retriever alignment is applied. Every bar reports the fraction of useful documents that rose, held steady, fell, entered anew, or left the retrieved set.

documents climb to higher ranks, while at most 18.6% slide lower. Furthermore, up to 15.9% of useful documents newly appear in the retrieved set, while no more than 5.7% are lost from it. This indicates that retriever alignment genuinely strengthens the ranking structure, lifting useful documents to better positions and bringing in extra evidence for correct KG validation.

Further Analysis

We probe RAVER more deeply by way of long-tail evaluation and qualitative case studies. Together these analyses gauge how robust it is on rare entities, how useful the retrieved evidence proves to be, and how well it sustains reliable evidence-grounded validation.

Performance on Long-Tail Triples. Long-tail triples pose an especially hard problem for LLM-based KG validation, since rare entities show up only sparsely in both the model’s training data and the KG structure. To measure performance under these

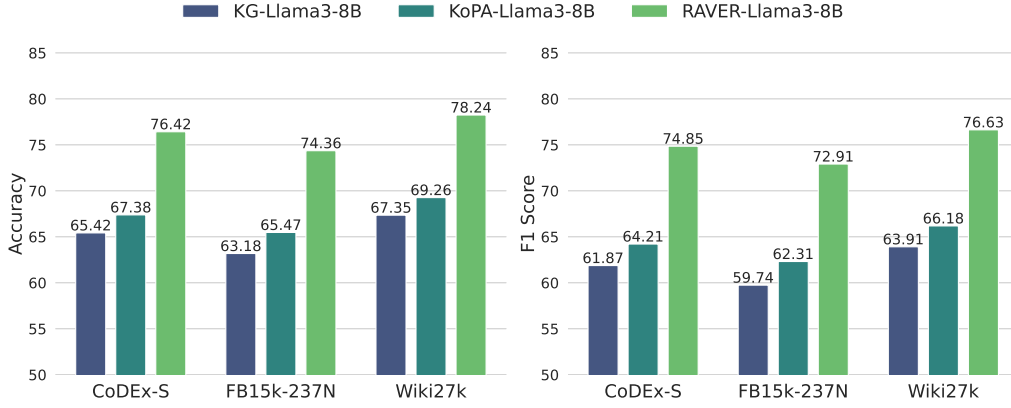


Figure 4.6: Long-tail triple validation performance measured across the datasets.

conditions, we pick triples that include at least one entity whose frequency lies in the bottom 5% of the dataset.

The validation outcomes on these long-tail subsets are reported in Figure 4.6. On every dataset RAVER reaches the highest Accuracy and F1-score, topping the strongest baseline on both metrics. Such gains suggest that external evidence helps make up for the scant parametric and structural information available about rare entities. Because it grounds its verdicts in retrieved documents, RAVER validates information-scarce triples more reliably than approaches that lean chiefly on internal knowledge or KG structure.

Case Study: Long-Tail Triple Validation. We look into a demanding case built around a rare entity drawn from the long-tail distribution. Figure 4.7 shows an instance of long-tail fact validation involving an entity that surfaces only very rarely in the training data. The triple in question is (*Sugamo Prison, located in, Tokyo*), and

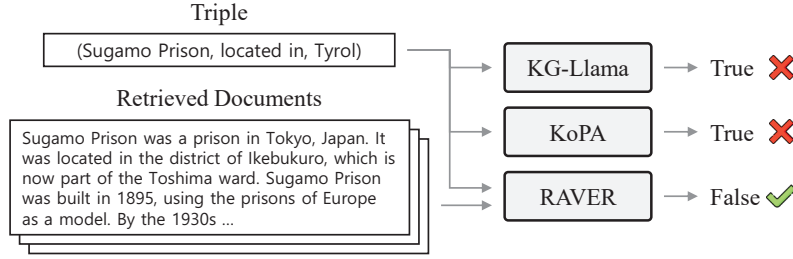


Figure 4.7: A sample of the validation outcomes produced by RAVER and by LLM-based baselines for a long-tail triple.

Sugamo Prison is a rare, low-frequency entity in the KG. When faced with such rare-entity triples, existing LLM-based validators frequently mislabel them as True, which points to inadequate knowledge acquisition and weak generalization, particularly after fine-tuning. They consequently fall back on incomplete internal knowledge and arrive at wrong judgments. RAVER, however, correctly returns False by reasoning over the retrieved external evidence, evidencing robust validation on long-tail facts. The case study makes plain that retrieval augmentation offsets the shortcomings of an LLM’s internal knowledge and supports more dependable evidence-grounded reasoning.

Case Study: Evidence Retrieval. For the shared query triple (*Céline Dion*, *occupation*, *film actor*), Table 4.5 lines up the top-1 document each retrieval method returns. Sparse retrievers hand back keyword-matched documents that are lexically on-topic yet offer little actionable evidence for validating the triple. Contriever lifts semantic relevance, but its retrieved content is often not directly informative for validation, since conventional retrievers are tuned for relevance rather than downstream usefulness. RAVER instead surfaces evidence that directly bears on the validation decision,

Table 4.5: Sample top-1 documents retrieved by the baseline methods and by our proposed approach.

Retriever	Top-1 Retrieved Document
TF-IDF	... Céline Dion . Contemporary Musicians, Volume 25. Gale Group, 1999. Céline Dion . Newsmakers 1995, Issue 4. Gale Research, 1995. Céline Dion . Rock on the Net. Retrieved 20 November 2005. Céline Dion
BM25	... René Angélil: the making of Céline Dion : the unauthorized biography. Dundurn Group. Bogdanov, Vladimir; Woodstra, Chris; Erlewine, Stephen Thomas (2001). Allmusic:The Definitive Guide to Popular Music. ...
Contriever	... Dion resides in a house in Henderson, Nevada, which was reportedly purchased for US\$1.5 million in 2003 and as of 2024 was now closer to US\$4.5 million in value after renovations. Tours Residencies Celine (2011–2019) ...
RAVER	... In 2023, Dion starred as herself in <i>Love Again</i> , her first acting appearance in a feature film. She also recorded five new songs for the soundtrack, released on 12 May; the first single, “Love Again”, ...

for example a passage recording *Dion*’s acting appearance in a feature film, which supplies a decisive cue for confirming the triple. On the whole, this case brings out that utility-aware retrieval pays off for KG validation and that RAVeR markedly raises the practical usefulness of the retrieved evidence.

Error Analysis

We carried out a systematic error analysis, manually reviewing 100 randomly drawn misclassified cases per KG benchmark, evenly split at a 1:1 ratio between `True` and `False` labels. This review surfaced four categories of error: 1) Incorrect Retrieval, 2) Evidence Absence, 3) Triple Misinterpretation, and 4) Incorrect Reasoning. The distribution of these categories is depicted in Fig. 4.8, and Table 4.6 offers representative instances drawn from a subset of them.

Incorrect Retrieval. This error happens when the retriever cannot surface suitable evidence even though trustworthy information for validating the triple is present in the document pool. Its root cause lies chiefly in retrieval limitations, and above all

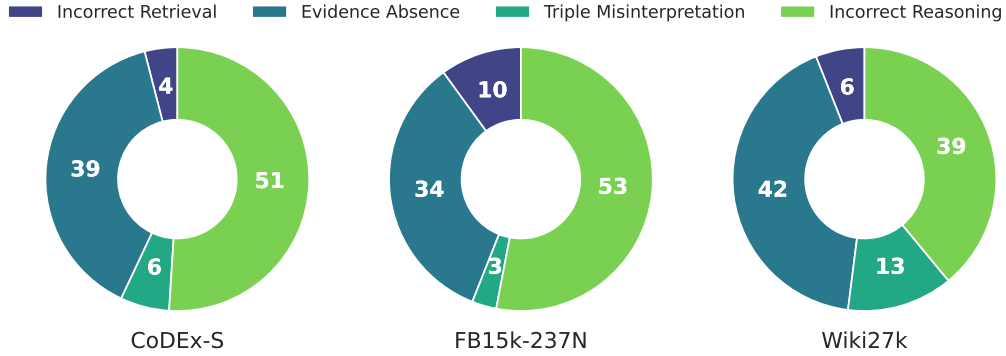


Figure 4.8: How the error types are distributed in retrieval-augmented KG validation.

in how the query is formed. Since our queries are built by simply concatenating the triple’s components, they may fail to match the relevant documents whenever the entity or relation names are abstract or ambiguous.

Evidence Absence. This error emerges when the document pool holds no direct evidence with which to validate a given triple. Two principal factors are behind it. For one, certain facts reside solely in non-textual sources such as tables, figures, or external links, which text-based retrieval does not reach. For another, documents covering rare or highly specific facts may be sparse or missing altogether at corpus-construction time. Under these circumstances, the validation errors trace back to gaps in corpus coverage rather than to retrieval quality, which makes reliable evidence-grounded validation inherently hard.

Triple Misinterpretation. This error occurs when suitable evidence is indeed retrieved, yet the LLM misreads the triple and delivers a wrong judgment. The model might misconstrue relation semantics, mix up the roles or the direction of the enti-

Table 4.6: Illustrative instances of triple misinterpretation and of incorrect reasoning.

(a) An instance of triple misinterpretation.		(b) An instance of incorrect reasoning.	
Triple	(Nashville, country, Germany) / False	Triple	(Julian Lennon, sibling, Yoko Ono) / False
Documents	Tom Roland, from Country Music Association International, explains country music’s global popularity: Country Music listeners around the globe have something in common with those in the United States. In Germany, for instance, Rohrbach identifies three ...	Documents	... he did not blame Ono for the breakup of the Beatles and credited Ono with inspiring much of Lennon’s post-Beatles work. Relationship with Julian Lennon Ono had a difficult relationship with her stepson Julian, but the relationship improved over the years.
Reasoning	The document mentions that Tom Roland explains that country music listeners around the globe have something in common with those in the United States, and that in Germany. This suggests that country music is popular in Germany and Nashville.	Reasoning	The document states that Julian Lennon is Yoko Ono’s stepson, and that she had a difficult relationship with him initially, but it improved over the years. This suggests that the improved relationship implies Julian Lennon and Yoko Ono eventually became true family.
Answer	Based on the provided documents, the triple is True.	Answer	Based on the provided documents, the triple is True.

ties, or mistake a merely contextual association for direct support of the triple. Errors of this kind commonly appear when the model leans on shallow textual cues instead of correctly parsing the structured relation.

Incorrect Reasoning. This error arises when the LLM cannot reach the right judgment despite having appropriate evidence at hand. Failures of this sort betray trouble in fusing evidence, tying related pieces of information together, and reasoning past surface-level cues. We note four recurring patterns: (1) an inability to pull explicit information out of a single document, (2) an inability to knit together evidence spread over several documents, (3) mistakes in logical or commonsense reasoning, and (4) hallucinated reasoning steps or unfounded intermediate claims. These patterns show that today’s LLMs are still limited in their capacity to reason dependably over retrieved evidence.

Discussions

Our error analysis brings to light a number of limitations in retrieval-augmented KG validation and points toward avenues for future research.

Diversity and Extensibility of Evidence. Useful evidence lives not only in textual passages but also in tables, figures, structured databases, and linked resources, none of which the present retrieval system taps. Because of this, certain triples resist reliable validation even when the supporting information does exist in non-textual or structured form. Moreover, our retrieval setup is aimed mainly at general-domain knowledge and may not transfer smoothly to specialized fields such as medicine, law, or finance, where domain-specific terminology and evidence sources come into play. Future efforts should therefore broaden evidence coverage by drawing on multimodal retrieval, structured-data integration, and domain-adaptive or web-scale retrieval.

Structured Knowledge Understanding. While LLMs can grasp basic entity and relation semantics, they continue to stumble on relational directionality, hierarchical organization, and compositional meaning within KGs. Such shortcomings stand out most clearly when validation hinges on fine-grained relation distinctions, schema-level comprehension, or reasoning across related entities and relations. Even with correct retrieved evidence, a weak grasp of the structured triple can lead the model to attach the evidence to the wrong relation or to read the triple in the wrong direction. Future work might bolster structured knowledge understanding via schema-aware training, explicit structural constraints, or KG-specific representations.

Retrieval-Reasoning Alignment. Even after relevant evidence has been retrieved, LLMs may still integrate it incorrectly or draw invalid conclusions, which signals a persisting gap between retrieval and reasoning. The problem grows sharper when the evidence spans multiple documents, embeds implicit relationships, or demands logical and commonsense inference. Our current framework refines retrieval with validation feedback, yet retrieval and reasoning remain, for the most part, two separate stages. Future work ought to tighten their coupling through reasoning-aware and iterative retrieval, in which intermediate reasoning pinpoints missing information and steers the next round of evidence gathering.

4.4 Summary

This chapter presented RAVER, a retrieval-augmented framework that anchors LLM-based KG validation in external evidence while tuning the retriever from validation outcomes. RAVER gauges each document’s utility from how much it contributes to correct judgments and adjusts the retriever on that basis. Pairing KL divergence with pairwise ranking to align retrieval scores with evidence usefulness, RAVER supplies better-quality evidence for factual reasoning. Our experiments reveal steady gains over strong baselines together with robustness on long-tail facts. We further examined how retrieval quality and retriever alignment shape validation, identifying evidence utility as a central factor. We closed by outlining future directions, spanning broader evidence sources, structured knowledge understanding, and retrieval-reasoning alignment.

5. Knowledge Graph Refinement via Operation-Guided Editing

This chapter addresses the problem of refining KGs that have been built with LLMs. Owing to recent progress in LLMs, KGs can now be constructed end-to-end from natural language documents; among such approaches, generative knowledge extraction (GKE) has become a representative paradigm in which triples are generated from text in a flexible and scalable fashion. Nevertheless, generation is never error-free, and the KGs obtained in this way frequently include factual inconsistencies that deviate from what the source documents state. These inconsistencies manifest in various ways—unsupported hallucinations, distorted relational semantics, and imprecise entity representations—and they undermine both the correctness of the graph and its usability.

Prior work on validating and refining KGs has concentrated chiefly on detecting and discarding erroneous triples, relying on hand-crafted rules, schema-level constraints, or plausibility signals derived from embeddings. LLM-based judging has also been investigated of late as a means of removing invalid triples. Deletion does eliminate certain errors, but a binary valid/invalid decision risks throwing away facts that are only partially wrong, and it cannot handle representation-level problems that reduce clarity without rendering a triple outright incorrect. Motivated by these limitations, we seek a refinement framework capable of finer-grained, text-grounded adjustments.

Toward this goal, this chapter introduces **GraphRefine**, a framework that refines GKE-produced KGs after construction. Taking a draft KG together with the document from which it was extracted, GraphRefine assigns each triple a refinement operation supported by the document and, where necessary, edits or rewrites the triple so that factual inconsistencies are diminished.

5.1 Preliminaries

5.1.1 Generative Knowledge Extraction

GKE is formulated here as the task in which an LLM derives, from a source document $\mathcal{D}$, a KG $\mathcal{G}_{\mathcal{D}}$; the graph itself is a collection of triples $\tau = (h, r, t)$. Formally, $\mathcal{G}_{\mathcal{D}}$ takes the form:

$$\mathcal{G}_{\mathcal{D}} = \{(h, r, t) \mid h, t \in \mathcal{E}, r \in \mathcal{R}\}, \quad (5.1)$$

with $\mathcal{E}$ standing for the entity set and $\mathcal{R}$ for the relation set. During extraction, the LLM emits its output tokens while conditioning on the document $\mathcal{D}$, which admits the following autoregressive formulation:

$$P(x_i \mid x_1, x_2, \dots, x_{<i}, \mathcal{D}), \quad (5.2)$$

in which x_i is the i -th token of the generated sequence. Parsing the model output yields a list of triples $[\tau_1, \tau_2, \dots]$, whose union constitutes $\mathcal{G}_{\mathcal{D}}$. Ideally, the extracted KG should cover the relevant factual content of the document as fully as possible without contradicting any fact the document states.

5.1.2 Knowledge Graph Refinement

A graph $\mathcal{G}_{\mathcal{D}}$ obtained via GKE is not guaranteed to be free of factual inconsistencies. KG refinement therefore seeks a transformed graph $\hat{\mathcal{G}}_{\mathcal{D}}$ whose content aligns more closely with the document $\mathcal{D}$ it is derived from. We formalize this process through a refinement operator $\mathcal{F}_{\theta}$ built on an LLM:

$$\hat{\mathcal{G}}_{\mathcal{D}} = \mathcal{F}_{\theta}(\mathcal{D}, \mathcal{G}_{\mathcal{D}}). \quad (5.3)$$

The operator $\mathcal{F}_{\theta}$ is supplied with both $\mathcal{D}$ and the initial graph $\mathcal{G}_{\mathcal{D}}$; by locating erroneous triples and repairing them, it strengthens the factual consistency of the graph. In this way, the inconsistencies incurred during GKE are reduced, and the refined KG reproduces the document’s original content with greater fidelity.

5.2 Proposed Approach: GraphRefine

This section details GraphRefine, our approach to refining KGs with LLMs. The discussion opens with a taxonomy of the factual inconsistencies observed in GKE and then turns to how an LLM can be fine-tuned to play the role of KG refiner.

5.2.1 Factual Inconsistencies in GKE

An examination of GKE-generated KGs reveals two dominant categories of factual inconsistency: (i) Correctness-level issues and (ii) Representation-level issues. Notably, both categories recur regardless of the dataset or the extraction model employed. We next explain what each category entails and consider the refinement ac-

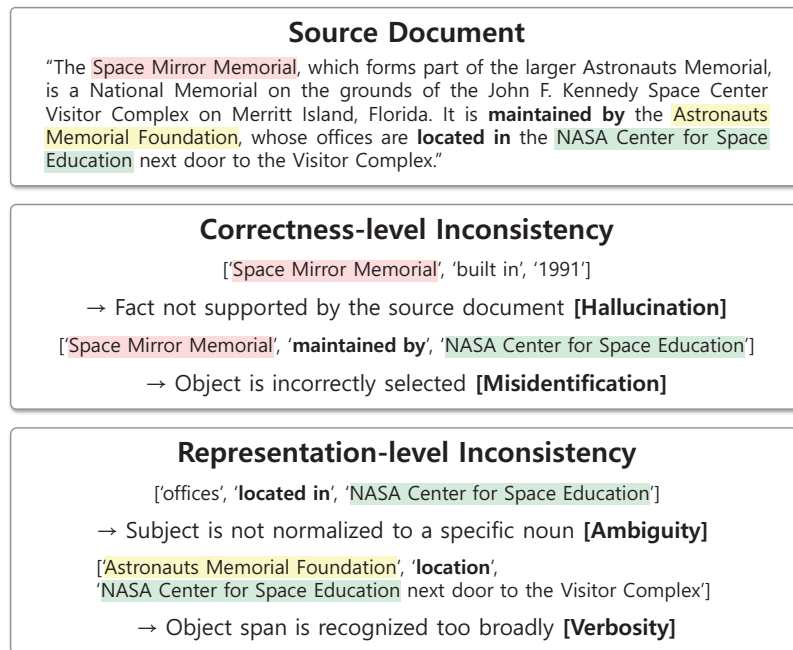


Figure 5.1: Illustrative cases of the factual inconsistencies that arise during generative knowledge extraction.

tions appropriate for it.

Correctness-level Inconsistencies. An inconsistency at the correctness level means that a triple asserts something factually untrue. Figure 5.1 illustrates two typical cases: triples hallucinated without any grounding in the document, and triples whose entity or relation semantics have been distorted so that the resulting statement is false. Errors of this kind erode the accuracy of the KG directly and can misguide the reasoning of downstream applications. The usual remedy is deletion, but removing such triples shrinks the coverage of the graph. Whenever the document contains evidence for the intended fact, semantically correcting the triple is generally preferable

to discarding it, because correction restores accuracy without sacrificing informational coverage.

Representation-level Inconsistencies. Representation-level inconsistencies concern triples that, while not factually false, still degrade the quality of the KG because their surface forms are imprecise or mutually inconsistent. Typical instances shown in Figure 5.1 include entity spans with unclear boundaries and expressions burdened with superfluous wording. Under such conditions, the graph cannot state the document’s facts in a precise and unambiguous manner, so the resulting inconsistency is one of representation, arising even where the facts being conveyed are themselves valid. Our remedy is rewriting that leaves the meaning untouched: it suppresses ambiguity and surface-form variation and thereby renders the triples clearer and more uniform.

5.2.2 Fine-tuning LLM as KG Refiner

We address the inconsistency types identified above by fine-tuning an LLM into a dedicated KG refiner. Presented with a source document alongside a triple drawn from it, the refiner picks the refinement action best suited to that triple and executes it. Concretely, for every triple the refiner selects exactly one of four operations:

- **KEEP:** The triple remains untouched when it agrees with the document factually and its form is already adequate.
- **DELETE:** The triple is removed as noise when the document does not support it or when it states an incorrect fact.

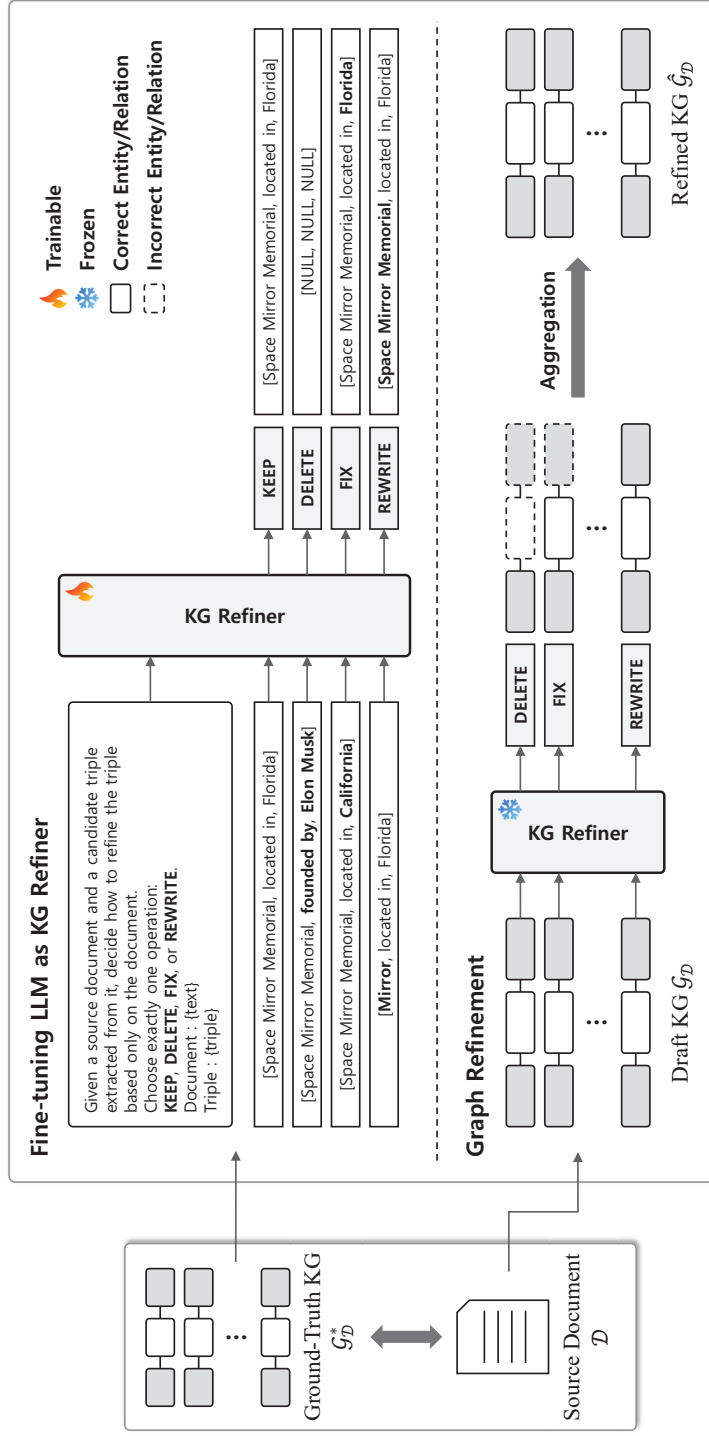


Figure 5.2: A high-level view of the GraphRefine pipeline. We first synthesize operation-tied supervision that captures the inconsistency categories seen in GKE, and an LLM is then fine-tuned on it to act as a KG refiner. Working one triple at a time, the refiner processes a draft KG, and its per-triple outputs are afterward combined into the refined graph.

- **FIX:** When the document supports a related fact but the triple states it incorrectly, we amend the triple’s meaning so that it agrees with the documentary evidence.
- **REWRITE:** When the meaning is sound yet its surface realization is awkward or inconsistent, we rephrase the triple into a cleaner, more consistent form.

Dataset Generation

Training data for the refiner are synthesized by constructing a separate triple set for each operation. We write $\mathcal{S}_{\mathcal{G}_{\text{train}}}$ for the set of gold-standard KGs taken from the training split; each $\mathcal{G} \in \mathcal{S}_{\mathcal{G}_{\text{train}}}$ comprises triples $\tau = (h, r, t)$ that are accompanied by a source document $\mathcal{D}$. In addition, $\mathcal{E}(\mathcal{D})$ denotes the entities appearing in $\mathcal{D}$, while $\mathcal{E}$ and $\mathcal{R}$ are the global entity and relation sets, respectively.

KEEP. Ground-truth triples are carried over without modification:

$$\mathcal{T}_{\text{KEEP}} = \bigcup_{\mathcal{G} \in \mathcal{S}_{\mathcal{G}_{\text{train}}}} \left\{ (h, r, t) \mid (h, r, t) \in \mathcal{G} \right\}. \quad (5.4)$$

DELETE. Triples destined for deletion are synthesized by corrupting the relation and the tail simultaneously, which yields hallucinations that cannot be recovered. We also impose the constraint $\tau_{\text{DEL}} \notin \mathcal{G}$ on every such triple.

$$\mathcal{T}_{\text{DELETE}} = \bigcup_{\mathcal{G} \in \mathcal{S}_{\mathcal{G}_{\text{train}}}} \left\{ \tau_{\text{DEL}} = (h, \tilde{r}, \tilde{t}) \mid \tilde{r} \in \mathcal{R} \setminus \{r\}, \tilde{t} \in \mathcal{E} \setminus \{t\} \right\}. \quad (5.5)$$

FIX. Erroneous triples for this operation are obtained by swapping the tail entity for another entity absent from the source document. Every generated triple must ad-

ditionally satisfy $\tau_{\text{FIX}} \notin \mathcal{G}$. An analogous substitution is likewise performed on the head entity.

$$\mathcal{T}_{\text{FIX}} = \bigcup_{\mathcal{G} \in \mathcal{S}_{\mathcal{G}_{\text{train}}}} \left\{ \tau_{\text{FIX}} = (h, r, \tilde{t}) \mid \tilde{t} \notin \mathcal{E}(\mathcal{D}) \right\}. \quad (5.6)$$

REWRITE. Representation-level noise is injected by disturbing the span of an entity mention in the document without altering what the entity refers to. Let $\text{span}(e, \mathcal{D}) = (p, q)$ be the word-level extent of entity e within $\mathcal{D}$, where p marks the starting position and q the ending position. A perturbed span (p', q') is then constructed in the following way:

$$\begin{aligned} \text{Expansion: } (p', q') &= (p - \delta_l, q + \delta_r), \\ \text{Shrinkage: } (p', q') &= (p + \delta_l, q - \delta_r), \end{aligned} \quad (5.7)$$

Here, the integers δ_l and δ_r determine how far the left and right boundaries move. The perturbed surface form is then given by $\tilde{e} = \mathcal{D}[p' : q']$. The formulation below covers perturbation of the tail entity; head entities are treated in exactly the same manner.

$$\mathcal{T}_{\text{REWRITE}} = \bigcup_{\mathcal{G} \in \mathcal{S}_{\mathcal{G}_{\text{train}}}} \left\{ \tau_{\text{REW}} = (h, r, \tilde{t}) \mid \tilde{t} = D_i[p'_t : q'_t] \right\}. \quad (5.8)$$

Training Objective

Combining the four operation-specific triple sets yields the synthetic training pool:

$$\mathcal{T}_{\text{train}} = \mathcal{T}_{\text{KEEP}} \cup \mathcal{T}_{\text{DELETE}} \cup \mathcal{T}_{\text{FIX}} \cup \mathcal{T}_{\text{REWRITE}}. \quad (5.9)$$

The pool $\mathcal{T}_{\text{train}}$ is then recast as instruction-following supervision on which the KG refiner is fine-tuned. Figure 5.2 illustrates the format: for every training instance, the source document $\mathcal{D}$, a corrupted input triple $\tilde{\tau}$ produced during corruption, together with an instruction $\mathcal{I}$ requesting refinement, are concatenated into one prompt. As the target response, the model must output an operation label y matching the ground-truth result of refinement. With the prompt and the target response expressed in tokenized form as $\mathcal{X}_{\text{in}}$ and $\mathcal{X}_{\text{out}}$, respectively, we perform teacher-forced supervised fine-tuning to maximize $p_{\theta}(\mathcal{X}_{\text{out}} \mid \mathcal{X}_{\text{in}})$. Concretely, the objective minimizes the standard negative log-likelihood computed over the generated response:

$$\mathcal{L}_{\text{SFT}} = - \sum_{i=1}^L \log p_{\theta}(x_i^{\text{out}} \mid \mathcal{X}_{\text{in}}, x_{<i}^{\text{out}}), \quad (5.10)$$

with θ the trainable parameters and L the length of the output. Through this instruction-tuning setup, the model jointly learns to choose a fitting operation and to produce the refined triple that follows from it.

5.2.3 Graph Refinement

At refinement time, each triple $\tau \in \mathcal{G}_{\mathcal{D}}$ of a GKE-produced draft KG is processed together with the source document $\mathcal{D}$: the refiner predicts an operation label and, when the operation calls for it, outputs a refined triple $\hat{\tau}$. This behavior is captured formally by the mapping:

$$(y, \hat{\tau}) = f_{\theta}(\mathcal{D}, \tau), \quad (5.11)$$

in which f_θ stands for the fine-tuned refiner. The refined KG $\hat{\mathcal{G}}_{\mathcal{D}}$ is subsequently assembled by collecting triples in accordance with the predicted operations:

$$\hat{\mathcal{G}}_{\mathcal{D}} = \bigcup_{\tau \in \mathcal{G}_{\mathcal{D}}} g(\tau; y, \hat{\tau}). \quad (5.12)$$

Here, the aggregation function $g(\tau; y, \hat{\tau})$ takes the form:

$$g(\tau; y, \hat{\tau}) = \begin{cases} \{\tau\}, & \text{if } y = \text{KEEP}, \\ \emptyset, & \text{if } y = \text{DELETE}, \\ \{\hat{\tau}\}, & \text{if } y \in \{\text{FIX}, \text{REWRITE}\}. \end{cases} \quad (5.13)$$

The outcome is a refined graph $\hat{\mathcal{G}}_{\mathcal{D}}$ whose factual inconsistencies have been reduced and which consequently mirrors the source document $\mathcal{D}$ more faithfully. Because GraphRefine makes no assumptions about which GKE system produced the draft, it can polish KGs coming from arbitrary extractors. Because of this independence, GraphRefine can be dropped into current LLM-driven construction workflows as a generic refinement step, all without touching the extractor that feeds it.

5.3 Experiments

5.3.1 Experimental Setup

Datasets

Our experiments draw on KG–text aligned corpora, in which every textual context is paired with its structured-knowledge counterpart. Two general-domain benchmarks, **GenWiki** [34] and **DocRED** [104], are combined with two domain-specific ones, **SciERC** [55] and **CDR** [45], allowing the methods to be examined over broad as

Table 5.1: Summary statistics for the four benchmark datasets. Avg. Triples gives the mean number of triples per KG, and Avg. Length gives the mean word count per document.

Dataset	General-Domain		Domain-Specific	
	GenWiki	DocRED	SciERC	CDR
Train KGs	69,788	3,027	395	1,000
Test KGs	1,000	985	100	500
Train Triples	295,380	38,180	3,674	10,327
Test Triples	4,235	12,275	974	5,204
Avg. Triples	4.23	12.58	9.39	10.35
Avg. Length	24.22	198.33	130.99	231.66

well as specialized subject areas. Statistics for the four datasets appear in Table 5.1, while Figure 5.3 plots the per-dataset distributions of triple counts and document lengths.

Evaluation Metrics

Evaluation in this area has traditionally relied on exact, rule-driven string matching. Because matching of this kind cannot tolerate semantic variation, it is ill-suited to judging the KGs that GKE produces. Beyond that, matching at the surface level reveals nothing about the broader quality aspects of the extracted triples, so a richer evaluation protocol is called for. Accordingly, we employ the soft-matching and semantic-level metrics **G-BLEU** (BL), **G-ROUGE** (RO), and **G-BERTScore** (BS) [26], reporting precision, recall, and macro F1-score for each. The multi-dimensional evaluation suite GenRES [33] is used as well; it scores the extracted KG triples along **Topical Similarity** (TS), **Uniqueness** (US), **Factualness** (FS), **Granularity** (GS), and **Completeness** (CS). Table 5.2 lists the exact configurations under which each metric is

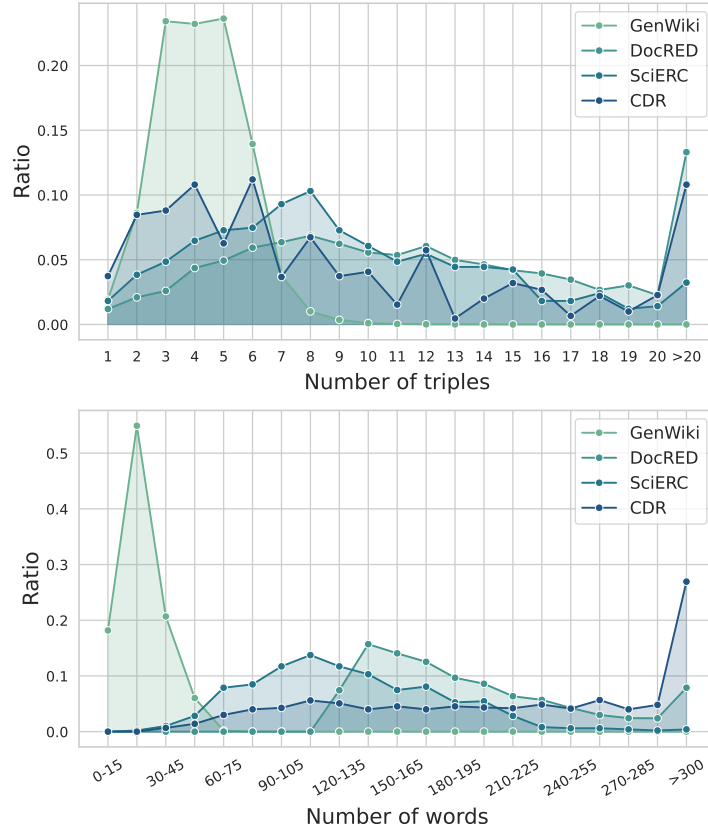


Figure 5.3: Statistical profiles of the datasets: the upper panel reports the mean triple count per document for each dataset, while the lower panel presents how document lengths are distributed.

computed. Wherever a setting is not stated explicitly here, the original implementations are followed.

G-BLEU. By adapting BLEU [65] to graphs, G-BLEU quantifies structural similarity through the n -gram overlap computed after predicted and gold triples are linearized into sentences. The emphasis of this metric lies on whether the generated triples reproduce the wording and structural patterns found in the reference triples. Throughout our experiments, the precision, recall, and macro F1-score derived from G-BLEU are denoted BL_P , BL_R , and BL_{F1} . As in [26], we use an n -gram order of $n = 4$.

G-ROUGE. In an analogous manner, G-ROUGE transfers ROUGE [47] to the KG setting: information coverage is gauged from n -gram co-occurrence counts taken over the predicted and reference triples once they have been linearized. The metric asks to what extent the essential content of the gold triples is retained by the generated ones. Its precision, recall, and macro F1-score are written throughout as RO_P , RO_R , and RO_{F1} . As in [26], the n -gram order is fixed at $n = 2$.

G-BERTScore. G-BERTScore [72] carries BERTScore [110] over to KG evaluation: each triple is viewed as its own sentence, and predicted and reference triples are then compared for their semantic similarity. The associated precision, recall, and macro F1-score are labeled BS_P , BS_R , and BS_{F1} in our experiments.

Table 5.2: Hyperparameter values and model choices adopted when computing each of the GenRES metrics.

Hyperparameter/Model		Setting
LDA topics	GenWiki	200
	DocRED	100
	SciERC	50
	CDR	50
Threshold ϕ		0.95
LLM		gpt-3.5-turbo-0125
Embedding		text-embedding-ada-002

Topical Similarity Score. The Topical Similarity Score (TS) captures how closely the triples in $\mathcal{G}_{\mathcal{D}}$ keep to the central themes of the document $\mathcal{D}$. Topic distributions are first estimated for $\mathcal{D}$ and $\mathcal{G}_{\mathcal{D}}$ with LDA, after which TS scores their divergence via:

$$\text{TS}(\mathcal{D}, \mathcal{G}_{\mathcal{D}}) = e^{-\text{KL}(\theta_{\mathcal{D}} \parallel \theta_{\mathcal{G}_{\mathcal{D}}})}, \quad (5.14)$$

in which $\theta_{\mathcal{D}}$ and $\theta_{\mathcal{G}_{\mathcal{D}}}$ are the respective topic distributions of $\mathcal{D}$ and $\mathcal{G}_{\mathcal{D}}$. Larger TS values signal that the triples and the document agree more strongly in topic.

Uniqueness Score. The Uniqueness Score (US) rewards diverse knowledge: triples that duplicate or nearly duplicate one another are penalized. With v_i denoting the embedding of triple τ_i in $\mathcal{G}_{\mathcal{D}}$, US takes the form:

$$\text{US}(\mathcal{G}_{\mathcal{D}}) = \frac{1}{n(n-1)} \sum_{i=1}^n \sum_{\substack{j=1 \\ j \neq i}}^n \mathbf{1}[\cos(v_i, v_j) < \phi], \quad (5.15)$$

in which $n = |\mathcal{G}_\mathcal{D}|$ and ϕ denotes a threshold on cosine similarity. The higher the US, the more varied the triples contained in $\mathcal{G}_\mathcal{D}$.

Factualness Score. The Factualness Score (FS) checks, for every triple $\tau \in \mathcal{G}_\mathcal{D}$, whether the document $\mathcal{D}$ provides support for it:

$$\text{FS}(\mathcal{D}, \mathcal{G}_\mathcal{D}) = \frac{1}{|\mathcal{G}_\mathcal{D}|} \sum_{\tau \in \mathcal{G}_\mathcal{D}} \mathbf{1}[\tau \text{ is supported by } \mathcal{D}]. \quad (5.16)$$

An LLM-based fact checker is consulted to make the per-triple support judgment. Larger FS values indicate that the triples in $\mathcal{G}_\mathcal{D}$ are factually more accurate.

Granularity Score. The Granularity Score (GS) examines whether triples stay overly composite or have instead been split into their atomic relational units. Let n_τ be the count of atomic triples that τ yields. This count is obtained by asking an LLM whether the triple in question admits further splitting into finer-grained ones. GS is then given by:

$$\text{GS}(\mathcal{G}_\mathcal{D}) = \frac{1}{|\mathcal{G}_\mathcal{D}|} \sum_{\tau \in \mathcal{G}_\mathcal{D}} \frac{1}{e^{n_\tau}}. \quad (5.17)$$

Higher GS values correspond to finer relational granularity and better decomposition.

Completeness Score. The Completeness Score (CS) quantifies the extent to which the relational content of the ground-truth KG $\mathcal{G}_\mathcal{D}^*$ is recovered by the extracted KG $\mathcal{G}_\mathcal{D}$. Writing v_τ for the embedding of an extracted triple τ and v_{τ^*} for that of a gold

triple τ^* , we evaluate CS through:

$$\text{CS}(\mathcal{G}_{\mathcal{D}}, \mathcal{G}_{\mathcal{D}}^*) = \frac{1}{|\mathcal{G}_{\mathcal{D}}^*|} \sum_{\tau^* \in \mathcal{G}_{\mathcal{D}}^*} \mathbf{1}[\max_{\tau \in \mathcal{G}_{\mathcal{D}}} \cos(v_{\tau}, v_{\tau^*}) \geq \phi], \quad (5.18)$$

with ϕ a cosine-similarity threshold. A larger CS means that the relational information of the ground-truth KG is covered more completely.

Baselines

Our comparison spans seven baselines in all: four plain LLMs together with three LLM-based approaches to KG construction. **Llama-2-7B**, **Llama-2-13B** [85]: With these open-source models, we probe how performance varies with model scale. **GPT-4o-mini**, **GPT-4o** [29]: These proprietary models exemplify an efficiency-focused variant and a high-capability variant, respectively. **EDC** [108], **KGGen** [59]: We run both under the default configurations shipped with their official releases, taking GPT-4o-mini as the backbone model. **GraphJudge** [26]: For this baseline as well, the default configuration is kept; KG construction is handled by GPT-4o-mini, while Llama-2-7B serves as the judging model that assesses the extracted triples.

Implementation Details

The KG refiner is built on Llama-2-7B [85], which we adapt through parameter-efficient fine-tuning with LoRA [24]. We train with the Adam optimizer [39] for 500 steps, of which the first 100 are warmup, and fix the learning rate at 3×10^{-4} . An effective batch size of 128 is obtained by combining a per-step micro-batch of 8 with gradient accumulation across 16 steps. We insert LoRA adapters into the `q_proj`

and v_{proj} projections, configured with $r = 8$, $\alpha = 16$, and dropout = 0.05. As validation data, we take 2,000 examples out of the GenWiki and DocRED training data, while SciERC and CDR instead rely on a 0.2 split. Each reported number averages three separate runs, with every run using its own random seed to build the fine-tuning dataset. Two NVIDIA V100 32GB GPUs were used for all experiments.

Knowledge Extraction. Because GraphRefine works independently of the extraction model, it can be layered post-hoc onto draft graphs originating from a wide array of GKE systems. For comparability across methods, the draft KGs in our experiments come from the Entity-Centric Text Denoising module of GraphJudge [26], powered by GPT-4o-mini.

Dataset Generation. For each instance, we generate a single synthetic sample for every inconsistency type, drawing on that instance’s gold KG and its source document. $\mathcal{T}_{\text{FIX}}$ is built by randomly picking one endpoint of the target triple—its head or tail entity—and corrupting it. $\mathcal{T}_{\text{REWRITE}}$ is produced by selecting a head or tail entity at random and applying expansion or shrinkage to its mention span within the document, where the word-level offset δ is drawn between 5 and 10; we do so only when the chosen entity’s mention genuinely occurs in the source text.

Prompt Templates

At refinement time, the model receives the source document alongside a candidate triple and must choose a suitable refinement operation using the document as its sole evidence. Depending on the operation it selects, the model then outputs the triple

unchanged, signals a deletion, or supplies a corrected or rewritten version. The refinement prompt is defined as follows:

Prompt Template for KG Refinement

```
Given a source document and a candidate triple extracted from it, decide
how to refine the triple based only on the document.

Choose exactly one operation: KEEP, DELETE, FIX, or REWRITE.
- KEEP: The triple is supported by the document and its representation is
appropriate; keep it unchanged.
- DELETE: The triple is not supported by the document or is factually
incorrect; remove it.
- FIX: The document supports a related fact, but the candidate triple is
semantically incorrect (e.g., an incorrect entity or relation); correct it
to match the document.
- REWRITE: The fact is supported, but the triple is ambiguous, imprecise,
or verbose; rewrite it into a clearer and more consistent form without
changing its meaning.

Return ONLY one item in the exact format:
['OPERATION', ['ENTITY 1', 'RELATIONSHIP', 'ENTITY 2']]
If OPERATION is DELETE, return:
['DELETE', ['NULL', 'NULL', 'NULL']]

Do not output any additional text or explain your decision.

document: $TEXT$
triple: $TRIPLES$
output:
```

To extract a draft KG, we prompt the LLM to identify the relationships among entities in a document and represent each relationship as a *[head, relation, tail]* triple. For each dataset, we adopt a distinct, domain-tailored prompt template following the prompt design conventions of GenRES [33].

Table 5.3: Comparison of overall performance across GenWiki, DocRED, SciERC, and CDR. KG quality is measured with BL/RO/BS (P/R/F1), and the multi-faceted GenRES scores (TS/US/FS/GS/CS) capture improvements from complementary angles. Bold marks the best score in each column and underlining the second-best.

Dataset	Method	BL _P	BL _R	BL _{F1}	RO _P	RO _R	RO _{F1}	BS _P	BS _R	BS _{F1}	TS	US	FS	GS	CS
GenWiki	Llama-2-7B	60.57	37.61	44.46	52.76	32.90	38.83	83.96	51.94	61.41	49.01	66.82	86.59	83.53	60.51
	Llama-2-13B	63.91	56.95	58.31	55.46	49.56	50.66	88.06	78.48	80.41	54.63	64.74	92.83	92.59	74.97
	GPT-4o-mini	64.39	62.22	62.02	55.57	53.91	53.62	87.65	84.68	84.43	71.76	<u>67.91</u>	98.74	95.22	77.36
	GPT-4o	64.78	63.51	62.80	56.00	54.97	54.30	87.04	85.32	84.38	<u>68.56</u>	66.60	<u>98.40</u>	95.03	80.65
	EDC	63.62	64.61	62.92	56.80	57.96	<u>56.30</u>	85.76	87.04	<u>84.79</u>	64.00	62.16	94.75	94.58	<u>81.70</u>
	KGGen	57.63	56.79	55.88	47.81	47.39	46.48	85.37	83.67	82.56	42.32	49.25	78.29	<u>98.58</u>	60.34
	GraphJudge	68.69	62.61	<u>63.40</u>	59.41	54.27	54.86	88.26	80.61	81.61	33.81	65.97	97.92	98.04	75.66
	GraphRefine	69.14	65.38	65.23	60.73	56.60	56.41	89.64	86.05	85.32	65.24	72.68	98.36	98.67	85.53
DocRED	Llama-2-7B	49.28	23.32	26.83	37.92	17.50	20.33	80.82	38.35	44.03	13.43	74.41	85.77	71.84	22.58
	Llama-2-13B	45.52	41.49	37.41	33.27	30.01	27.05	73.27	66.28	60.18	19.76	76.10	85.45	80.45	33.04
	GPT-4o-mini	34.72	58.33	39.98	25.19	43.24	29.20	54.55	89.48	62.35	46.70	<u>88.34</u>	96.25	82.76	45.72
	GPT-4o	35.24	59.34	40.71	25.87	44.41	30.06	54.44	89.74	62.48	<u>43.95</u>	86.91	96.63	84.57	48.79
	EDC	34.03	59.62	39.88	25.06	45.02	29.61	52.82	90.24	61.37	37.75	87.47	90.61	81.78	50.78
	KGGen	41.13	49.84	41.07	27.62	33.79	27.62	67.75	81.58	67.62	19.99	71.86	69.66	92.24	32.42
	GraphJudge	47.87	56.20	<u>47.57</u>	36.09	42.57	<u>35.86</u>	70.46	82.41	<u>70.06</u>	18.83	80.25	95.24	<u>96.02</u>	<u>54.74</u>
	GraphRefine	50.41	60.26	52.94	37.28	44.60	38.29	71.23	85.34	73.84	36.72	89.46	<u>96.38</u>	97.10	59.63
SciERC	Llama-2-7B	64.77	26.77	34.65	58.65	24.71	31.61	89.79	38.98	49.24	58.29	50.88	78.77	73.60	37.10
	Llama-2-13B	64.36	28.79	36.37	55.92	24.62	31.24	91.07	41.94	52.14	63.69	52.18	75.95	76.08	29.75
	GPT-4o-mini	55.68	58.92	54.02	48.17	51.49	46.70	76.83	82.13	74.82	<u>85.03</u>	<u>93.18</u>	97.14	72.68	57.64
	GPT-4o	55.49	59.23	54.10	48.11	52.25	47.01	76.87	82.51	75.10	85.90	92.34	98.22	69.99	<u>60.42</u>
	EDC	54.36	62.91	<u>55.08</u>	47.59	55.47	48.22	73.83	85.77	74.91	75.20	91.96	93.46	69.80	64.64
	KGGen	53.92	56.62	52.33	46.22	48.84	44.82	77.32	82.08	<u>75.24</u>	80.93	90.14	94.05	81.93	44.71
	GraphJudge	59.26	53.69	52.91	52.43	47.11	46.51	82.13	74.84	73.47	74.38	87.28	<u>97.47</u>	<u>83.98</u>	49.74
	GraphRefine	57.85	58.13	55.43	51.15	50.32	<u>47.76</u>	80.64	77.34	76.29	79.61	94.42	97.31	86.33	56.75
CDR	Llama-2-7B	19.88	8.05	9.02	15.17	6.01	6.77	43.28	17.58	20.06	33.73	89.39	42.86	72.75	6.61
	Llama-2-13B	34.26	23.07	22.01	25.39	16.95	16.12	69.59	45.40	44.34	59.86	72.55	81.78	60.14	18.42
	GPT-4o-mini	30.51	54.14	34.88	21.04	38.38	24.22	50.76	87.58	57.59	82.86	91.55	<u>96.33</u>	74.76	32.50
	GPT-4o	32.94	53.67	36.22	23.21	38.70	25.67	53.93	85.53	58.87	<u>79.47</u>	87.80	96.80	76.28	35.03
	EDC	29.35	54.24	33.84	20.38	38.86	23.70	48.91	87.62	55.88	78.21	92.22	92.45	68.00	<u>35.68</u>
	KGGen	37.49	50.98	38.38	25.40	35.40	26.18	61.57	81.93	62.68	67.82	82.66	86.58	88.69	29.33
	GraphJudge	38.78	53.84	<u>40.16</u>	27.62	39.13	28.74	61.03	82.96	<u>62.93</u>	61.33	84.62	93.36	<u>91.17</u>	35.17
	GraphRefine	38.12	55.36	42.73	28.49	40.71	31.26	61.24	84.22	64.10	66.54	<u>92.15</u>	94.21	93.40	36.54

5.3.2 Experimental Results

Our empirical study of GraphRefine is organized around three research questions.

RQ1. To what degree does GraphRefine suppress the factual inconsistencies produced during GKE and raise overall KG quality? **RQ2.** Across how broad a range of GKE models can GraphRefine serve as a post-hoc refiner? **RQ3.** To what extent do GraphRefine’s gains carry over when the model is transferred to a previously unseen dataset?

Overall Performance Comparison (RQ1)

As reported in Table 5.3, GraphRefine curbs the factual inconsistencies generated during GKE and simultaneously lifts KG quality along several dimensions. On the F1-based G-Scores (BL/RO/BS), it outperforms the existing construction methods as well as GraphJudge—the deletion-oriented post-processing baseline—on all four datasets. Head-to-head against GraphJudge, GraphRefine raises precision without giving up recall, which testifies that refining through operations is preferable to filtering that depends on deletion alone. The multi-faceted GenRES scores tell a similar story: factualness (FS) stays high while granularity (GS) and uniqueness (US) improve substantially. Refinement therefore accomplishes more than preserving factual accuracy: it reshapes triples into a more atomic, mutually consistent form while paring back redundancy and settling unstable surface expressions. Strong factual coverage (CS) is achieved by GraphRefine as well. Overall, the evidence indicates that GraphRefine dampens inconsistencies yet keeps the essential content, softening the tension between removing noise and preserving coverage.

To probe these effects further, we performed a human evaluation on KGs sampled from two datasets belonging to different domains, DocRED and SciERC. The distribution of inconsistency types across our annotations is plotted in Figure 5.4. While GraphJudge relieves inconsistencies at the correctness level (hallucination and misidentification, for example), it accomplishes comparatively little against representation-level ones such as ambiguity and verbosity. This pattern implies that deletion by itself is often insufficient for representation-level problems, which instead demand rewrit-

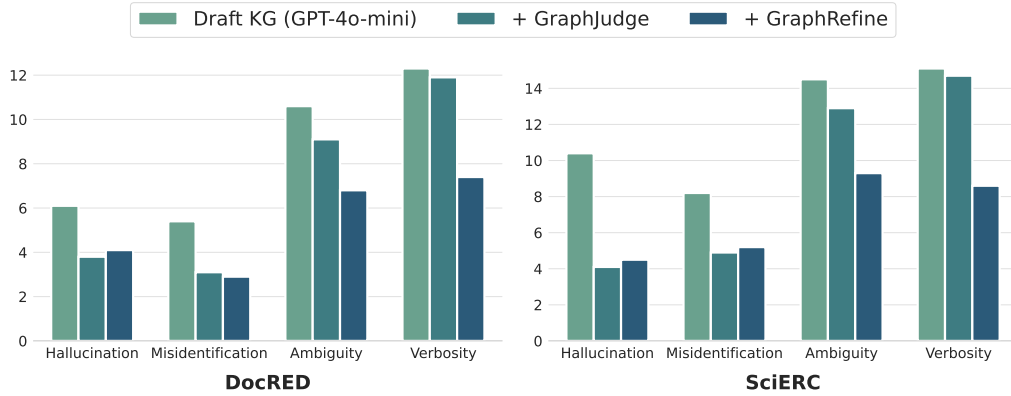


Figure 5.4: Across the manually annotated KGs of DocRED and SciERC, the breakdown of the various factual inconsistency types; here “+” denotes drafts that have undergone refinement.

ing. GraphRefine, by comparison, drives down the shares of both inconsistency categories, demonstrating that refinement through operations captures errors that a purely deletion-based filter can miss.

Model-Agnostic Applicability (RQ2)

Figure 5.5 compares, along multiple aspects, the KGs generated by two different base KGE models—covering both the raw drafts and the versions obtained once GraphJudge or GraphRefine is applied. Broadly speaking, GraphRefine delivers dependable improvements irrespective of the extractor, benefiting the accuracy-oriented scores (BL/RO/BS-F1) and the multi-faceted GenRES measures alike. For both datasets, it keeps factualness (FS) steady or better while pushing up granularity (GS) and uniqueness (US), and coverage (CS) frequently rises without any loss in quality. GraphJudge, in turn, does reduce correctness-level errors, yet the triples it removes tend to

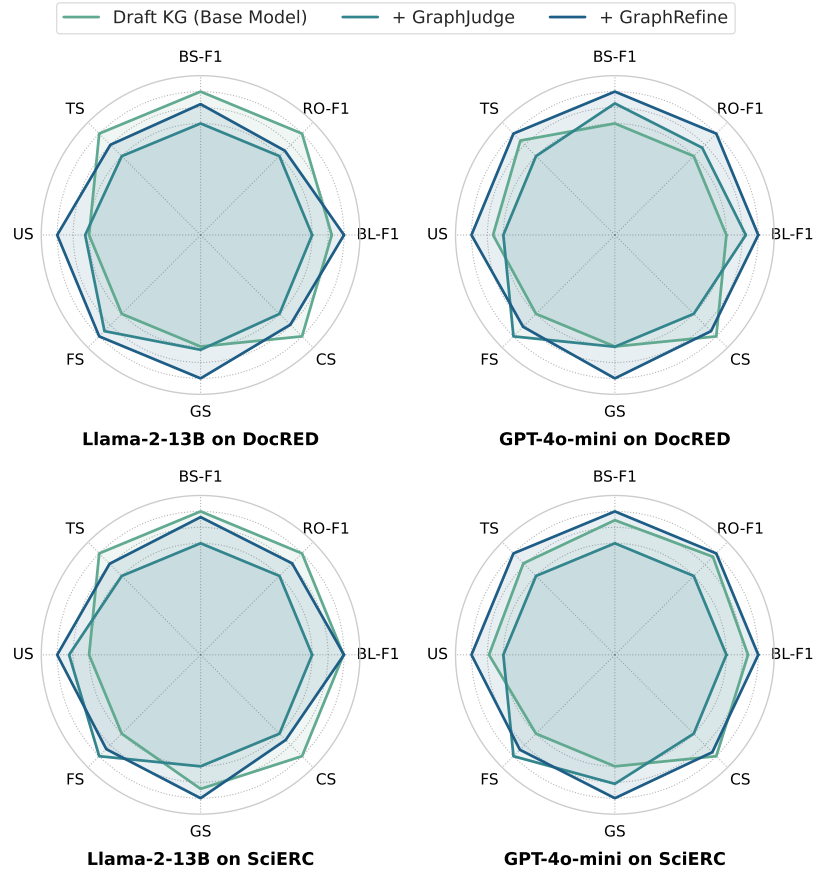


Figure 5.5: Multi-dimensional comparison between draft KGs and their post-hoc refined counterparts. For clearer visualization, we min-max normalize each metric and then map it into the $[0.7, 0.9]$ interval.

Table 5.4: Cross-dataset generalization measured by transferring a model between domain pairs (train $\rightarrow$ test); Δ indicates the difference relative to the in-domain training result.

Src $\rightarrow$ Tgt	BL _{F1} (Δ)	RO _{F1} (Δ)	BS _{F1} (Δ)
Doc $\rightarrow$ Gen	64.60($\downarrow$ 0.63)	55.62($\downarrow$ 0.79)	84.19($\downarrow$ 1.13)
Gen $\rightarrow$ Doc	52.12($\downarrow$ 0.82)	37.54($\downarrow$ 0.75)	72.59($\downarrow$ 1.25)
Doc $\rightarrow$ CDR	41.01($\downarrow$ 1.72)	29.45($\downarrow$ 1.81)	62.92($\downarrow$ 1.18)
CDR $\rightarrow$ Doc	52.41($\downarrow$ 0.53)	37.91($\downarrow$ 0.38)	72.85($\downarrow$ 0.99)
Gen $\rightarrow$ Sci	56.46($\uparrow$ 1.03)	48.45($\uparrow$ 0.69)	77.87($\uparrow$ 1.58)
Sci $\rightarrow$ Gen	58.12($\downarrow$ 7.11)	51.40($\downarrow$ 5.01)	76.17($\downarrow$ 9.15)

drag overall KG quality down. The drop is sharper for Llama-2-13B: its draft KGs are already small, so every additional removal costs coverage and representational quality more directly. It is worth noting that even with smaller models, GraphRefine lifts factualness together with GS and US, underscoring how well it can correct and rewrite triples on the basis of the document. In short, whether the extractor is an open-source or a proprietary LLM, GraphRefine produces consistent gains, which supports its role as a model-agnostic refinement module applied after construction.

Cross-Dataset Generalization (RQ3)

How well GraphRefine generalizes across datasets is summarized in Table 5.4. When transfer stays within a domain (Gen $\rightarrow$ Doc and Doc $\rightarrow$ Gen), the transferred models perform essentially as well as their in-domain counterparts, indicating that transfer between datasets of similar domains is stable. Performance dips somewhat once the transfer crosses domains (CDR $\rightarrow$ Doc and Doc $\rightarrow$ CDR), yet the results remain largely consistent from metric to metric. Such behavior suggests that what GraphRe-

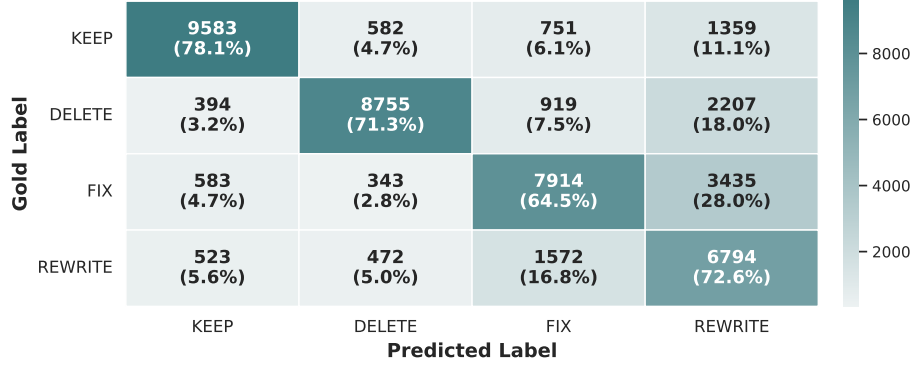


Figure 5.6: Confusion matrix over refinement operations, computed for GraphRefine on the synthetically labeled test split of DocRED.

fine acquires is a document-grounded refinement skill rather than surface regularities specific to one dataset. A striking case is training on GenWiki and evaluating on SciERC (Gen $\rightarrow$ Sci), where the transferred model actually beats the SciERC in-domain model—a sign that exposure to a more heterogeneous training distribution improves transferability. The reverse direction (Sci $\rightarrow$ Gen), by contrast, suffers a markedly steeper decline, showing that transfer becomes harder as the gap between domains grows. These observations point to mixed-domain training or domain adaptation as promising directions for narrowing such gaps in future work.

Further Analysis

To understand the source of GraphRefine’s gains, we next examine how the individual refinement operations behave and trace their respective contributions to the evaluation metrics.

Operation-Level Confusion Analysis. GraphRefine’s operation predictions are examined on synthetic test samples constructed from DocRED. Figure 5.6 presents the resulting confusion matrix: the model separates edit from non-edit decisions reasonably well but shows a noticeable pull toward REWRITE. For triples whose gold label is KEEP or DELETE, the correct non-edit prediction dominates (78.1% and 71.3%), although a considerable fraction (11.1% and 18.0%) is assigned REWRITE instead. Of the two edit operations, FIX turns out to be the harder one: 64.5% of such cases are classified correctly, while 28.0% end up as REWRITE, which reflects how much meaning correction and surface-form normalization overlap when both are judged against document evidence. REWRITE itself is comparatively reliable (72.6%), indicative of a consistent rewriting policy. Taken as a whole, the model gravitates toward REWRITE-style refinement—beneficial for representation quality, though better calibration could avert edits that are not strictly necessary.

BLEU/ROUGE/BERTScore. All of the operations have the potential to raise BL/RO/BS, since these metrics measure how similar the output is to the gold triples at the surface and semantic levels. Unsupported triples are eliminated by DELETE, faulty entities/re-lations are repaired by FIX, and superfluous modifiers are pruned by REWRITE, each step bringing the output nearer to the references. The gains on BL/RO/BS therefore combine two effects: less noise and closer representational alignment.

Topical Similarity (TS). The comparatively low TS values in Table 5.3 stem mainly from the output characteristics of the base extractor rather than from GraphRefine it-

self. TS captures the topical fit between extracted triples and the document, so when topical coherence or surface forms are already unstable at extraction time, refinement has little leverage for pushing TS upward. Figure 5.7 exhibits the same tendency: applied on top of extractors with high TS (e.g., GPT-4o-mini and GPT-4o), GraphRefine can in fact increase TS. In other words, GraphRefine is not inherently detrimental to TS—the declines observed are attributable chiefly to the extractor beneath it.

Uniqueness (US). GraphRefine drives US higher by cutting redundancy among the triples that populate the extracted KG. Once REWRITE has standardized the triples and rendered them more atomic, one and the same fact rarely ends up encoded in several distinct surface forms. A smaller contribution comes from FIX: by converting erroneous triples into legitimate ones, it curbs noisy variants and yields a clearer separation between distinct facts. The net effect is a reduction in duplicated paraphrases, together with fewer instances in which separate meanings are conflated into nearly identical forms—and both effects raise US.

Factualness (FS). GraphRefine does not top the FS ranking in every setting, yet its factualness remains robust throughout. Refinement anchored in the document suppresses content lacking support while statements the document backs are kept. DELETE and FIX in particular remove or repair unmistakable factual errors, which is why FS stays high in practice.

Granularity (GS). KGs generated through LLM-based GKE often contain overly descriptive phrasing, and several attributes are sometimes packed into a single triple.

The pronounced GS gains of GraphRefine come from REWRITE, which reorganizes such outputs into clearer, more atomic triples—dropping needless modifiers and surfacing the core relation—without departing from the source document. Finer triple granularity follows, and GS rises accordingly.

Completeness (CS). The strength of CS reflects the fact that GraphRefine goes beyond deletion when editing, so supported information survives refinement. Instead of discarding triples that are partly wrong yet recoverable, FIX converts them into statements the document supports. REWRITE, for its part, regularizes surface forms—disambiguating mention spans and shortening verbose phrasing—so that outputs approach the ground truth. Working together, the operations cut errors without eroding coverage, and CS remains strong as a result.

GraphJudge vs. GraphRefine

In Figure 5.7, GraphJudge and GraphRefine are contrasted over multiple metrics when applied to the draft KGs of every combination of base extractor and dataset.

Overall Results. In the majority of settings, the polygon traced by GraphRefine is the largest, reflecting steady improvements on the accuracy metrics (BL/RO/BS-F1) and likewise on the broader range of GenRES scores. Granularity (GS) climbs repeatedly, as does uniqueness (US), while factualness (FS) stays level or edges upward. Coverage (CS) likewise stays whole or even expands—a sign that the gains arise because triples are corrected and rewritten with the document as anchor, not merely deleted. GraphJudge does trim correctness-level errors, but the filtering it performs



Figure 5.7: Multi-dimensional comparisons of draft KGs against their post-hoc refined versions produced by GraphJudge or GraphRefine, covering every dataset (GenWiki, DocRED, SciERC, and CDR) and every base extractor (Llama-2-7B, Llama-2-13B, GPT-4o-mini, and GPT-4o). Every metric is min-max normalized and then mapped onto the range $[0.7, 0.9]$ for readability.

through deletion frequently depresses CS and tends to be accompanied by weaker representation quality (GS/US). This trade-off is most evident for the Llama-2 models (7B and 13B): because their draft KGs are comparatively small at the outset, each additional deletion erodes coverage more severely. Table 5.5 sets the average triple count per KG side by side for the raw drafts and for what GraphJudge and GraphRefine output. As the numbers confirm, GraphJudge sharply reduces the triple count in absolute terms, whereas GraphRefine keeps it largely intact.

Results by Base Extractor. With Llama-2-7B and Llama-2-13B as extractors, the gains from GraphRefine are larger on every dataset, and GS and US improve consistently. Although GraphJudge can lift particular accuracy metrics, those lifts commonly arrive together with falling CS and weaker representation-level scores, exposing the limits of refinement that only deletes. With GPT-4o-mini and GPT-4o, whose draft KGs are already of high quality, GraphRefine still adds steady extra gains, most dependably in representation quality (GS/US). GraphRefine is therefore not tightly bound to the error patterns of any particular model. Rather, by anchoring its corrections and rewrites in the document, GraphRefine can act as a post-hoc component across a wide variety of base extractors.

Summary across Datasets. Taken over GenWiki, DocRED, SciERC, and CDR, GraphRefine reliably boosts BL/RO/BS-F1, holds factualness (FS) steady, and lifts granularity (GS) together with uniqueness (US). Coverage (CS), too, is usually held steady or even raised, which confirms that these gains stem from editing and rewrit-

Table 5.5: Per-KG triple counts (averages) for the draft KGs produced by each base extractor on every dataset, together with the counts remaining once GraphJudge or GraphRefine has been applied.

Dataset	Model	Number of Triples		
		Draft KG	GraphJudge	GraphRefine
GenWiki	Llama-2-7B	2.35	2.08	2.16
	Llama-2-13B	3.75	3.27	3.34
	GPT-4o-mini	4.14	3.79	3.95
	GPT-4o	4.22	3.97	4.08
DocRED	Llama-2-7B	4.84	3.79	4.11
	Llama-2-13B	11.20	8.01	9.42
	GPT-4o-mini	20.48	15.82	18.13
	GPT-4o	20.51	16.17	19.40
SciERC	Llama-2-7B	3.32	2.74	2.86
	Llama-2-13B	3.77	3.33	3.38
	GPT-4o-mini	9.68	7.20	8.01
	GPT-4o	9.83	7.49	8.54
CDR	Llama-2-7B	1.47	1.14	1.25
	Llama-2-13B	4.63	3.67	3.81
	GPT-4o-mini	15.88	10.72	12.47
	GPT-4o	14.46	10.94	12.11

Table 5.6: Operation-level ablation of GraphRefine evaluated on DocRED and SciERC; the abbreviations D, F, and R stand for DELETE, FIX, and REWRITE.

Dataset	Method	BL _{F1}	RO _{F1}	BS _{F1}	TS	US	FS	GS	CS
DocRED	D+F+R	52.94	38.29	75.83	36.72	89.46	96.38	97.10	59.63
	D+F	50.10	36.91	72.69	25.43	83.59	96.17	96.35	57.49
	D+R	49.84	37.23	71.08	27.29	88.13	95.32	97.21	55.15
	D	47.32	35.80	68.64	18.31	78.49	96.10	95.82	52.96
SciERC	D+F+R	55.43	48.07	76.29	79.61	94.42	97.31	86.33	56.75
	D+F	53.26	46.74	74.85	74.58	88.25	97.20	83.76	53.82
	D+R	52.84	46.38	73.18	76.14	92.61	96.45	85.61	50.37
	D	51.76	46.12	72.80	72.69	86.03	97.35	83.39	48.13

ing kept aligned with the document rather than from mere deletion. Across domains, then, GraphRefine delivers reliable improvements when applied after construction.

Ablation Study

An ablation over the refinement operations of GraphRefine is given in Table 5.6. We refer to the complete model—the one trained on DELETE, FIX, and REWRITE together—as D+F+R. The variants D+F and D+R omit REWRITE and FIX from training, respectively. The variant D relies on DELETE only.

Overall Results. Of all configurations, D+F+R attains the strongest results on both datasets, which suggests that equipping the refiner with a wider repertoire of operations benefits KG quality more than deletion can on its own. In comparison with D, the full model shows its largest margins on the accuracy metrics (BL/RO/BS), while also topping the representation-related indicators TS and US. The conclusion is that GraphRefine works best when correction grounded in the document is coupled with rewriting rather than resting on deletion alone.

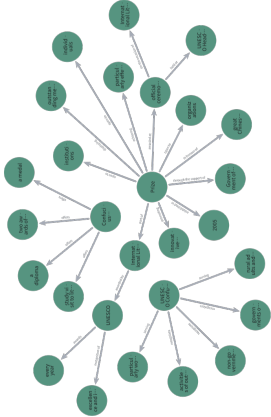
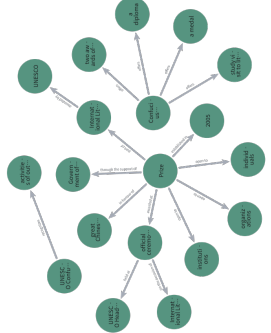
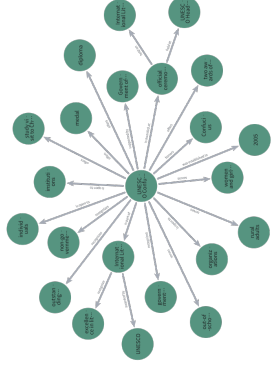
Limitations of Deletion-Only Refinement. On both datasets, the deletion-only variant D posts the lowest BL/RO/BS scores, and its TS and US fall especially sharply—a sign of deteriorated representation quality. Its coverage (CS), too, is the lowest of any variant, which shows that removal-driven refinement, while capable of reducing errors, typically sacrifices retained information and structural quality in return. D+F+R does the reverse: it lifts metrics on both the correctness and representation fronts while keeping CS higher, reinforcing the argument that successful refinement has to go beyond deletion to include editing steps.

Contributions of FIX and REWRITE. Since both D+F and D+R beat D, FIX and REWRITE evidently contribute in complementary ways. The D+F variant improves BL/RO/BS consistently and pushes FS and CS higher as well—evidence that FIX draws on document evidence to repair faulty entities or relations, strengthening factual accuracy while coverage is preserved. D+R instead excels on the representation-oriented indicators, US and GS above all, demonstrating that REWRITE removes redundancy and recasts triples into surface forms that are more atomic and internally consistent. Even so, CS falls for D+R in some settings, hinting that rewriting on its own cannot always retain all supported information; the most stable improvements emerge when it operates alongside FIX.

Case Study

Focusing on one source document, Table 5.7 sets three outputs side by side: the draft KG from GPT-4o-mini, the result of GraphJudge, and the result of GraphRe-

Table 5.7: Case study of KG extraction and refinement for a single document, comparing the draft KG with the outputs of GraphJudge and GraphRefine.

Original Document: The UNESCO Confucius Prize for Literacy recognizes the activities of outstanding individuals, governments or governmental agencies and non-governmental organizations (NGOs) working in literacy serving rural adults and out-of-school youth, particularly women and girls. The Prize was established in 2005 through the support of the Government of the People’s Republic of China in honour of the great Chinese scholar Confucius. It is part of the International Literacy Prizes, which UNESCO awards every year in recognition of excellence and inspiring experiences in the field of literacy throughout the world. The Confucius Prize offers two awards of US\$ 20,000 each, a medal and a diploma, as well as a study visit to literacy project sites in China. The Prize is open to institutions, organizations or individuals displaying outstanding merit in literacy, achieving particularly effective results and promoting innovative approaches. The selection of prizewinners is made by an International Jury appointed by UNESCO’s Director-General, which meets in Paris once a year. The Prize is awarded at an official ceremony held for that purpose at UNESCO Headquarters in Paris on the occasion of International Literacy Day (8 September).	Draft KG (GPT-4o-mini) 	GraphJudge 	GraphRefine 
--	---	--	--

fine. Although the draft KG already covers a large share of the document’s key facts, the same entity is mentioned inconsistently, which pulls the graph apart. For instance, one and the same target surfaces under three labels—*Prize*, *Confucius Prize*, and *UNESCO Confucius Prize for Literacy*—so its related facts end up scattered over several subject nodes, and the graph’s connectivity suffers.

GraphJudge. For GraphJudge, quality gains on the draft KG come mainly from removing triples. As Table 5.7 shows, it discards triples judged dubious or weakly supported, leaving a smaller and cleaner graph. While this pruning reduces noise and factual errors, it simultaneously eliminates document-grounded details—among them several beneficiary groups and descriptive attributes. As a result, coverage shrinks and the entity in focus receives a thinner description. Moreover, deletion by itself leaves the reference-splitting problem intact: the details remain spread over *Prize*, *Confucius Prize*, and *UNESCO Confucius Prize for Literacy*.

GraphRefine. GraphRefine instead corrects and rewrites problematic triples through FIX and REWRITE operations. As illustrated in Table 5.7, ambiguous mentions such as *Prize* are replaced with the full entity name *UNESCO Confucius Prize for Literacy*, consolidating previously fragmented information and improving graph connectivity. GraphRefine also rewrites overly broad or verbose expressions into clearer and more concise triples. For example, conflated beneficiary groups are separated into distinct entities, while unnecessary modifiers are removed without altering the underlying facts. As a result, GraphRefine preserves most of the information contained in

the draft KG while producing a more coherent, compact, and well-connected graph.

5.4 Summary

This chapter introduced GraphRefine, a refinement method applied after construction to lessen the factual inconsistencies found in KGs generated through LLM-based GKE. Studying the graphs produced by numerous GKE models over multiple datasets allowed us to organize the observed factual inconsistencies into a systematic typology. On the basis of this typology, we adapted an LLM into a KG refiner by training it on synthetic supervision that is specific to each operation, enabling it to tag every triple with a refinement decision and produce the corresponding edited triple. Across experiments that draw on a wide range of evaluation measures, GraphRefine is shown to lift the overall quality of the KG, holding inconsistencies in check while keeping the essential information intact. Its benefits extend past correctness alone: representation-level quality—granularity and uniqueness among others—also improves, and the resulting triples become more atomic and mutually consistent. Finally, the consistent improvements seen from one base extractor and dataset to the next reinforce that GraphRefine is model-agnostic and transfers well across datasets—traits that lend it broad practical utility.

6. Conclusions

This dissertation is motivated by a fundamental gap between KGs and real-world facts: although KGs are widely used as structured repositories of factual knowledge, large-scale KGs constructed and updated through automated pipelines inevitably contain erroneous, outdated, and context-mismatched triples. Such inaccuracies compromise the reliability of KGs and propagate errors to downstream applications, making KG accuracy enhancement a critical prerequisite for trustworthy knowledge-centric systems. To address this challenge, we proposed a comprehensive and unified framework for improving KG accuracy toward better alignment with real-world facts, advancing three interconnected directions: **accuracy improvement**, **validation**, and **refinement**. Crucially, our framework moves beyond plausibility-oriented assessment based solely on internal KG patterns, and instead emphasizes evidence-grounded verification and faithful, coverage-preserving updates.

The core perspective of this dissertation is that maintaining accurate KGs at scale requires an end-to-end capability: efficiently discovering candidate errors under limited supervision, validating their factual correctness with explicit external evidence and reliable reasoning, and refining erroneous triples through controlled actions that improve correctness without unnecessary information loss. Accordingly, we develop complementary methods that address each stage while forming a coherent pipeline for practical KG maintenance.

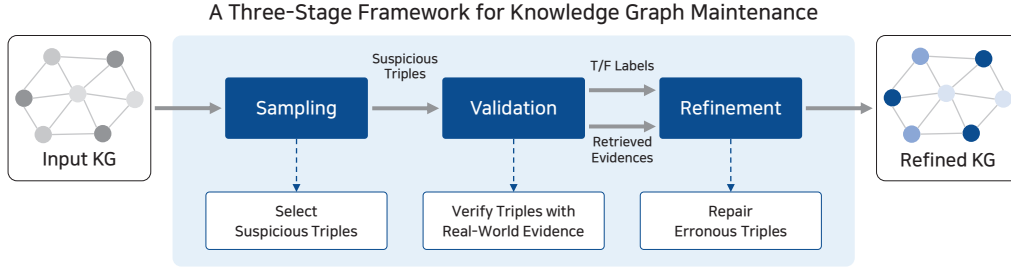


Figure 6.1: Unified framework for improving the factual consistency of KGs. The framework transforms an input KG into a refined KG through three connected stages: suspicious triple sampling, evidence-grounded validation, and evidence-supported refinement. This pipeline enables systematic KG maintenance by identifying, verifying, and correcting triples that may be inconsistent with real-world facts.

6.1 Key Contributions

Taken together, the three studies presented in this dissertation form a unified framework for KG accuracy enhancement. As illustrated in Figure 6.1, the framework transforms an input KG into a refined KG through three connected stages: sampling suspicious triples, validating them with retrieved evidence, and refining the KG based on the validation results and supporting evidence. Within this pipeline, AIKA addresses the question of *which triples should be inspected first*, RAVER addresses *how triples can be reliably verified with external evidence*, and GraphRefine addresses *how erroneous triples should be corrected while preserving valid information*. Thus, the contributions of this dissertation are not isolated solutions, but complementary components of a systematic KG maintenance process.

The research journey began by addressing the challenge of improving KG accuracy under limited supervision. In **AIKA**, we proposed an active learning frame-

work that prioritizes informative triples for verification and learning, enabling cost-effective improvement of KG accuracy with minimal human effort. AIKA systematically allocates annotation budgets to triples that most effectively improve detection quality, mitigating the inefficiency of passive labeling and reducing redundancy in verification. Through extensive evaluations under varying supervision budgets, AIKA demonstrated consistent gains in error detection effectiveness and enabled substantial accuracy improvements with significantly reduced verification cost.

Subsequently, we shifted focus to verifiable KG correctness judgments grounded in external evidence. We developed **RAVER**, a retrieval-augmented KG validation framework that grounds correctness decisions in explicitly retrieved evidence and performs evidence-grounded reasoning for factual verification. A central contribution of RAVER is retriever alignment, which tightens the coupling between evidence retrieval and downstream reasoning by optimizing retrieval behavior for what the reasoner truly needs, namely decisive and relevant evidence rather than merely topically similar documents. By reducing the mismatch between retrieval and reasoning, RAVER improves both the reliability and interpretability of validation, and yields decisions that are more consistently supported by evidence across diverse datasets and error regimes.

Finally, we addressed the limitation of deletion-centric KG maintenance. To preserve KG coverage while correcting errors, we proposed **GraphRefine**, an operation-guided refinement framework that performs fine-grained and controlled updates to erroneous triples. GraphRefine formulates refinement as an explicit operation selection problem (e.g., KEEP, DELETE, FIX, REWRITE), and conditions editing on the selected

operation to ensure targeted and faithful modifications. This operation-guided design enables correcting facts when feasible and reserving deletion for cases where repair is not possible, thereby mitigating the common precision–coverage trade-off. Comprehensive evaluations showed that GraphRefine produces more faithful and effective KG updates than deletion-only baselines, improving correctness while retaining substantially more valid information.

In summary, this dissertation contributes a systematic approach to KG accuracy enhancement that unifies sample-efficient accuracy improvement, evidence-grounded validation with aligned retrieval and reasoning, and coverage-preserving refinement via controlled editing. Across extensive experiments on diverse benchmarks, the proposed methods consistently advance the state of the art in effectiveness, verifiability, and practicality of KG accuracy maintenance, bringing KGs closer to trustworthy alignment with real-world facts.

6.2 Future Works

The ultimate goal of this dissertation is to bridge the gap between KGs and real-world facts in a principled, scalable, and reliable manner. While this dissertation has addressed key aspects of KG factuality, including evidence-grounded validation and structured refinement, several important directions remain for future research.

In particular, future work can extend the scope of this dissertation in three directions. First, although this work primarily relied on textual evidence as a source of real-world facts, future KG validation and refinement systems should incorporate multi-modal evidence. Second, recent advances in agentic AI suggest that KG main-

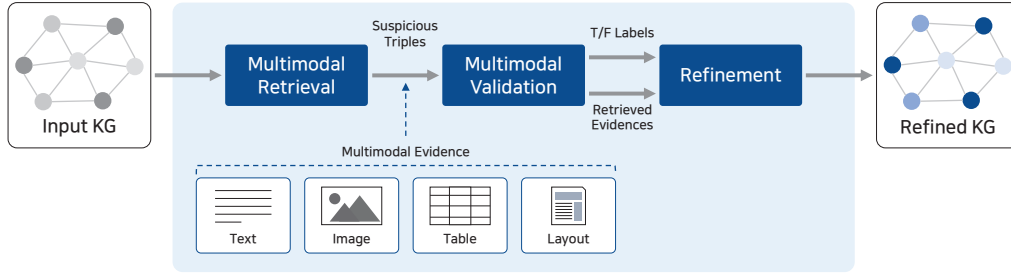


Figure 6.2: Conceptual overview of a multi-modal KG maintenance pipeline that integrates suspicious triple selection, evidence-grounded validation, and refinement.

tenance can benefit from autonomous and collaborative agent-based workflows. Third, KGs themselves can play a central role in improving the reliability, controllability, and factual grounding of AI systems.

6.2.1 Extending Evidence from Text to Multi-Modal Sources

This dissertation primarily considered textual corpora as sources of real-world evidence. Although text provides rich contextual information for factual verification, many facts are more reliably captured in other forms, including tables, images, videos, audio, structured logs, and semi-structured documents. These sources can provide complementary evidence for numerical, temporal, visual, spatial, and event-related facts.

Future work should therefore extend KG validation and refinement beyond text-centric pipelines toward multi-modal evidence grounding. As illustrated in Figure 6.2, a future KG maintenance pipeline could identify suspicious triples, retrieve relevant evidence across modalities, validate the triples based on the retrieved evidence, and use the results to guide refinement. For example, tables may provide reliable evidence

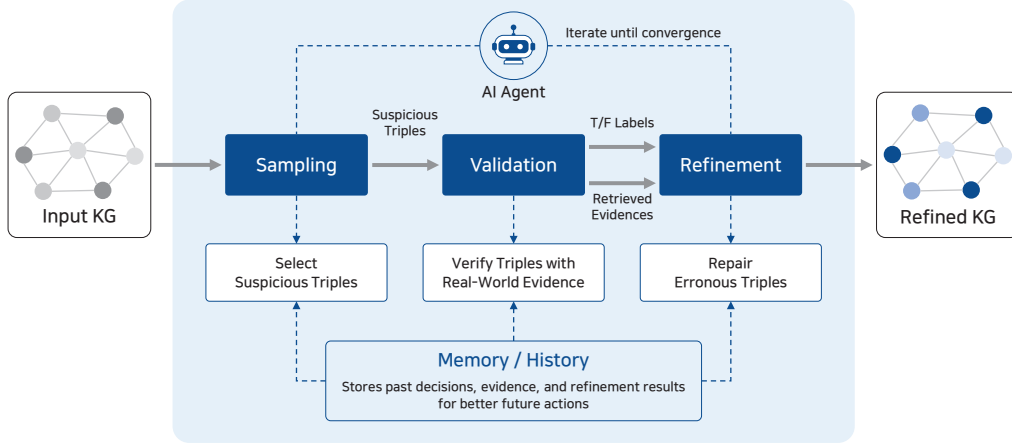


Figure 6.3: Conceptual overview of an agentic KG maintenance framework that iteratively coordinates triple selection, validation, and refinement using memory and feedback from previous actions.

for numerical and temporal facts, while images and visual metadata can support entity attributes, spatial relations, and event-related information.

A central challenge is how to retrieve, represent, and reason over heterogeneous evidence in a unified manner. Future systems require retrieval mechanisms capable of identifying relevant evidence across modalities and reasoning models that can integrate heterogeneous inputs while accounting for differences in reliability, granularity, and temporal validity. Extending KG maintenance to multi-modal evidence can reduce reliance on text-only signals and improve robustness when textual evidence is incomplete, ambiguous, outdated, or unavailable.

6.2.2 Agentic KG Maintenance

Another promising direction is to leverage agentic AI for KG maintenance. The methods developed in this dissertation can be viewed as core components of a broader

KG maintenance process: suspicious triples should be selected, relevant evidence should be retrieved, factual consistency should be assessed, and erroneous or ambiguous triples should be refined. In real-world settings, however, this process is continuous rather than one-time, as KGs evolve with newly emerging facts, outdated information, and newly available evidence.

Future work may therefore formulate KG maintenance as an agentic and iterative workflow. As illustrated in Figure 6.3, an AI agent can orchestrate the overall maintenance process by repeatedly selecting suspicious triples, validating them against real-world evidence, and refining erroneous triples until a convergence criterion is reached. The agent can further use memory and interaction history, including previous validation decisions, retrieved evidence, and refinement results, to improve subsequent actions and avoid repeated or inconsistent decisions.

Such agentic maintenance is particularly useful when KG quality control requires multiple capabilities, including retrieval, reasoning, editing, consistency checking, and human-in-the-loop decision making. These capabilities may be coordinated by a single orchestrating agent or distributed across specialized agents, such as retrieval, validation, refinement, and auditing agents. A central research challenge is how to design effective orchestration policies: which action or agent should be invoked, when additional evidence is needed, how conflicting outputs should be resolved, and when uncertain cases should be escalated to human experts. By integrating agentic decision making, iterative feedback, and memory into the KG lifecycle, future research can move toward scalable, semi-autonomous KG maintenance systems that continuously align KGs with changing real-world facts.

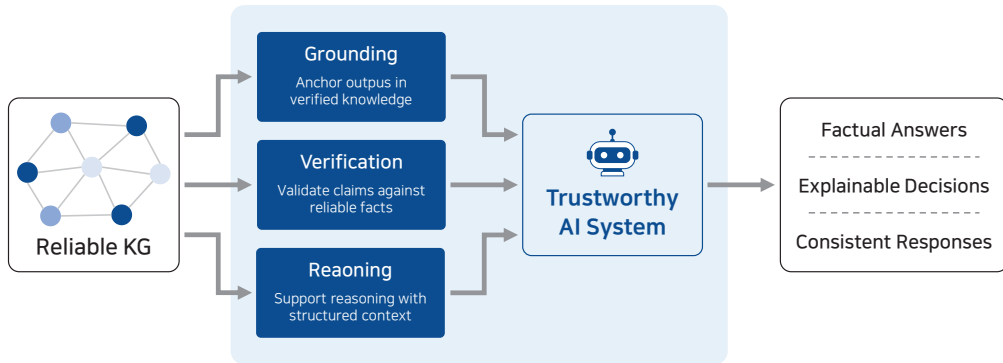


Figure 6.4: A reliable KG enhances the trustworthiness of AI systems through grounding, verification, and reasoning support, leading to more factual answers, explainable decisions, and consistent responses.

6.2.3 Using KGs to Improve the Reliability of AI Systems

Beyond improving KGs themselves, future research should also investigate how KGs can enhance the reliability of AI systems. Recent AI systems, especially LLM-based systems, have demonstrated strong generative and reasoning capabilities. However, they still suffer from factual hallucination, insufficient grounding, inconsistent reasoning, and difficulty in controlling or updating internal knowledge. These limitations suggest that symbolic knowledge structures such as KGs can serve as an important foundation for trustworthy AI systems.

KGs provide explicit, structured, and auditable representations of facts. As illustrated in Figure 6.4, a reliable KG can improve AI reliability through three main roles: grounding, verification, and reasoning support. First, KGs can ground AI outputs in verified knowledge rather than relying solely on opaque parametric knowledge. Second, they can verify generated claims against reliable facts, helping detect

factual inconsistencies and reduce hallucination. Third, they can support reasoning by providing structured context, relations, and evidence paths that make the reasoning process more transparent and interpretable.

Through these roles, KGs can guide retrieval-augmented generation, support factual consistency checking, improve explainability, and enable controllable knowledge updating through externally maintained facts. Ultimately, this can lead to more factual answers, explainable decisions, and consistent responses. This direction also creates a bidirectional relationship between KGs and AI systems: AI models can help validate, refine, and expand KGs, while KGs can help AI systems become more factual, interpretable, and controllable. This aligns with the broader objective of this dissertation: bridging structured knowledge and real-world facts to build more reliable knowledge-intensive AI systems.

References

- [1] Akiko Aizawa. An information-theoretic perspective of tf-idf measures. *Information Processing & Management*, 39(1):45–65, 2003.
- [2] David Arthur and Sergei Vassilvitskii. k-means++ the advantages of careful seeding. In *Proceedings of the eighteenth annual ACM-SIAM symposium on Discrete algorithms*, pages 1027–1035, 2007.
- [3] Jordan T Ash, Chicheng Zhang, Akshay Krishnamurthy, John Langford, and Alekh Agarwal. Deep batch active learning by diverse, uncertain gradient lower bounds. *arXiv preprint arXiv:1906.03671*, 2019.
- [4] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. *Advances in neural information processing systems*, 26, 2013.
- [5] Armand Boschini. Torchkg: Knowledge graph embedding in python and pytorch. *arXiv preprint arXiv:2009.02963*, 2020.
- [6] Jack Boylan, Shashank Mangla, Dominic Thorn, Demian Gholipour Ghandari, Parsa Ghaffari, and Chris Hokamp. Kgvalidator: a framework for automatic validation of knowledge graph construction. *arXiv preprint arXiv:2404.15923*, 2024.
- [7] Li Cai, Xin Mao, Yuhao Zhou, Zhaoguang Long, Changxu Wu, and Man Lan.

A survey on temporal knowledge graph: Representation learning and applications. *arXiv preprint arXiv:2403.04782*, 2024.

- [8] Hanzhu Chen, Xu Shen, Qitan Lv, Jie Wang, Xiaoqi Ni, and Jieping Ye. Sac-kg: Exploiting large language models as skilled automatic constructors for domain knowledge graph. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4345–4360, 2024.
- [9] Yingjian Chen, Haoran Liu, Yinhong Liu, Jinxiang Xie, Rui Yang, Han Yuan, Yanran Fu, Peng Yuan Zhou, Qingyu Chen, James Caverlee, et al. Graphcheck: Breaking long-term text barriers with extracted knowledge graph-powered fact-checking. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14976–14995, 2025.
- [10] Tim Dettmers, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. Convolutional 2d knowledge graph embeddings. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- [11] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- [12] Junnan Dong, Qinggang Zhang, Xiao Huang, Qiaoyu Tan, Daochen Zha, and

- Zhao Zihao. Active ensemble learning for knowledge graph error detection. In *Proceedings of the sixteenth ACM international conference on web search and data mining*, pages 877–885, 2023.
- [13] Takuma Ebisu and Ryutaro Ichise. Toruse: Knowledge graph embedding on a lie group. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- [14] Mohamed H Gad-Elrab, Daria Stepanova, Jacopo Urbani, and Gerhard Weikum. Exfakt: A framework for explaining facts over knowledge graphs and text. In *Proceedings of the twelfth ACM international conference on web search and data mining*, pages 87–95, 2019.
- [15] Mohamed H Gad-Elrab, Daria Stepanova, Jacopo Urbani, and Gerhard Weikum. Tracy: Tracing facts over knowledge graphs and text. In *The world wide web conference*, pages 3516–3520, 2019.
- [16] Luis Galárraga, Christina Teflioudi, Katja Hose, and Fabian M Suchanek. Fast rule mining in ontological knowledge bases with amie+. *The VLDB Journal*, 24(6):707–730, 2015.
- [17] Luis Antonio Galárraga, Christina Teflioudi, Katja Hose, and Fabian Suchanek. Amie: association rule mining under incomplete evidence in ontological knowledge bases. In *Proceedings of the 22nd international conference on World Wide Web*, pages 413–422, 2013.
- [18] Junyang Gao, Xian Li, Yifan Ethan Xu, Bunyamin Sisman, Xin Luna Dong,

- and Jun Yang. Efficient knowledge graph accuracy evaluation. *Proceedings of the VLDB Endowment*, 12(11):1679–1691, 2019.
- [19] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1762–1777, 2023.
- [20] Daniel Gerber, Diego Esteves, Jens Lehmann, Lorenz Bühmann, Ricardo Usbeck, Axel-Cyrille Ngonga Ngomo, and René Speck. Defacto—temporal and multilingual deep fact validation. *Journal of Web Semantics*, 35:85–101, 2015.
- [21] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [22] Saiping Guan, Xueqi Cheng, Long Bai, Fujun Zhang, Zixuan Li, Yutao Zeng, Xiaolong Jin, and Jiafeng Guo. What is event knowledge graph: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(7):7569–7589, 2022.
- [23] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *International conference on machine learning*, pages 3929–3938. PMLR, 2020.
- [24] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li,

- Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [25] Elwin Huaman and Dieter Fensel. Knowledge graph curation: a practical framework. In *Proceedings of the 10th International joint conference on knowledge graphs*, pages 166–171, 2021.
- [26] Haoyu Huang, Chong Chen, Zeang Sheng, Yang Li, and Wentao Zhang. Can llms be good graph judge for knowledge graph construction? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 10940–10959, 2025.
- [27] Jiacheng Huang, Zequn Sun, Qijin Chen, Xiaozhou Xu, Weijun Ren, and Wei Hu. Deep active alignment of knowledge graph entities and schemata. *Proceedings of the ACM on Management of Data*, 1(2):1–26, 2023.
- [28] Xiao Huang, Jingyuan Zhang, Dingcheng Li, and Ping Li. Knowledge graph embedding based question answering. In *Proceedings of the twelfth ACM international conference on web search and data mining*, pages 105–113, 2019.
- [29] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.
- [30] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118*, 2021.

- [31] Mohamad Yaser Jaradeh, Kuldeep Singh, Markus Stocker, Andreas Both, and Sören Auer. Information extraction pipelines for knowledge graphs. *Knowledge and Information Systems*, 65(5):1989–2016, 2023.
- [32] Shengbin Jia, Yang Xiang, Xiaojun Chen, and Kun Wang. Triple trustworthiness measurement for knowledge graph. In *The World Wide Web Conference*, pages 2865–2871, 2019.
- [33] Pengcheng Jiang, Jiacheng Lin, Zifeng Wang, Jimeng Sun, and Jiawei Han. Genres: Rethinking evaluation for generative relation extraction in the era of large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 2820–2837, 2024.
- [34] Zhijing Jin, Qipeng Guo, Xipeng Qiu, and Zheng Zhang. Genwiki: A dataset of 1.3 million content-sharing text and graphs for unsupervised graph-to-text generation. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 2398–2409, 2020.
- [35] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.
- [36] Jaehun Jung, Bokyung Son, and Sungwon Lyu. Attnio: Knowledge graph exploration with in-and-out attention flow for knowledge-grounded dialogue. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pages 3484–3497, 2020.

- [37] Jiho Kim, Sungjin Park, Yeonsu Kwon, Yohan Jo, James Thorne, and Edward Choi. Factkg: Fact verification via reasoning on knowledge graphs. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16190–16206, 2023.
- [38] Jiseong Kim and Key-sun Choi. Unsupervised fact checking by counterweighted positive and negative evidential paths in a knowledge graph. In *Proceedings of the 28th international conference on computational linguistics*, pages 1677–1686, 2020.
- [39] Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [40] Andreas Kirsch, Joost Van Amersfoort, and Yarin Gal. Batchbald: Efficient and diverse batch acquisition for deep bayesian active learning. *Advances in neural information processing systems*, 32, 2019.
- [41] Jonathan Lajus, Luis Galárraga, and Fabian Suchanek. Fast and exact rule mining with amie 3. In *European Semantic Web Conference*, pages 36–52. Springer, 2020.
- [42] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [43] Dawei Li, Bohan Jiang, Liangjie Huang, Alimohammad Beigi, Chengshuai

- Zhao, Zhen Tan, Amrita Bhattacharjee, Yuxuan Jiang, Canyu Chen, Tianhao Wu, et al. From generation to judgment: Opportunities and challenges of llm-as-a-judge. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 2757–2791, 2025.
- [44] Dawei Li, Zhen Tan, Tianlong Chen, and Huan Liu. Contextualization distillation from large language model for knowledge graph completion. In *Findings of the Association for Computational Linguistics: EACL 2024*, pages 458–477, 2024.
- [45] Jiao Li, Yueping Sun, Robin J Johnson, Daniela Sciaky, Chih-Hsuan Wei, Robert Leaman, Allan Peter Davis, Carolyn J Mattingly, Thomas C Wiegers, and Zhiyong Lu. Biocreative v cdr task corpus: a resource for chemical disease relation extraction. *Database*, 2016, 2016.
- [46] Muzhi Li, Cehao Yang, Chengjin Xu, Xuhui Jiang, Yiyan Qi, Jian Guo, Hongfung Leung, and Irwin King. Retrieval, reasoning, re-ranking: A context-enriched framework for knowledge graph completion. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 4349–4363, 2025.
- [47] Chin-Yew Lin. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pages 74–81, 2004.
- [48] Yankai Lin, Zhiyuan Liu, Maosong Sun, Yang Liu, and Xuan Zhu. Learning

- entity and relation embeddings for knowledge graph completion. In *Proceedings of the AAAI conference on artificial intelligence*, volume 29, 2015.
- [49] Ben Liu, Jihai Zhang, Fangquan Lin, Cheng Yang, and Min Peng. Filter-then-generate: Large language models with structure-text adapter for knowledge graph completion. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 11181–11195, 2025.
- [50] Bing Liu, Harris Scells, Guido Zuccon, Wen Hua, and Genghong Zhao. Activeea: Active learning for neural entity alignment. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3364–3374, 2021.
- [51] Xiangyu Liu, Yang Liu, and Wei Hu. Knowledge graph error detection with contrastive confidence adaption. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 8824–8831, 2024.
- [52] Xingyu Liu, Jielong Tang, Mengyang Li, Junmei Han, Gang Xiao, and Jianchun Jiang. Sesicl: Semantic and structural integrated contrastive learning for knowledge graph error detection. *IEEE Access*, 12:56088–56096, 2024.
- [53] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- [54] Yuxing Lu and Jinzhuo Wang. Karma: Leveraging multi-agent llms for automated knowledge graph enrichment. *arXiv preprint arXiv:2502.06472*, 2025.

- [55] Yi Luan, Luheng He, Mari Ostendorf, and Hannaneh Hajishirzi. Multi-task identification of entities, relations, and coreference for scientific knowledge graph construction. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3219–3232, 2018.
- [56] Xin Lv, Yankai Lin, Yixin Cao, Lei Hou, Juanzi Li, Zhiyuan Liu, Peng Li, and Jie Zhou. Do pre-trained models benefit knowledge graph completion? a reliable evaluation and a reasonable approach. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 3570–3581, 2022.
- [57] Chuangtao Ma, Yongrui Chen, Tianxing Wu, Arijit Khan, and Haofen Wang. Large language models meet knowledge graphs for question answering: Synthesis and opportunities. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 24589–24608, 2025.
- [58] Stefano Marchesin, Gianmaria Silvello, et al. Efficient and reliable estimation of knowledge graph accuracy. *Proceedings of the VLDB Endowment*, 17(9):2392–2403, 2024.
- [59] Belinda Mo, Kyssen Yu, Joshua Kazdan, Joan Cabezas, Proud Mpala, Lisa Yu, Chris Cundy, Charilaos Kanatsoulis, and Sanmi Koyejo. Kggen: Extracting knowledge graphs from plain text with language models. *arXiv preprint arXiv:2502.09956*, 2025.
- [60] Seungwhan Moon, Pararth Shah, Anuj Kumar, and Rajen Subba. Opendialkg: Explainable conversational reasoning with attention-based walks over knowl-

- edge graphs. In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 845–854, 2019.
- [61] Songjie Niu, Kaisen Yang, Rui Zhao, Yichao Liu, Zonglin Li, Hongning Wang, and Wenguang Chen. Tree-kg: An expandable knowledge graph construction framework for knowledge-intensive domains. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 18516–18529, 2025.
- [62] Prakhar Ojha and Partha Talukdar. Kgeval: Accuracy estimation of automatically constructed knowledge graphs. In *Proceedings of the 2017 conference on empirical methods in natural language processing*, pages 1741–1750, 2017.
- [63] Stefano Ortona, Venkata Vamsikrishna Meduri, and Paolo Papotti. Robust discovery of positive and negative rules in knowledge bases. In *2018 IEEE 34th International Conference on Data Engineering (ICDE)*, pages 1168–1179. IEEE, 2018.
- [64] Natalia Ostapuk, Jie Yang, and Philippe Cudré-Mauroux. Activelink: deep active learning for link prediction in knowledge graphs. In *The world wide web conference*, pages 1398–1408, 2019.
- [65] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pages 311–318, 2002.

- [66] Abhishek Pradhan, Ketan Kumar Todi, Anbarasan Selvarasu, and Atish Sanyal. Knowledge graph generation with deep active learning. In *2020 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2020.
- [67] Yifan Qi, Weiguo Zheng, Liang Hong, and Lei Zou. Evaluating knowledge graph accuracy powered by optimized human-machine collaboration. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*, pages 1368–1378, 2022.
- [68] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Brij B Gupta, Xiaojiang Chen, and Xin Wang. A survey of deep active learning. *ACM computing surveys (CSUR)*, 54(9):1–40, 2021.
- [69] Stephen Robertson, Hugo Zaragoza, et al. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389, 2009.
- [70] Devendra Singh Sachan, Mike Lewis, Dani Yogatama, Luke Zettlemoyer, Joelle Pineau, and Manzil Zaheer. Questions are all you need to train a dense passage retriever. *Transactions of the Association for Computational Linguistics*, 11:600–616, 2023.
- [71] Tara Safavi and Danai Koutra. Codex: A comprehensive knowledge graph completion benchmark. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 8328–8350, 2020.
- [72] Swarnadeep Saha, Prateek Yadav, Lisa Bauer, and Mohit Bansal. Explagraphs:

- An explanation graph generation task for structured commonsense reasoning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7716–7740, 2021.
- [73] Ozan Sener and Silvio Savarese. Active learning for convolutional neural networks: A core-set approach. *arXiv preprint arXiv:1708.00489*, 2017.
 - [74] Seungmin Seo, Byungkook Oh, Eunju Jo, Sanghak Lee, Dongho Lee, Kyong-Ho Lee, Donghoon Shin, and Yeonsoo Lee. Active learning for knowledge graph schema expansion. *IEEE Transactions on Knowledge and Data Engineering*, 34(12):5610–5620, 2021.
 - [75] Jianhao Shen, Chenguang Wang, Ye Yuan, Jiawei Han, Heng Ji, Koushik Sen, Ming Zhang, and Dawn Song. Palt: Parameter-lite transfer of language models for knowledge graph completion. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 3833–3847, 2022.
 - [76] Baoxu Shi and Tim Wener. Discriminative predicate path mining for fact checking in knowledge graphs. *Knowledge-based systems*, 104:123–133, 2016.
 - [77] Weijia Shi, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Richard James, Mike Lewis, Luke Zettlemoyer, and Wen-tau Yih. Replug: Retrieval-augmented black-box language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8364–8377, 2024.
 - [78] Prashant Shiralkar, Alessandro Flammini, Filippo Menczer, and Giovanni Luca

- Ciampaglia. Finding streams in knowledge graphs to support fact checking. In *2017 IEEE International Conference on Data Mining (ICDM)*, pages 859–864. IEEE, 2017.
- [79] Xuhui Sui, Ying Zhang, Yu Zhao, Baohang Zhou, and Xiaojie Yuan. Multi-modal knowledge graph error detection with disentanglement vae and multi-grained triplet confidence. In *Proceedings of the ACM on Web Conference 2025*, pages 3481–3491, 2025.
- [80] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. Rotate: Knowledge graph embedding by relational rotation in complex space. *arXiv preprint arXiv:1902.10197*, 2019.
- [81] Zafar Habeeb Syed, Michael Röder, and Axel-Cyrille Ngonga Ngomo. Unsupervised discovery of corroborative paths for fact validation. In *International Semantic Web Conference*, pages 630–646. Springer, 2019.
- [82] Zafar Habeeb Syed, Michael Röder, and Axel-Cyrille Ngonga Ngomo. Factcheck: Validating rdf triples using textual evidence. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*, pages 1599–1602, 2018.
- [83] Chen Tang, Hongbo Zhang, Tyler Loakman, Chenghua Lin, and Frank Guerin. Enhancing dialogue generation via dynamic graph knowledge aggregation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4604–4616, 2023.

- [84] Kristina Toutanova, Danqi Chen, Patrick Pantel, Hoifung Poon, Pallavi Choudhury, and Michael Gamon. Representing text for joint embedding of text and knowledge bases. In *Proceedings of the 2015 conference on empirical methods in natural language processing*, pages 1499–1509, 2015.
- [85] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [86] Cassia Trojahn, Renata Vieira, Daniela Schmidt, Adam Pease, and Giancarlo Guizzardi. Foundational ontologies meet ontology matching: A survey. *Semantic Web*, 13(4):685–704, 2022.
- [87] Théo Trouillon, Johannes Welbl, Sebastian Riedel, Éric Gaussier, and Guillaume Bouchard. Complex embeddings for simple link prediction. In *International conference on machine learning*, pages 2071–2080. PMLR, 2016.
- [88] Nikhita Vedula and Srinivasan Parthasarathy. Face-keg: Fact checking explained using knowledge graphs. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*, pages 526–534, 2021.
- [89] Xiang Wang, Xiangnan He, Yixin Cao, Meng Liu, and Tat-Seng Chua. Kgat: Knowledge graph attention network for recommendation. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 950–958, 2019.

- [90] Xiang Wang, Tinglin Huang, Dingxian Wang, Yancheng Yuan, Zhenguang Liu, Xiangnan He, and Tat-Seng Chua. Learning intents behind interactions with knowledge graph for recommendation. In *Proceedings of the web conference 2021*, pages 878–887, 2021.
- [91] Xiangyu Wang, Lyuzhou Chen, Taiyu Ban, Muhammad Usman, Yifeng Guan, Shikang Liu, Tianhao Wu, and Huanhuan Chen. Knowledge graph quality control: A survey. *Fundamental Research*, 1(5):607–626, 2021.
- [92] Yaqing Wang, Fenglong Ma, and Jing Gao. Efficient knowledge graph validation via cross-graph representation learning. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, pages 1595–1604, 2020.
- [93] Zhen Wang, Jianwen Zhang, Jianlin Feng, and Zheng Chen. Knowledge graph embedding by translating on hyperplanes. In *Proceedings of the AAAI conference on artificial intelligence*, volume 28, 2014.
- [94] Yanbin Wei, Qiushi Huang, Yu Zhang, and James Kwok. Kicgpt: Large language model with knowledge in context for knowledge graph completion. In *Findings of the association for computational linguistics: EMNLP 2023*, pages 8667–8683, 2023.
- [95] Wenhan Xiong, Mo Yu, Shiyu Chang, Xiaoxiao Guo, and William Yang Wang. One-shot relational learning for knowledge graphs. In *Proceedings of the 2018*

Conference on Empirical Methods in Natural Language Processing, pages 1980–1990, 2018.

- [96] Derong Xu, Ziheng Zhang, Zhenxi Lin, Xian Wu, Zhihong Zhu, Tong Xu, Xiangyu Zhao, Yefeng Zheng, and Enhong Chen. Multi-perspective improvement of knowledge graph completion with large language models. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 11956–11968, 2024.
- [97] Bingcong Xue and Lei Zou. Knowledge graph quality management: A comprehensive survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(5):4969–4988, 2022.
- [98] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [99] Bishan Yang, Wen-tau Yih, Xiaodong He, Jianfeng Gao, and Li Deng. Embedding entities and relations for learning and inference in knowledge bases. *arXiv preprint arXiv:1412.6575*, 2014.
- [100] Rui Yang, Jiahao Zhu, Jianping Man, Li Fang, and Yi Zhou. Enhancing text-based knowledge graph completion with zero-shot large language models: A focus on semantic enhancement. *Knowledge-Based Systems*, 300:112155, 2024.
- [101] Yuhao Yang, Chao Huang, Lianghao Xia, and Chenliang Li. Knowledge graph contrastive learning for recommendation. In *Proceedings of the 45th interna-*

- tional ACM SIGIR conference on research and development in information retrieval*, pages 1434–1443, 2022.
- [102] Liang Yao, Chengsheng Mao, and Yuan Luo. Kg-bert: Bert for knowledge graph completion. *arXiv preprint arXiv:1909.03193*, 2019.
- [103] Liang Yao, Jiazhen Peng, Chengsheng Mao, and Yuan Luo. Exploring large language models for knowledge graph completion. In *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5. IEEE, 2025.
- [104] Yuan Yao, Deming Ye, Peng Li, Xu Han, Yankai Lin, Zhenghao Liu, Zhiyuan Liu, Lixin Huang, Jie Zhou, and Maosong Sun. Docred: A large-scale document-level relation extraction dataset. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 764–777, 2019.
- [105] Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. Qa-gnn: Reasoning with language models and knowledge graphs for question answering. In *Proceedings of the 2021 conference of the North American chapter of the association for computational linguistics: human language technologies*, pages 535–546, 2021.
- [106] Hongbin Ye, Ningyu Zhang, Hui Chen, and Huajun Chen. Generative knowledge graph construction: A review. In *Proceedings of the 2022 conference on empirical methods in natural language processing*, pages 1–17, 2022.
- [107] Kaisheng Zeng, Chengjiang Li, Lei Hou, Juanzi Li, and Ling Feng. A com-

- prehensive survey of entity alignment for knowledge graphs. *AI Open*, 2:1–13, 2021.
- [108] Bowen Zhang and Harold Soh. Extract, define, canonicalize: An llm-based framework for knowledge graph construction. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9820–9836, 2024.
- [109] Qinggang Zhang, Junnan Dong, Keyu Duan, Xiao Huang, Yezi Liu, and Linchuan Xu. Contrastive knowledge graph error detection. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, pages 2590–2599, 2022.
- [110] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675*, 2019.
- [111] Yichi Zhang, Zhuo Chen, Lingbing Guo, Yajing Xu, Wen Zhang, and Hua-jun Chen. Making large language models perform better in knowledge graph completion. In *Proceedings of the 32nd ACM international conference on multimedia*, pages 233–242, 2024.
- [112] Zhao Zhang, Fuwei Zhang, Fuzhen Zhuang, and Yongjun Xu. Knowledge graph error detection with hierarchical path structure. In *Proceedings of the 32nd ACM international conference on information and knowledge management*, pages 4430–4434, 2023.

- [113] Lingfeng Zhong, Jia Wu, Qian Li, Hao Peng, and Xindong Wu. A comprehensive survey on automatic knowledge graph construction. *ACM Computing Surveys*, 56(4):1–62, 2023.
- [114] Xiangru Zhu, Zhixu Li, Xiaodan Wang, Xueyao Jiang, Penglei Sun, Xuwu Wang, Yanghua Xiao, and Nicholas Jing Yuan. Multi-modal knowledge graph construction and application: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 36(2):715–735, 2022.

Abstract in Korean

지식그래프와 실세계 사실 간의 간극 완화를 위한 연구

지식그래프(Knowledge Graph, KG)는 개체와 이들 간의 의미적 관계를 관계형 트리플의 그래프 구조로 표현함으로써 현실 세계의 지식을 구조화한다. 지식그래프는 질의응답, 추천, 대화형 에이전트, 팩트체크 등 다양한 응용 분야를 지원하는 핵심 지식 인프라로 활용되고 있다. 그러나 이러한 광범위한 활용 가능성에도 불구하고, 실제 환경의 지식그래프에는 현실 세계의 사실과 일치하지 않는 정보가 불가피하게 존재한다. 이러한 불일치는 자동화된 지식그래프 구축 과정에서 발생하는 오류, 지식그래프 통합 과정의 실패, 그리고 과거에는 정확했으나 현실 세계의 변화로 인해 더 이상 유효하지 않게 된 트리플 등 다양한 원인에서 발생한다. 이러한 부정확성은 지식그래프 기반 응용 시스템의 신뢰성을 저해하므로, 현실 세계의 사실과의 정합성을 확보하는 것은 신뢰할 수 있는 지식 기반 시스템을 구축하기 위한 핵심 요구사항이다.

지식그래프와 현실 세계의 사실 간 격차를 해소하기 위해서는 지식그래프 유지 관리를 지속적인 과정으로 바라볼 필요가 있으며, 이는 상호 보완적인 세 단계, 즉 주의가 필요한 트리플의 선별, 사실적 정확성의 검증, 그리고 오류가 있거나 부적절하게 표현된 트리플의 정제로 구성될 수 있다. 그러나 각 단계에는 서로 다른 도전 과제가 존재한다. 첫째, 대규모 지식그래프에는 정확한 사실과 오류가 포함된 사실이 혼재된 수많은 트리플이 존재하므로, 모든 트리플을 사람이 직접 검증하는 것은 현실적으로 불가능하다. 둘째, 희소하거나 모호하며 기존에 관찰되지 않은

사실의 경우, 지식그래프 내부의 구조적 신호나 모델의 파라미터 지식만으로는 신뢰할 수 있는 검증이 어려우므로 외부 근거와 이를 활용한 추론이 필요하다. 셋째, 문제가 있는 일부 트리플은 부분적으로 정확한 정보를 포함하고 있어 삭제하는 대신 수정하거나 다시 표현할 수 있으므로, 지식그래프 정제를 단순한 삭제 문제로 한정해서는 안 된다.

본 학위논문은 이러한 문제를 해결하기 위해 지식그래프 정확도 향상, 지식그래프 검증, 그리고 지식그래프 정제의 세 가지 상호 연계된 연구 방향을 제시한다. 첫째, 지식그래프 정확도 향상을 위한 능동 학습 프레임워크인 AIKA를 제안한다. AIKA는 제한된 주석 예산하에서 지식그래프 임베딩 모델을 반복적으로 학습하고, 트리플의 점수를 산출한 뒤, 오류 가능성이 높은 트리플을 선별하여 사람이 직접 교정하도록 한다. AIKA는 불확실성 기반 선별과 균형 및 군집 기반 샘플링을 결합함으로써 빈도가 높은 관계에 대한 편향을 완화하는 동시에 지식그래프의 정확도를 효율적으로 향상시킨다.

둘째, 근거 기반 추론을 수행하는 검색 증강 지식그래프 검증 프레임워크인 RAVER를 제안한다. RAVER는 검증 대상 트리플과 관련된 외부 근거를 검색하고, 검색된 문서를 기반으로 대규모 언어 모델(Large Language Model, LLM)이 해당 트리플의 사실적 정확성을 판단하도록 한다. 또한 근거 검색의 품질을 향상시키기 위해 검색기를 LLM이 평가한 근거의 유용성과 정렬한다. 이를 통해 단순히 주제적으로 유사한 문서가 아니라, 후속 검증 과정에서 실질적으로 유용한 문서를 우선적으로 검색할 수 있도록 한다.

셋째, 생성형 지식 추출을 통해 구축된 지식그래프를 위한 연산 기반 정제 프레임워크인 GraphRefine을 제안한다. GraphRefine은 LLM 기반 정제 모델을 활용하여 생성된 트리플을 원본 문서와 비교하여 검토하고, 각 트리플에 명시적인 연산을

할당한 뒤 이에 따라 지식그래프를 수정한다. 이를 통해 근거가 없는 트리플을 제거하고, 잘못된 구성 요소를 수정하며, 모호하거나 지나치게 장황한 표현을 다시 작성하는 동시에 유효한 정보를 보존할 수 있다.

다양한 벤치마크를 대상으로 수행한 광범위한 실험을 통해, 본 논문에서 제안한 방법들이 서로 다른 데이터셋, 모델 구조 및 평가 환경에서 지식그래프의 정확도, 검증 신뢰도, 그리고 정제 품질을 일관되게 향상시킴을 확인하였다. 또한 실험 결과는 사실적 일관성을 효과적으로 유지하기 위해서는 오류가 있는 트리플을 단순히 탐지하는 것뿐만 아니라, 어떤 사실을 우선적으로 검토해야 하는지를 결정하고, 신뢰할 수 있는 외부 근거에 기반하여 사실 여부를 검증하며, 오류의 유형에 따라 적절한 정제 전략을 적용하는 과정이 함께 고려되어야 함을 보여준다.

종합하면, 본 논문은 의심스러운 트리플의 효율적인 선택, 외부 근거에 기반한 사실 검증, 그리고 연산 기반 정제를 통해 지식그래프와 현실 세계의 사실 간 격차를 줄이기 위한 통합적 관점을 제시한다. 이러한 상호보완적 단계를 연결함으로써, 본 논문은 지속적인 지식그래프 유지관리를 위한 체계적 기반을 마련하고, 구조화된 지식, 외부 근거, 그리고 대규모 언어 모델 기반 추론을 결합하는 것의 중요성을 제시한다. 나아가 이러한 관점은 멀티모달 근거의 통합, 자율적인 에이전트 기반의 지식그래프 유지관리, 변화하는 현실 세계의 지식에 대한 지속적인 적응, 그리고 정확하고 설명 가능하며 검증 가능한 구조화된 지식에 기반한 신뢰할 수 있는 인공지능 시스템으로 연구를 확장할 수 있는 가능성을 제시한다.

핵심되는 말 : 지식그래프 정확도, 지식그래프 검증, 지식그래프 정제, 능동 학습, 검색 증강, 대규모 언어 모델